

Combinatorial Optimization

Maciej Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

- 1 Introduction
- 2 Complexity theory framework for CO problems analysis
 - 2.1 NP-hardness
 - 2.2 Problem relaxation
 - 2.3 Approximation algorithms
 - 2.4 Hardness of approximation
- 3 Solving CO problems in practice
- 4 Greedy algorithms and matroids
- 5 Graph problems and algorithms
 - 5.1 Shortest paths
 - 5.2 Binary relation transitive closure
 - 5.3 Network flows
 - 5.4 Network flows with lower bounds on arc flows
 - 5.5 Network flows applications
 - 5.6 Minimum cost network flows
 - 5.7 Matching in bipartite graphs
 - 5.8 Graph coloring
- 6 Packing and cutting¹

See also slides in Polish which have additional content.

¹ As much as time allows.

- 1 W.Lipski, Kombinatoryka dla programistów, WNT, W-wa, 1982;
- 2 M.R.Garey, D.S.Johnson, Computers and Intractability: A Guide to the Theory of NP-completeness, W. H. Freeman, San Francisco, 1979;
- 3 J.Błazewicz, W.Cellary, R.Słowiński, J.Węglarz, Badania operacyjne dla informatyków, WNT, W-wa, 1983;
- 4 J.Błazewicz, Złożoność obliczeniowa problemów kombinatorycznych, WNT, W-wa, 1988; also Wydawnictwo PP 1193, 1984;
- 5 A.V.Aho, J.E.Hopcroft, J.D.Ullman, Projektowanie i analiza algorytmów komputerowych, PWN, W-wa, 1983 (+new edition);
- 6 R.Ahuja, T.Magnati, J.Orlin, Network Flows, Theory, Algorithms, and Applications, Prentice Hall, Englewood Cliffs, 1993;
- 7 M.Sysło, N.Deo, J.Kowalik, Algorytmy optymalizacji dyskretniej z programami w języku Pascal, PWN, W-wa, 1993;
- 8 K.Manuszewski, Grafy Algorytmicznie trudne do kolorowania, praca doktorska, WETI, Gdańsk, 1997;
- 9 W.Cook, W.Cunningham, W.Pulleyblank, A.Schrijver, Combinatorial optimization, J.Wiley & Sons, 1998;
- 10 T.Cormen, C.Leiserson, R.Rivest, C.Stein, Wprowadzenie do algorytmów, Wydawnictwo Naukowe PWN, Warszawa, 1997; *Introduction to Algorithms, MIT Press, 1990 (+ later editions)*
- 11 R.Burkard, M.Dell'Amico, S.Martello, Assignment Problems, Society of Industrial and Applied Mathematics, Philadelphia, 2009
- 12 W.Kordecki, A.Łyczkowska-Hanćkowiak, Matematyka dyskretna dla informatyków, Helion, Gliwice, 2018.
- 13 L.Banachowski, A.Kreczmar, Elementy analizy algorytmów, WNT, W-wa, 1989;
- 14 E.Lawler, Combinatorial Optimization: Networks and Matroids, Holt, Rinehart and Winston, New York, 1976;
- 15 J.Błazewicz, K.Ecker, E.Pesch, G.Schmidt, M.Sterna, J.Węglarz, Handbook on Scheduling. From Theory to Practice, Springer Verlag, Berlin, New York, 2019 (+earlier editions);

See also <http://www.cs.put.poznan.pl/mdrozdowski/dyd/mat.html>

Also Internet sources, Wikipedia (with caution)

Requirements: Algorithms and data structures, introduction to algorithms, discrete mathematics, complexity theory, NP-completeness

Assessment: written exam².

CO can be found also on other courses: → operations research: metaheuristics, enumerative (exact) algorithms, e.g. B&B, linear programming.

Found an error? Inform me please! ;-)

see google: "wtf man i just wanted to learn how to program video games"

²subject to COVID-19 regulations, if any

Complexity Theory Framework for the Analysis of Combinatorial Optimization Problems

M.Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

What will be subject of this lecture now and why?

We are interested in the *methodology* (or a framework) which defines how to deal with CO problems. Our particular targets are:

- complexity of combinatorial search and optimization,
- *approximation* in polynomial time with any solution quality guarantees,

and then to:

- to delineate the *border* between easy and hard problems,
- to delineate the *border* between problems which can, or cannot be approximated with a guarantee.

For these purposes we will need new notions and definitions ...

NP-hardness

In this section we will analyze equivalence of decision and optimization problems with respect to their hard/easy solvability.

Def. Search problem:

A search problem Π consists of a set D_Π of finite instances and for each instance $I \in D_\Pi$ a set $Z_\Pi(I)$ of finite solutions.

Some algorithm A *solves* search problem Π if for each instance $I \in D_\Pi$: returns answer "NO"³ if $Z_\Pi(I) = \emptyset$, otherwise A provides some solution $z \in Z_\Pi(I)$.

Any decision problem Π can be rendered as a search problem by defining $Z_\Pi(I) = \{"YES"\}$ if $I \in Y_\Pi$, otherwise $Z_\Pi(I) = \emptyset$.

$\Rightarrow$ decision problems are special cases of search problems

³ it can be any form of a signal agreed upon by convention

In search problems different objects than "yes" / "no" answers are returned as solutions.

Hence, we cannot use polynomial transformation to show equivalence of search problems because polynomial transformation preserves polynomial time "yes" / "no" answer(ability).

In order to show equivalence of some search problems with respect to their computational complexity, we will use polynomial time Turing reduction (polynomial Turing reduction for short).

Def. Polynomial Turing reduction from problem Π_1 to problem Π_2 , denoted $\Pi_1 \propto_T \Pi_2$, is an algorithm A that solves Π_1 by using some hypothetical subroutine P for solving Π_2 , so that A is polynomial time provided that P runs on DTM in polynomial time.

It will be said that Π_1 is T -reducible to Π_2 .

Let us return to some facts about the mutual relationships between optimization problems and their decision versions⁴:

- Every optimization problem Π_1 has its decision version Π_2 with threshold value y of the objective function
- The decision version Π_1 can be solved by the same procedure P as the optimization version Π_2 , plus it is necessary to compare the optimum value of the objective y^* with the threshold y .
- If P can be run in polynomial time, then also the decision version Π_1 can be solved in polynomial time.
- Hence: Can the decision version Π_1 be harder⁵ Answer: NO!
Decision version Π_1 can't be harder.
- Decision version Π_1 is not harder than optimization version ($\leq$).

⁴ this should have been mentioned in "algorithms and data structures" lecture.

⁵ We are interested only in distinguishing between easy (polynomial) vs hard (not necessarily polynomial), more detailed representation of complexity (e.g. polynomial degree, coefficients and constants) is immaterial.

Further conclusions:

- So if decision version $\Pi_1 \in NP_c$ ⁶, then can the optimization version Π_2 be easy (i.e. polynomial)? Answer: No! (unless $P = NP$).
- Observe that by using a polynomial procedure P solving Π_2 to solve Π_1 we constructed a polynomial Turing reduction $\Pi_1 \propto_T \Pi_2$ from decision problem Π_1 to search/optimization problem Π_2 .
- Since $\Pi_1 \in NP_c$, problem Π_2 is at least as hard as whole class NP. Why? Because NP_c problems are the hardest in NP.

⁶ NP_c - NP-complete

Def. NP-hard problem

Some problem Π is NP-hard⁷ if there is a problem $\Pi' \in NP_c$ such that $\Pi' \propto_T \Pi$.

Observations:

- Decision problems are special case of search problems, search problems are special case of combinatorial optimization problems, so if the problem on the right-hand-side of $\propto_T$ is optimization, then in particular it can be also one of its simplified versions.
- Actually, Turing reduction can be conducted from an NP_c problem Π' to any type of problem Π (see definitions of $\propto_T$, NPh).
- Hence, not only optimization versions of some NP_c problem can be NPh, but any type of problem Π can be NP-hard (the problem on the right-hand-side of $\propto_T$).

⁷ Name NP-hard (NPh for short) stems from the fact that NPh problems are at least as hard as class NP. 

- Note that $\Pi' \propto_{\mathcal{T}} \Pi$ implies $\text{complexity}(\Pi') \leq \text{complexity}(\Pi)$.
(because Π' can be solved in poly.-time by a procedure for Π).
- Now we want to invert the direction of inequality $\rightarrow \geq$ in complexity between problems.
- This would be equivalent to proving that there can be some optimization problems which are not harder than their decision counterparts.
- To prove this we need an example of $\propto_{\mathcal{T}}$ going in the opposite direction. We'll do it on the example of Knapsack Problem (KP).

Theorem

Optimization Knapsack Problem (OKP) $\propto_T$ Decision Knapsack Problem (DKP).

Proof.

We will use an intermediate Decision Knapsack Problem Extension (DKPE) to simplify operation on the hypothetical P procedure solving (DKP).

DECISION KNAPSACK PROBLEM EXTENSION (DKPE)

Instance: A finite set $A = \{a_1, \dots, a_n\}$, sizes $s(a_i) \in \mathbb{N}^{+8}$ and values $w(a_i) \in \mathbb{N}$ for all elements $a_i \in A$, constants $b, y \in \mathbb{N}$ and subset $A_k \subset A$, $A_k = \{a_{[1]}, \dots, a_{[k]}\}$, where $a_{[i]} \neq a_{[j]}$ for $i \neq j$.

Question: $\exists A'$: $A_k \subset A' \subseteq A$ and $\sum_{a_i \in A'} s(a_i) \leq b$,
 $\sum_{a_i \in A'} w(a_i) \geq y$?

- Note that DKP is a decision problem.
- Positive answer and $A_k \subset A' \subseteq A$ imply that set A_k can be *extended* to $A' \subseteq A$.
- Note that: $\text{DKPE} \in \mathbf{NP}$, $\text{DKPE} \propto_{\mathcal{T}} \text{DKP}^9$. Hence, if we show that $\text{OKP} \propto_{\mathcal{T}} \text{DKPE}$, then also $\text{OKP} \propto_{\mathcal{T}} \text{DKP}$.

⁸ natural numbers without 0

⁹ Homework assignment: prove it

- To prove $\text{OKP} \propto_T \text{DKPE}$ we have to show that a hypothetical procedure solving DKPE can be used polynomial number of times to find:
 - a) optimum value y^* of the objective function
 - b) what set A' of elements provides thus optimum.
- Assume $P(A, \bar{s}, \bar{w}, A_k, b, y)$ is this hypothetical procedure solving DKPE in polynomial time for parameters: set of all elements A , vector of element sizes $\bar{s}$, vector of element values $\bar{w}$, subset to be extended A_k , size of the knapsack b , threshold of the objective function y .

- Optimum y^* can be determined by binary search and starting from each element $a_i \in A$ to be a set to extend.

$y^* = -\infty$;

foreach $a_i \in A$ **do**

$y_{min} = \min_{\ell} w(a_{\ell}); y_{max} = n \times \max_{\ell} \{w(a_{\ell})\};$

while $y_{max} - y_{min} > 1$ **do**¹⁰

$y_i := \lceil (y_{max} + y_{min})/2 \rceil$;

if $P(A, \bar{s}, \bar{w}, \{a_i\}, b, y_i)$ **then** $y_{min} := y_i$ **else** $y_{max} = y_i$;

if $y^* < y_{min}$ **then** $\{y^* = y_{min}; A_k = \{a_i\}\}$;

- To calculate y_i^* procedure $P(A, \bar{s}, \bar{w}, \{a_i\}, b, y_i)$ was called at most $n \lceil \log_2 y_{max} \rceil$ times. Hence, polynomial number of times.

¹⁰ Is 1 a good threshold? Does it hold for any optimization problem?

- Knowing y^* and the first element in set $A_k = \{a_\ell\}$, the complete set A^* of elements providing the optimum y^* can be found in the following procedure:

for each $a_i \in A - A_k$ **do**

if $P(A, \bar{s}, \bar{w}, A_k \cup \{a_i\}, b, y^*)$ **then** $\{A_k = A_k \cup \{a_i\}\}$

- The above procedure works because if $\sum_{a_j \in A_k} w(a_j) < y^*$ then there must be some element a_i such that $A_k \cup \{a_i\}$ can be extended to y^* .
- Set A^* of elements providing the optimum y^* is found in at most n calls of procedure P .
- Hence, OKP can be solved by at most $O(n(\log n + \max_\ell \{w_\ell\}))$ calls of procedure P and $\text{OKP} \propto_T \text{DKPE} \propto_T \text{DKP}$.

□

- Thus, we've shown that $OKP \propto_T DKP$ and consequently $complexity(OKP) \leq complexity(DKP)^{11}$
- Hence, OKP can be solved in polynomial time if $P=NP$.
- Hence, new complexity classes can be defined in relation to class NP, but also for other problems than the decision problems.

¹¹Again, only polynomial vs non-polynomial distinction matters, fine details of complexity functions are immaterial.

Def. NP-easy problem

Some search problem Π_1 is said to be NP-easy, if there is some problem $\Pi_2 \in NP$, such that $\Pi_1 \propto_T \Pi_2$.

Def. NP-equivalent problem

Search problems which are both NP-hard and NP-easy are called NP-equivalent.

- In a nutshell NP-hard – at least as difficult as NP, NP-easy - at most as hard as NP, NP-equivalent – the same complexity as NP.
- Conclusion: we can study complexity of search or optimization problems in relation to classes NP, NPC, which however, were defined for decision problems.

Question (homework):

Consider two possible resolutions 1) $P = NP$, 2) $P \neq NP$. What would be the complexity (polynomial vs non-polynomial) of a) NPh, b) NP-easy, c) NP-equivalent problems?

- Beyond NP-hardness also **strong NP-hardness** is used. Strong NP-hardness can be defined analogously to Turing reduction, but the hypothetical procedure P must preserve the conditions of *pseudopolynomial transformation*: reduction in pseudopolynomial time, instance size is not exponentially reduced, the biggest numerical value is not exponentially increased.

However, in many research papers authors show only that the decision version is strongly NP-complete (which sometimes is insufficient).

Quick summary

Π_1 – a decision problem, Π_2 – some problem, eg. search problem

def. $\Pi_1 \in NP_c$ & $\Pi_1 \propto_T \Pi_2 \Rightarrow \Pi_2 \in NPh$

observation: $\Pi_1 \propto_T \Pi_2 \Rightarrow \text{complexity}(\Pi_1) \leq \text{complexity}(\Pi_2)$

def. $\Pi_1 \in NP$ & $\Pi_1 \propto_T \Pi_2 \Rightarrow \Pi_2 \in NP - \text{easy}$

observation: $\Pi_1 \propto_T \Pi_2 \Rightarrow \text{complexity}(\Pi_1) \geq \text{complexity}(\Pi_2)$

Def. $\Pi_2 \in NPh$ & $\Pi_2 \in NP - \text{easy} \Rightarrow \Pi_2 \in NP - \text{equivalent}$

Problem relaxation

Why relaxing CO problems?¹²

Relaxation (i.e. simplifying) of some CO problem is a way to delineate the border between easy and hard versions of some problems.

Thus, the goal is to distinguish problem versions which are *still polynomial*-time solvable from the versions which are *already* NP-hard (not solvable in polynomial if $P \neq NP$).

- Polynomial time algorithms for the relaxed versions of some CO problems can be still practically usable.
- Solutions of relaxed versions can provide lower bounds for algorithms like Branch & Bound.
- Examples of relaxed versions of CO problems and algorithms for such simplified special cases will be presented.

¹²Problem is too hard – change the problem!

Example: Deterministic scheduling

Relaxation in deterministic scheduling may consist in:

- allowing for preemption, i.e. suspension of execution and continuation without cost possibly on a different machine,
- assuming equal processing times, e.g. unitary: $\forall j, p_j = 1$;
- assuming only special types of precedence constraint graphs, e.g. chains, trees, forests, ...

Consider relaxation $P||C_{max} \rightarrow P|pmtn|C_{max}$ which means allowing preemption.

- $P||C_{max}$, $P|pmtn|C_{max}$ are notations of deterministic scheduling problems in so-called $\alpha|\beta|\gamma$ notation.
- Idea of $\alpha|\beta|\gamma$ notation in short: α - scheduling environment (factory, computer), β - features of task set, γ - objective function. Problem $P|pmtn|$ denotes:
 - $\alpha = P$: scheduling on some number m of parallel and identical processors $P_1, \dots, P_m$;
 - $\beta = pmtn$ tasks are preemptive and beyond this assumptions on the tasks are standard: no precedence constraints, no deadlines, all tasks are ready to be executed at time 0, etc. ... Tasks have processing times $p_1, \dots, p_n$ (hence there are n tasks);
 - $\gamma = C_{max}$ - schedule length (makespan) is the objective function.

McNaughton's algorithm for $P|pmtn|C_{max}$ (1959)

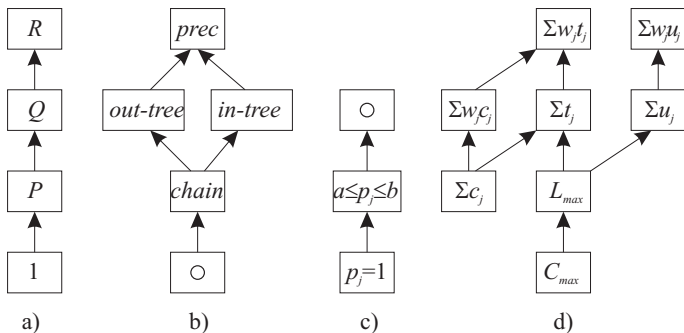
- 1 calculate optimum schedule length $C_{max}^* = \max\{\max_{j=1}^n \{p_j\}, \frac{1}{m} \sum_{j=1}^n p_j\}$
- 2 start constructing schedule on processor $i = 1$ at time $t = 0$;
- 3 **while** there are unscheduled tasks **do**
- 4 choose arbitrary unscheduled task T_j // e.g. with processing p_j
- 5 append T_j on the current processor at current time t ;
 if $t + p_j < C_{max}^*$, $\{ t = t + p_j; \}$ // T_j is scheduled
 else // this task would have to finish after C_{max}^* on the current processor
 $\{$ schedule piece of task T_j of length $C_{max}^* - t$ on processor i ;
 $i++$ //switch to processor $i + 1$;
 put on the new current processor the remaining piece of $p_j - (C_{max}^* - t)$ of T_j ;
 $t = p_j - (C_{max}^* - t); \}$
- 6 **enddo**

→ show an example

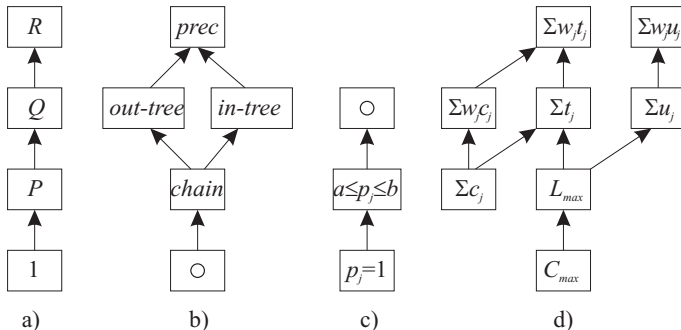
Homework: How to adjust the preemptive schedule to non-preemptive case? Can we prove anything about distance of such a solution from the optimum non-preemptive schedule?

Remark: McNaughton's algorithm will be used in the algorithm for $P|pmtn, r_j|L_{max}$, later in this lecture.

Example of complexity inference by relaxation for deterministic scheduling theory in $\alpha|\beta|\gamma$ framework.



- a) type of machine in α filed: P - identical, Q - uniform (different speeds), R - unrelated (a task has arbitrarily different processing time on any machine);
- b) type of precedence constraints between the tasks (in β filed): $\circ$ - no constraints, $in/out - tree$ - in/out-trees, $prec$ - arbitrary graph;
- c) processing time (in β filed): $p_j = 1$ - unit processing time, $a \leq p_j \leq b$ - bounded processing time, $\circ$ - arbitrary processing time;
- d) criteria in γ field: C_{max} - schedule length, L_{max} - maximum lateness, Σc_j - sum of completion times, $\Sigma w_j c_j$ - weighted sum of completion times, Σt_j - sum of tardiness, Σu_j - number of late tasks, $\Sigma w_j u_j$ - weighted number of late jobs, $\Sigma w_j t_j$ - weighted sum of tardiness.



If a problem represented by some box is **NPh**, then all its successors are also **NPh**, because they are more general cases. E.g. $P || C_{max} \in \mathbf{NPh} \Rightarrow Q || C_{max} \in \mathbf{NPh}$.

If a problem represented by some box is polynomial-time solvable (poly), then all its predecessors are also polynomial-time solvable, as special cases. E.g. $1 | prec | C_{max} \in \text{poly} \Rightarrow 1 | chain | C_{max} \in \text{poly}$.

Example: Knapsack problem (optimization version)

- Data of knapsack problem: w_i - values, s_i - sizes, n - number of elements, b - knapsack size. Request:

$$\max\{\sum_{i=1}^n w_i x_i : \sum_{i=1}^n s_i x_i \leq b, x_i \in \{0, 1\}, \text{ for } i = 1, \dots, n\}$$

- A relaxed version of knapsack problem allows divisibility of elements, i.e. only some part of an element can be put in the knapsack. Request:

$$\max\{\sum_{i=1}^n w_i x_i : \sum_{i=1}^n s_i x_i \leq b, x_i \in [0, 1], \text{ for } i = 1, \dots, n\}$$

- The relaxed version of knapsack is called *Continuous Knapsack Problem* (CKP).
- Observe that in the optimum solution of CKP elements are chosen to knapsack in the order of non-increasing ratios w_i/s_i until exhausting elements or free space in the knapsack.¹³

¹³ Homework: prove that this order is optimum. Hint: consider comparing and exchanging pairs of elements in and out of knapsack.

Algorithm for continuous knapsack problem [Dantzig, 1957]

- 1 sort elements decreasingly by w_i/s_i ; $i = 0$;
- 2 **while** $(\sum_{j=1}^i s_j < b) \ \&\& \ (i < n)$ **do** $i = i + 1$;
 if $(\sum_{j=1}^i s_j < b)$ {put whole element i into the knapsack}
 else {put into the knapsack part $\sum_{j=1}^i s_j - b$ of element i }
 enddo

Complexity: $O(n \log n)$

- This algorithm may be improved to run in $O(n)$ time¹⁴
 [Balas,Zemel, 1980].

¹⁴Homework: How to do it? Hint: similarly to searching for median in a set of numbers in $O(n)$ time, or like separating subsets in the quicksort. Key question: what is the key to separate subsets of elements?

- Computational experiments for the standard knapsack problem show that elements with the largest w_i/s_i are always inserted into the knapsack, elements with the smallest w_i/s_i are always omitted, and the whole problem revolves around a small subset of elements near the pivotal element inserted only fractionally.¹⁵

Approximation algorithms

Definition

Combinatorial optimization problem Π is defined by:

- set D_Π of finite instances;
- $\forall I \in D_\Pi$, set $Z_\Pi(I)$ of finite solutions;
- $\forall I \in D_\Pi, \forall z \in Z_\Pi(I)$ value $f_\Pi(I, z)$ of solution z for instance I .

- Function $f_\Pi(I, z)$ is called objective, objective function, goal function, quality measure.
- We will be assuming that values of $f() \in \mathbb{Q}$, i.e. are rational numbers (quotients).¹⁶

¹⁶Some sources assume $f \in \mathbb{R}$. However, consider what practical problems emerge if reals $\mathbb{R}$ are to be represented in computers.

Definition

If Π is minimization problem (respectively maximization problem), then **optimum solution** for instance $I \in D_\Pi$ is a solution $z^* \in Z_\Pi(I)$ such that $\forall z \in Z_\Pi(I), f_\Pi(I, z^*) \leq f_\Pi(I, z)$ (for maximization: $f_\Pi(I, z^*) \geq f_\Pi(I, z)$).
Value of $f_\Pi(I, z^*)$ will be denoted $OPT(I)$ in short.

Definition

*Algorithm A will be called a **heuristic** for problem Π if for each instance $I \in D_\Pi$ it delivers a solution $z \in Z_\Pi(I)$.*

Value of $f_\Pi(I, z)$ will be denoted $A(I)$ in short.

- Alternative names: approximate algorithm, suboptimum algorithm, heuristic algorithm.
- Currently term *approximate algorithm* is used mainly in reference to algorithms with some provable quality guarantee.

Definition

*Algorithm A is called an **optimization** algorithm if $\forall I, A(I) = OPT(I)$.*

- Note that according to the current state of knowledge (unless $P = NP$) no polynomial algorithm exists for hard CO problems. Hence, optimization algorithms run in exponential time.
- An alternative name for optimization algorithms for hard CO problems is *exact* algorithm.

- A trade-off between solution quality and algorithm runtime exists.¹⁷
- Considering the fact that heuristics abandon guarantees of optimum solutions, it makes no sense to abandon also polynomial runtime requirement.
- Hence, a conventional assumption is that only for polynomial heuristics can any quality guarantees make sense.
- Then heuristics are assessed with respect to the quality of the delivered solutions, by applying:
 - worst case analysis (analytical approach),
 - probabilistic quality assessment (e.g. average distance from the optimum, probabilistic guarantees of convergence to some range of distance from the optimum, analytical approach),
 - experimental assessment (empirical approach).

¹⁷ This will be discussed in the later sections.

Let Π be minimization (resp. maximization) problem and let A be a heuristic solving Π . For all $I \in D_\Pi$ we define fractions:¹⁸

$$S_A(I) = \frac{A(I)}{OPT(I)} \quad (\text{resp. for maximization: } S_A(I) = \frac{OPT(I)}{A(I)})$$

Definition

Absolute worst case performance ratio¹⁹ S_A is defined as:

$$S_A = \inf\{r \geq 1 : \forall I \in D_\Pi, S_A(I) \leq r\}$$

¹⁸ what is the range of $S_A(I)$?

¹⁹ Performance ratio for short. Why is this ratio *worst case*?

Definition

Asymptotic worst case performance ratio²⁰ S_A^∞ is defined as:

$$S_A^\infty = \inf\{r \geq 1 : \exists \mathbb{Z}, \forall I \in D_\Pi \& OPT(I) \geq \mathbb{Z}, S_A(I) \leq r\}$$

- What values of S_A, S_A^∞ are good?
- What is the relationship between S_A and S_A^∞ ? Which is bigger S_A or S_A^∞ .
- When ratios S_A, S_A^∞ are exactly achieved for some instance I then they are said to be *tight*.
- What is the advantage of showing that there is an instance I for which ratios S_A, S_A^∞ are exactly achieved (i.e. are tight)?

Example: performance guarantees of list algorithms for scheduling problem $P||C_{max}$

- Respective fields in notation $P||C_{max}$ mean:
 - $P \rightarrow$ parallel identical processors $P_1, \dots, P_m$ (m items),
 - $|| \rightarrow \mathcal{T} = \{T_1, \dots, T_n\}$ – set of n nonpreemptable tasks with standard features: processing times p_j , ready to be executed at time 0, no deadlines, no special resources needed, no precedence constraints, etc.
 - $C_{max} \rightarrow$ schedule length is the optimality criterion.

List algorithm for $P||C_{max}$ – put tasks on a list according to some order, assign tasks from the list sequentially to free processors.

→ example

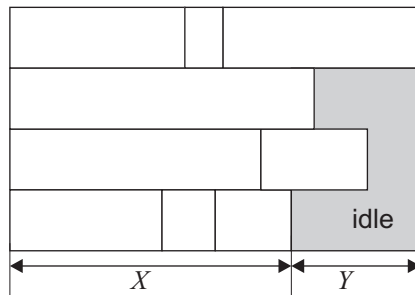
Theorem

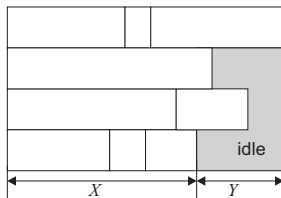
Performance ratio of list algorithm for $P||C_{max}$ is

$$S_{list_algo.} = 2 - \frac{1}{m}$$

Proof.

Every schedule built by the list algorithm has two parts: 1st part of length X where all processors are busy and 2nd part of length Y at the end of the schedule where some processors are idle because there are no more tasks to be assigned.





Since $\sum_{j=1}^n p_j$ is the whole amount of work $\frac{\sum_{j=1}^n p_j}{m}$ is the maximum length of a schedule in which all processors work, but from the total work $\sum_{j=1}^n p_j$ it is necessary to subtract the minimum work which will be done in the 2nd part of length Y .

$$\frac{\sum_{j=1}^n p_j}{m} \leq OPT \quad \text{Because schedule cannot be shorter than when all processors work in parallel.}$$

$$Y \leq \max\{p_j\} \leq OPT \quad \text{Because 2nd part cannot be longer than any single task, and the schedule cannot be shorter than any single task.}$$

$$S_{alg.list} \leq \max_{I \in D_\pi} \frac{X+Y}{OPT} \leq \frac{\frac{\sum_{j=1}^n p_j - Y}{m} + Y}{OPT} = \frac{\frac{\sum_{j=1}^n p_j}{m} + Y(1 - \frac{1}{m})}{OPT} \leq \frac{OPT + OPT(1 - \frac{1}{m})}{OPT} = 2 - \frac{1}{m}$$

□

Here we show that $S_{alg.list} = 2 - 1/m$ is indeed achieved by some instance. Thus, this ratio is *tight*.

Consider an instance:

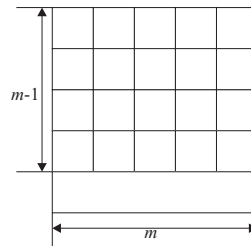
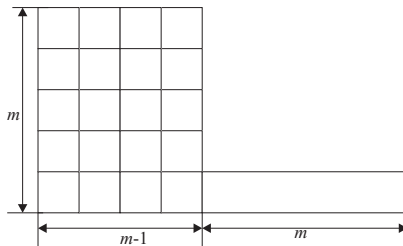
m - number of processors, $n = (m-1)m + 1$ - number of tasks, $p_1 = \dots = p_{m(m-1)} = 1, p_n = m$.

Assume that tasks are assigned to the processors in the order of task indices (left picture).

Then list schedule has length $C_{max}^{alg.list} = m - 1 + m$.

Optimum schedule (right picture) length is $OPT = m$.

Hence $\frac{C_{max}^{alg.list}}{OPT} = \frac{m-1+m}{m} = 2 - \frac{1}{m}$.



Since there is an instance for which the ratio is exactly achieved we can write "=" a not only " $\leq$ ". Thus,

$$S_{alg.list} = 2 - \frac{1}{m}.$$

Longest Processing Time (LPT) Algorithm for $P||C_{max}$

order the tasks according to the nonincreasing order of processing times p_j , assign tasks from this list sequentially to free processors.

Theorem (Graham 69)

LPT performance ratio for $P||C_{max}$ is

$$S_{LPT} = \frac{4}{3} - \frac{1}{3m}$$

It will be only shown that an instance for which ratio $S_{LPT} = \frac{4}{3} - \frac{1}{3m}$ is achieved exists.
Hence, this ratio cannot be smaller.

Consider an instance:

m - processor number, $n = 2m + 1$ - task number,

$(p_1, \dots, p_n) = (2m - 1, 2m - 1, 2m - 2, 2m - 2, \dots, m + 1, m + 1, m, m, m)$

which means that we have pairs of tasks of equal processing time, tasks in each next pair have processing times shorter by 1, the last 3 tasks have length m .

LPT

2m-1		m	m	
2m-1		m		przestój (ang.: idle)
2m-2		m+1		
2m-2		m+1		
2m-m/2		2m-m/2-1		
2m-m/2		2m-m/2-1		
				3m-1 4m-1

OPT

2m-1	m+1
2m-1	m+1
2m-2	m+2
2m-2	m+2
2m-m/2	2m-m/2
m	m
	m

3m

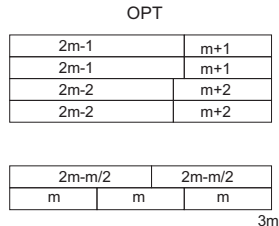
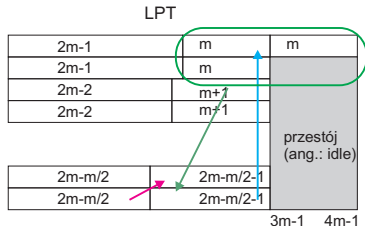
It will be only shown that an instance for which ratio $S_{LPT} = \frac{4}{3} - \frac{1}{3m}$ is achieved exists.
Hence, this ratio cannot be smaller.

Consider an instance:

m - processor number, $n = 2m + 1$ - task number,

$(p_1, \dots, p_n) = (2m - 1, 2m - 1, 2m - 2, 2m - 2, \dots, m + 1, m + 1, m, m, m)$

which means that we have pairs of tasks of equal processing time, tasks in each next pair have processing times shorter by 1, the last 3 tasks have length m .



Arrows on the figure show task moves necessary to change the LPT schedule to the optimum schedule.

Online scheduling

- Suppose complete information about the tasks is not initially available, but tasks and their data are revealed by an *adversary* one by one.²¹
- Some algorithm is said to be applied *online* when it is executed only with this partial information about the tasks.
- Can LPT be applied online? Can list algorithm be applied online?
- The two ratios of LPT and list algorithm are different.
What does it tell us about the power of the adversary?
- Can you imagine extending the "online" idea to other CO problems? Knapsack? TSP? Graph coloring?

²¹BTW, there are various models of the process of revealing this information sequentially and partially, differing in what information is revealed and when.

Example: performance ratio for vertex cover.

• VERTEX COVER PROBLEM

Data: Graph $G(V, E)$. Request: Find the smallest set $V' \subseteq V$ such that $\forall (i,j) \in E \ i \in V' \vee j \in V'$. (In other words, each edge $(i,j) \in E$ has at least one end covered by a vertex in V' .)

→show an example

Algorytm VC

- 1 $V' = \emptyset; E' = E;$
- 2 **while** $E' \neq \emptyset$
- 3 choose any edge $\{i,j\} \in E'$; //let's denote by X the set of chosen edges
- 4 $V' = V' \cup \{i\} \cup \{j\};$
- 5 remove from E' all edges incident with $z \in \{i,j\};$
- 6 **endwhile.**

→show an example

Theorem

Performance ratio of algorithm VC is

$$S_{Alg.VC} \leq 2$$

Proof.

- Observe that vertices on the ends of the edges chosen to set X define vertex cover because every edge is covered when algorithm stops because $E' = \emptyset$ (i.e. algorithm is correct),
- Observe that edges in X are not incident because in line 5 all edges incident with the edge $\{i, j\}$ chosen to X are removed.
- Since no pair of edges in X is incident, optimum cover requires at least as many vertices as the number of edges in X , i.e. $|X| \leq OPT$.
- $|V'| = 2|X|$, because each time an edge is chosen to X two vertices are added to cover V' .
- Hence $|V'| \leq 2 * OPT$.

□

Example: performance ratios for Traveling Salesman Problem (TSP).

- TRAVELING SALESMAN PROBLEM (TSP)

Data: Directed graph $G(V, A)$ with weights (a.k.a. distances) $d_{ij} \in \mathbb{N}$ for each arc $(i, j) \in A$. Request: Find the shortest cycle C over all nodes of V .

- Remark 1: cycles C need not be Hamiltonian.
- Remark 2: for simplicity of presentation it is assumed that all pairs of nodes are connected, i.e. $\forall_{i,j} d_{i,j} < \infty$.

Were it otherwise, the shortest path between i, j can substitute for a missing arc (i, j) .

- **Def.** SYMMETRIC TSP: In symmetric TSP $\forall_{i,j} d_{ij} = d_{ji}$.
- That is pairs of arcs $(i, j), (j, i)$ can be treated like edges.

Algorithm 1

- 1 build Minimum Spanning Tree MST for $G(V, A)$,
- 2 traverse the MST using depth-first search (DFS) algorithm starting from any node.

Theorem

Performance ratio of Algorithm 1 for symmetric TSP is

$$S_{A1} \leq 2$$

Proof.

- Every cycle (also the optimum cycle for TSP) without one edge is a tree
- MST is the shortest tree among all trees spanning $G(V, A)$.
- Hence $d(MST) \leq OPT$, where $d(MST)$ – is the length of all edges in the MST
- Each edge in MST is traversed twice by DFS.



- An algorithm with better performance ratio can be constructed for TSP with triangle inequality.

Def. TSP WITH TRIANGLE INEQUALITY:

satisfies $\forall_{i \neq j \neq k} d_{ij} \leq d_{ik} + d_{kj}$

- Observation 1: It may be advantageous to take shortcuts.
- Observation 2: In algorithm 1, if we were to visit a vertex for the 2nd time by DFS, then take a shortcut to the next unvisited vertex. However, this alone does not improve performance ratio 2.

- Observation 3: Depending on the starting position in DFS and choices of the next vertex to visit we can take better and worse shortcuts.
- Observation 4: It must be guaranteed that we take only "good" shortcuts.
- Observation 5: If we take shortcuts in MST then each used edge is used once. Hence the traversed edges form Euler cycle.
- Observation 6: If the solution is supposed to be an Euler cycle, and MST edges will be traversed anyway, then the shortcuts in MST can be added only between MST nodes of odd degree to satisfy conditions of for Euler cycle existence.

Christofides Algorithm [Christofides 1976]

- 1 Build MST T
- 2 Build clique K on all odd-degree vertices of T
- 3 Build minimum weight perfect²² matching M in clique K
- 4 Add edges of M to T (edges in $E' = T \cup M$ create a multigraph).
- 5 Build an Eulerian cycle C in graph $G(V, E')$. If some vertices are repeated, then skip them by taking shortcuts.

Theorem

Performance ratio of Christofides algorithm for symmetric TSP with triangle inequality is

$$S_{\text{Christofides}} \leq 1.5$$

²²In perfect matching all nodes of graph are covered.

Proof.

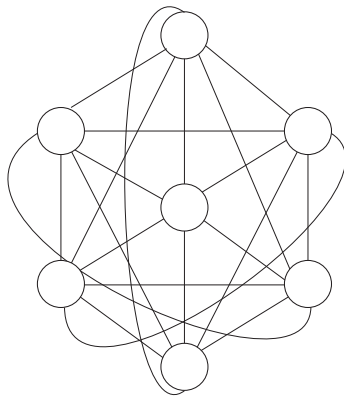
Let us start by showing that shortcuts in M connect all odd-degree vertices in the MST T and hence Eulerian cycle exists in $T \cup M$.

- Only odd-degree nodes of MST need shortcuts because of the necessary and sufficient condition for Eulerian cycle existence.
- Since MST is a tree the number of odd-degree vertices, that is vertices of clique K , is always even. (Homework: prove that the number of odd-degree vertices in any tree is even.)
- Since the number of vertices in K is even, perfect matching M in K always exists.
- Since M is perfect in K , and K is built on odd-degree nodes, then M adds 1 edge to each odd-degree node of K , and thus an Eulerian cycle can be built in $T \cup M$.

Now let's prove that the sum of used edge weights is at most $1.5 * OPT$.

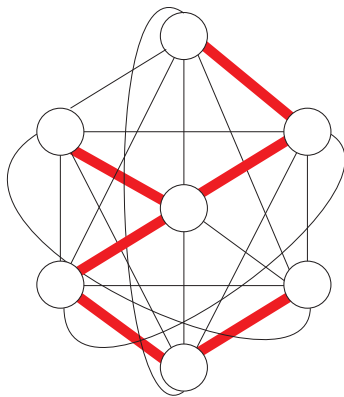
- The solution comprises edges of MST T and shortcut edges from matching M .
- The solution is an Eulerian cycle $\Rightarrow$ each edge is traversed once and its weight is counted once.
- Sum of edge weights in MST: $d(MST) \leq OPT$.
- The number of odd-degree vertices in MST T (that is vertices of clique K) is not greater than the number of vertices in the original graph $G(V, A)$, i.e. $|K| \leq |V|$.
- Since $|K| \leq |V|$ and triangle inequality is satisfied the length of the optimum solution for K alone cannot be longer than for the whole graph. Let us write it $OPT(K) \leq OPT$.
- Consider now the sequence of edges $e_1, e_2, e_3, \dots, e_{|K|}$ in the optimum TSP cycle only for clique K . The number of edges is even because $|K|$ is even. Both set $M_1 = \{e_1, e_3, e_5, \dots, e_{|K|-1}\}$, and set $M_2 = \{e_2, e_4, \dots, e_{|K|}\}$ are matchings.
- The sum of edge weights in one of the two matchings M_1, M_2 must be smaller than or equal to $0.5 * OPT(K)$ because M_1, M_2 together make length $OPT(K)$.
- In step 3 of the algorithm matching M is minimum weight in K , which means $d(M) \leq \min\{d(M_1), d(M_2)\} \leq 0.5 * OPT(K) \leq 0.5 * OPT$.
- Overall, the sum of edge weights in the solution satisfies: $d(T) + d(M) \leq OPT + 0.5 * OPT$. □

Example



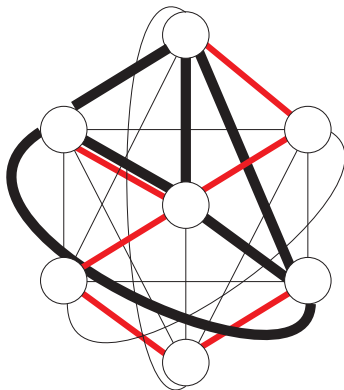
Initial graph $G(V, A)$

Example



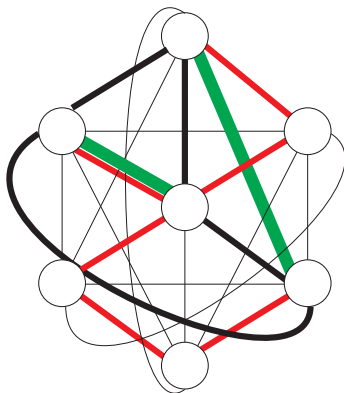
MST T

Example



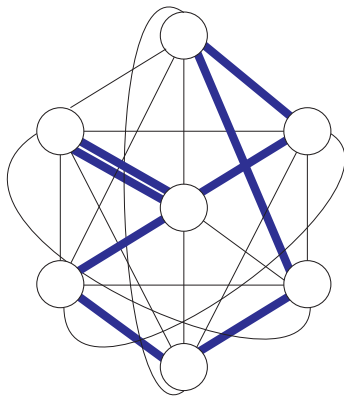
Clique K on MST T vertices of odd degree

Example



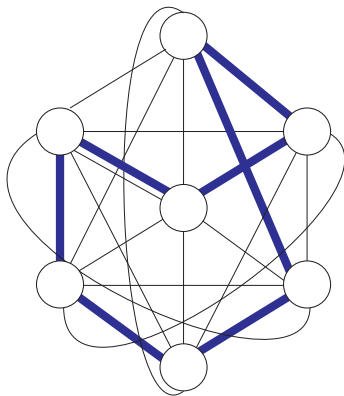
Minimum weight perfect matching in K

Example



Eulerian cycle in $T \cup M$

Example



And a shortcut for the end.

Definition

Approximation scheme for optimization problem Π is an algorithm A_ε that for a given instance $I \in D_\Pi$ and accuracy $\varepsilon > 0$ outputs solution $z \in Z_\Pi(I)$ such that $S_{A_\varepsilon} \leq 1 + \varepsilon$.

- Remark: the above definition does not say anything about runtime guarantees.

Definition

A_ε is a **polynomial time approximation scheme (PTAS)** if A_ε is an approximation scheme and for each fixed $\varepsilon > 0$, A_ε is a polynomial time algorithm.

- Important remark: ε in PTAS is a fixed constant, otherwise there need not be polynomial time guarantees.

Definition

*Approximation scheme A_ϵ is called a **fully polynomial time approximation scheme (FPTAS)** if A_ϵ runs in time bounded by a polynomial function of instance size $N(I)$ and $\frac{1}{\epsilon}$.*

Example: PTAS for knapsack

- Data: w_i - element value, s_i - element size, n - number of elements, b - knapsack size.

Algorithm AP k [Sahni 1975]

- 1 enumerate all k -element subsets of the set of all n elements;
- 2 extend each such subset by adding other elements in the order of nonincreasing ratios w_i/s_i until exhausting available knapsack space;
- 3 choose best subset.

Theorem

$$S_{APk} \leq 1 + \frac{1}{k}$$

- Complexity of APk is $O(n^k)$ which results from the number $\binom{n}{k}$ of subsets which must be enumerated.
- Assuming that $\frac{1}{k} = \varepsilon$, complexity of APk is $O(n^{\frac{1}{\varepsilon}})$, which is polynomial only if ε is fixed.

Example: FPTAS for knapsack**Algorithm B_ϵ** [Ibarra, Kim, 1975]

- 1 Transform input instance I into a new instance I' replacing element values by $w'_i = \lfloor \frac{w_i}{K} \rfloor$, where $K > 0$ // K will be specially chosen in the following
- 2 Solve I' by applying an alternative dynamic programming algorithm with complexity $O(nUB(I'))$, where $UB(I')$ is an upper bound on the value of the solution. // This algorithm is on the following slides
- 3 Assume the elements in set S' chosen for instance I' as the solution for the original instance I .

Theorem **B_ϵ** is an FPTAS.

Proof.

Computational complexity is $O(nUB(I'))$, where $UB(I')$ is an upper bound on the solution value. Assuming $W = \max_i \{w_i\}$, $K = \frac{W}{(\frac{1}{\varepsilon} + 1)n}$ we get $UB(I') = \frac{nW}{K}$, and complexity is $O(n^2 \frac{W}{K}) = O(n^3(1 + \frac{1}{\varepsilon}))$.

Quality of approximation.

Let us denote:

S' - optimum solution for instance I' from alternative dynamic programming algorithms,

S^* - optimum solution for I (the original instance).

$$\begin{aligned} B_\varepsilon(I) &= \sum_{i \in S'} w_i \geq \sum_{i \in S'} w'_i K = // \text{because we use values } w_i \text{ in the solution for } I, \text{ not the rounded values } w'_i \\ K \sum_{i \in S'} w'_i &\geq K \sum_{i \in S^*} w'_i = // \text{because } S' \text{ is an } \underline{\text{optimum}} \text{ solution for } I' \text{ while } S^* \text{ need not be optimum for } I' \\ \sum_{i \in S^*} K w'_i &\geq \sum_{i \in S^*} [w_i - K] \geq \\ \sum_{i \in S^*} w_i - nK &= OPT(I) - nK = OPT(I) - \frac{W}{(\frac{1}{\varepsilon} + 1)} \end{aligned}$$

Since $W \leq OPT(I)$ we get: $B_\varepsilon \geq OPT(I)(1 - \frac{1}{\frac{1}{\varepsilon} + 1}) = OPT(I) \frac{\varepsilon}{\varepsilon + 1}$, and from here $S_{B_\varepsilon}(I) = \frac{OPT(I)}{B_\varepsilon(I)} \leq$

$$\frac{OPT(I)}{OPT(I) \frac{\varepsilon}{\varepsilon + 1}} = \varepsilon + 1.$$

□

- Tabulate the function:²³

$g(i, j) =$ *minimum size of elements chosen from set*
 $\emptyset \cup \{1, \dots, i\}$ *which give gain equal to* j .

for $i = 0, \dots, n, j = 0, \dots, UB$, where UB is an upper bound on the value of knapsack, e.g. $UB = \sum_{i=1}^n w_i$.

- Function $g(i, j)$ can be calculated recursively:

$$g(0, 0) = 0; \quad g(i, 0) = 0 \text{ for } i = 1, \dots, n$$

$$g(0, j) = \infty \text{ for } j = 1, \dots, UB.$$

$$g(i, j) = \begin{cases} \min\{g(i-1, j-w_i) + s_i, g(i-1, j)\} & \iff w_i \leq j \\ g(i-1, j) & \text{otherwise} \end{cases}$$

- The maximum possible value of the knapsack (i.e. value of the optimum) is found in the table cell (n, j^*) for which
 $j^* = \max_j \{j : g(n, j) \leq b\}.$

²³We will refer to it as "the **second** method".

Example

$$\bar{s} = [5, 3, 2, 4, 3], \bar{w} = [3, 4, 2, 6, 1] \Rightarrow UB = 16. \quad b = 10$$

ij	1	2	3	4	5	6	7	8	9	10	11	12	13	...
1			5											
2			5	3			8							
3		2	5	3	7	5	8		10					
4		2	5	3	7	4	8	6	9	7		9		
5	3	2	5	3	6	4	7	6	9	7	10	9		

Column and row 0 were skipped for clarity. Empty cells contain ∞ .
 Entries with $g(i, j) > b$ are treated as ∞ as infeasible.

- Though decision versions of hard CO problems transform to each other (because they are all *NPC*), polynomial transformation does not preserve solution distance from the optimum. Hence, we cannot imply from the existence of the mutual polynomial time transformation any conclusion on performance ratios.
- Hence, there can be CO problems which
 - admit FPTAS/PTAS (superb!),
 - approximation algorithm with constant performance ratio (well, well, well),
 - but also problems without any algorithm with finite performance ratio i.e. $\neg \exists A : S_A < \infty$ (that's bad).
- We will discuss now limitations on the performance ratio guarantees which can be obtained in the best possible situation.

Definition

Best achievable asymptotic performance ratio for *combinatorial optimization problem* Π :

$$S_{\min}(\Pi) = \inf \left\{ r \geq 1 : \begin{array}{l} \exists \text{ polynomial approximation algorithm } A \\ \text{for } \Pi \text{ with } S_A^\infty = r \end{array} \right\}$$

- $S_{\min}(\Pi)$ is a measure of approximation feasibility for problem Π by any polynomial approximation algorithm.
- Hence, bounds on $S_{\min}(\Pi)$ tell us something about the problem rather than on the algorithms.

- Different degrees of NPh problem Π approximability can be ordered, from the worst to the best:
 - No polynomial approximation algorithm is known $\Rightarrow S_{\min}(\Pi) = \infty$.
 - There is polynomial approximation algorithm A with performance ratio $S_{\min}(\Pi) < \infty$ which is a function of instance size, e.g. $S_{\min}(\Pi) = O(n)$.
 - There is polynomial approximation algorithm A with performance ratio $1 < S_{\min}(\Pi) < K$ where K is some constant value (see e.g. LPT for $P||C_{\max}$).
 - $S_{\min}(\Pi) = 1$

- For the case of $S_{\min}(\Pi) = 1$ the following special cases can be distinguished, from the worst to the best:
 - PTAS exists for Π ,
 - FPTAS exists for Π ,
 - algorithm A with performance ratio $S_A^\infty = 1$ exists for Π ,
 - problem Π can be solved by a polynomial algorithm which guarantees that $|A(I) - OPT(I)| \leq k$, where k is some constant.

- It can be shown that for some problems certain types of approximation algorithms cannot exist unless $P=NP$. Examples follow ..

Theorem

If $P \neq NP$, then no polynomial algorithm A for knapsack can guarantee that $|A(I) - OPT(I)| \leq k$, where $k > 0$ is constant.

Proof $\rightarrow$ by scaling (or escalating) $k + 1$ times values w_i .

- Note that this proof can be used also in other problems involving integer-valued solutions and arbitrary numerical values determining solution value (number problems): TSP, $P||C_{max}$, etc.

Theorem

If for an NPh CO problem Π there is a polynomial q such that

$$\forall I \in D_{\Pi}, OPT(I) < q(N(I), Max(I)) \quad (*)$$

then the existence of an FPTAS for Π implies existence of a pseudopolynomial optimization algorithm for Π .

- Is it difficult to satisfy conditions of this theorem?
- Observation: This theorem says that if Π satisfies conditions (*) and Π is sNPh, then the existence of an FPTAS for Π would imply $P=NP$.
- Observation: $P \neq NP \Rightarrow$ sNPh problems admit no FPTASes.
- FPTASes and pseudopolynomial algorithms are tightly connected. Existence of a pseudopolynomial algorithm very often implies existence of an FPTAS (see example for knapsack).

Proof of the previous theorem.

Assume that $\varepsilon = \frac{1}{q(N(I), \text{Max}(I))}$, there is an FPTAS A_ε for problem Π and conditions (*) set in the theorem are satisfied.

Complexity $p(N(I), \frac{1}{\varepsilon})$ of the FPTAS A_ε is pseudopolynomial:

$$p(N(I), \frac{1}{\varepsilon}) = p(N(I), q(N(I), \text{Max}(I))) = r(N(I), \text{Max}(I))$$

where p, q, r are polynomials.

Quality of the solutions (minimization) as A_ε is an FPTAS:

$$A_\varepsilon(I) \leq \text{OPT}(I)(1 + \varepsilon) \Rightarrow A_\varepsilon(I) - \text{OPT}(I) \leq \text{OPT}(I)\varepsilon$$

$$\text{By the assumed value of } \varepsilon: \text{OPT}(I)\varepsilon = \frac{\text{OPT}(I)}{q(N(I), \text{Max}(I))}$$

since $\text{OPT}(I) < q(N(I), \text{Max}(I))$, by conditions (*) of the theorem, we get: $1 > \text{OPT}(I)\varepsilon$

and hence: $1 > \text{OPT}(I)\varepsilon \geq A_\varepsilon(I) - \text{OPT}(I)$

So for the integer-valued²⁴ solutions A_ε guarantees optimum solution is pseudopolynomial time. □

- Let us remind that if $P \neq NP$ then no pseudopolynomial algorithms exist for sNPh problems.

²⁴The same applies to any limited-resolution numerical value solutions. All problems with instances of finite size have limited resolution of solution numerical values.

Theorem

Let Π be minimization problem with solution values in $\mathbb{N}$. Suppose that for certain threshold value $k \in \mathbb{N}$ the decision problem Π_D : "Given $I \in D_\Pi$ is $OPT(I) \leq k$?" is NPc, then if $P \neq NP$, there is no polynomial approximation algorithm A for Π satisfying $S_A < 1 + \frac{1}{k}$.

Proof "by example"²⁵

Example: decision problem of graph 3-coloring is NPc (i.e. $k = 3$). By the above theorem if $P \neq NP$, then there is no polynomial approximation algorithm A for graph coloring, such that $S_A < \frac{4}{3}$.

Proof. $\rightarrow$ because such algorithm A solves decision 3-coloring in polynomial time.

- The above theorem also implies that there is no PTAS/FPTAS.

CO problem analysis framework

Consider some CO problem Π and its decision version Π_D .

- 1 Is there a polynomial algorithm for Π ?

Yes: problem is "easy"; exit(); // Π is not interesting anymore

- 2 $\Pi_D \in \text{NPc}$? // proof required

undecided: $\Rightarrow$ problem remains "open";

Yes: we continue ... ; // Π is NPh

- 3 Relax Π to make it α) as general as possible but still polynomial time solvable, β) as simple as possible but still NPc/NPh
// narrowing the range between easy and hard cases

- 4 $\Pi_D \in \text{sNPc}$?

Yes: If $P \neq NP \Rightarrow \neg \exists$ pseudopolynomial algorithm for Π_D, Π

No: $\exists$ pseudopolynomial algorithm for Π_D, Π

- 5 Is there any polynomial approximation algorithm A for Π with performance ratio $S_A < \infty$?

No: Π is not approximable; exit(); // Π is called APX-hard

undecided: $\Rightarrow$ approximability of Π remains "open";

Yes: narrowing the range of performance ratios between upper bounds (performance guarantee of some algorithm A) and lower bounds (theoretical estimation what is possible for Π in the worst case).

Challenges in experimental evaluation

- How to obtain test instances? They should be "representative" (but for what?). There are collections of benchmark instances for some classic CO problems. Problems with benchmark instances - aging of the instances, symbiosis/coevolution of algorithms and benchmarks.
- How to assess solution quality when the optimum is unknown? Comparison with lower bound (minimization), between algorithms, with random solution.
- How to bring the results to a common denominator of computing platform? Hardware, compilers are progressing. Programming environments change (Fortran, C, C++, Java, ...) No good answer.

Challenges in experimental evaluation ...

- How to guarantee that competing algorithms receive the same computational power? (e.g. a matter of quality of programming, implementation differences)
- When to stop computations, esp. considering runtime-solution quality trade-off?
- How to choose a better algorithm when on a population of instances no algorithm dominates the other (no free lunch theorem). Some solution: assume some convention - medians, averages, Formula 1 ranking, etc.

Solving CO problems in practice

M.Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

Dedicated greedy algorithms

- Usually greedy algorithms²⁶ build solutions step-by-step by iteratively adding elements of the solution while minimizing/maximizing some numerical score of the added elements.
- Usually only one solution is constructed.
- Advantage: Consequently they have low-order polynomial complexity (are fast).
- Advantage: solutions can be easily understood/explained.
- These algorithms are *dedicated* in this sense that no one of them constitutes a general framework and it cannot be ported to some other CO problem.

²⁶ Greedy algorithms will be considered again in the context of matroids, i.e. when they deliver optimum solutions.

Metaheuristics

Greedy Randomized Adaptive Search Procedure (GRASP)

- Key ideas:
 - randomize greedy algorithm selection order²⁷
 - Restricted Candidate List (RCL)
 - repeat iteratively until stopping criterion.

²⁷ Can this help? Why? How?

Local search metaheuristics

- Hill Climber
 - Multistart Local Search, Iterated Local Search
 - Tabu Search
 - Variable Neighborhood Search
 - Simulated Annealing
-
- Key idea: move(s), solution neighborhood.

Evolutionary (a.k.a. nature-inspired, population-based)

These methods have populations of solutions. For example:

- Genetic Algorithm (GA)

- Particle Swarm Optimization update $\overrightarrow{vector}$ for individual $\overline{x}_i$:

$$\overrightarrow{speed}_i = \alpha \overrightarrow{speed}_i + \beta \times RND \times (\overline{PobB} - \overline{x}_i) + \gamma \times RND \times (\overline{IndB}_i - \overline{x}_i)$$

$$\overline{x}_i = \overline{x}_i + \overrightarrow{speed}_i, \text{ where:}$$

$\overrightarrow{speed}_i$ – direction of change vector, $\overline{x}_i$ – individual i vector,

$\overline{PobB}$ – population best solution, $\overline{IndB}_i$ – individual i best visited solution,

α, β, γ – control parameters, $RND \sim U(0, 1)$.

- Differential Evolution trial $\overrightarrow{vector}$ for individual $\overline{x}_i$:

$$\overline{v} = \overline{x}_{r1} + \alpha(\overline{x}_{r2} - \overline{x}_{r3}), \text{ where:}$$

$\overline{x}_i$ – individual i vector, $r1, r2, r3 \sim U\{1, PopSiz\}$, $PopSiz$ – population size, α – control parameter.

$\overline{u} = crossover(x_i, v, n, L)$ – exchange dimensions of x_i, v from position n to $n + L - 1$, where:

$$n, L \sim U\{1, PopSiz\}$$

- Ant Colony Optimization (ACO, a.k.a. ant systems)

- and many others: artificial bee colony, harmony search ...

Nature and population -based
Meta-heuristics (323: 100%)

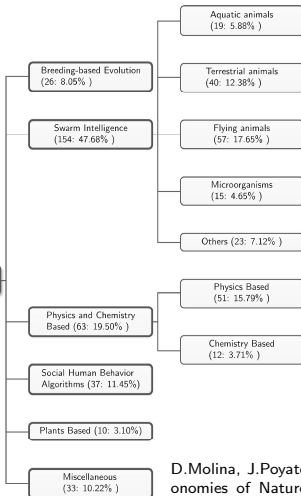
[illegible][illegible]

Table 5: Same and assigned same kinetics with the Protein (same degree)			
Protein level (%)			
algorithm name	Average	Size	Reference
Non-Inspired Optimisation	505	204	[17]
Non-Mem Optimisation Algorithm	505	204	[17]
Swarm Dynamics Optimisation	505	204	[17]
Spinal Optimisation Algorithm	SPH4	508	[17]
Self-Start-Particle	SPP	189	[17]
Fluency-Particle Swarm Algorithm	FPSO	207	[17]
Swarm Size Algorithm	NS	203	[38]
Non-Local Optimisation	NLO	203	[38]
Non-Local Optimisation	NLO	206	[38]
Non-Local Optimisation	NLO	207	[38]
Non-Local Optimisation	NLO	203	[38]
Non-Local Optimisation	NLO	201	[38]
Non-Local Optimisation	NLO	203	[38]

[illegible]

D.Molina, J.Poyatos, J.Del Ser, S.Garcia, A.Hussain, F.Herrera, Comprehensive Taxonomies of Nature- and Bio-inspired Optimization: Inspiration Versus Algorithmic Behavior, Critical Analysis Recommendations, Cognitive Computation 12, 897–939 (2020). <https://doi.org/10.1007/s12559-020-09730-8>

Non-classic metaheuristics

- Hybrid metaheuristics – combine different types of algorithms for CO problems. For example, Hybrid Genetic Algorithms = GA + Greedy or GA + Local search
- Hiperheuristics – the search process is moved from the space of instance solutions to the space of low level heuristics (LLH). A hiperheuristic chooses an LLH to search the solution space most effectively. This turns to be a problem of online classifying instances and current solutions to classes of LLHs which improve solutions most effectively $\Rightarrow$ machine learning.

- Overall, what matters are:
 - 1 how the differences between population members are calculated (differential vector, reference solutions)
 - 2 how the differences are used to build new individuals
 - 3 how the processes of diversification (exploration)
 - 4 and intensification (exploitation) are controlled,
 - 5 what is randomized (where randomness is poured-in).

See: <https://en.wikipedia.org/wiki/Metaheuristic>

And do not forget about machine learning, reinforced learning :)

Exact algorithms

Other names: exhaustive alg., exhaustive search

- Exact algorithms have exponential runtime in the worst case.
- In practice very much depends on identifying hard instances, sources of their hardness, defining dedicated pruning strategies, quite often runtimes of exact algorithms can be acceptable²⁸.
- Main types of exact algorithms:
 - Brute force enumeration – considered dumb and doable only for very small sizes
 - Dynamic programming – like the ones for knapsack

■ Branch and bound – key ingredients:

- 1) enumeration of solutions has tree-like form with partial solutions represented by nodes inside the tree and complete solutions in the tree leaves; branching is expansion of some internal tree node;
- 2) pruning the tree by use of lower and upper bounds; for minimization lower bound is an estimation of the objective function from below for some partial solution (internal tree node) and upper bound is the objective function value of an existing solution;
- 3) typical tree searching rules: DFS, depth first least lower bound (DF-LLB).

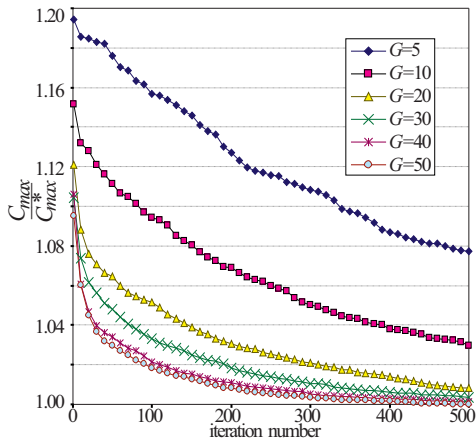
- Reduction to integer linear programs (ILP)²⁹, mixed integer linear programs (MIPs/MILPs) quadratic programs and applying external solvers such as CPLEX, Gurobi, Hexaly³⁰, Ip_solve, SCIP, glpk, Google OR Tools, ...
- Monte Carlo Tree Search – originally designed for games, but can be repurposed to CO algorithms searching a tree. Key idea: expand actions (children) of the best active node at random, rollout the decision process by choosing next action (children) uniformly at random until reaching a leaf node, use information on the average leaves objective function to choose the next best node to expand. In some sense randomized Branch and Bound. Hence, MCTS is a heuristic!

²⁹ Examples of ILP/MIP in graph coloring section.

³⁰ Contemporary leading MIP solvers (=Cplex, Gurobi) are themselves combinations of heuristics and B&B methods often multithreaded (parallel)

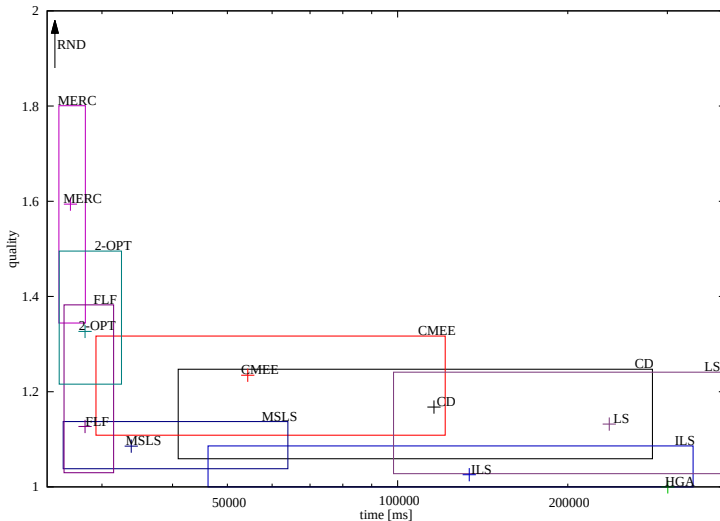
Runtime-quality trade-off

A problem of scheduling parallel computations, genetic algorithm. Mean relative distance from the best known solution C_{max}/C_{max}^* vs iteration number ($\neq$ time) and population size G .



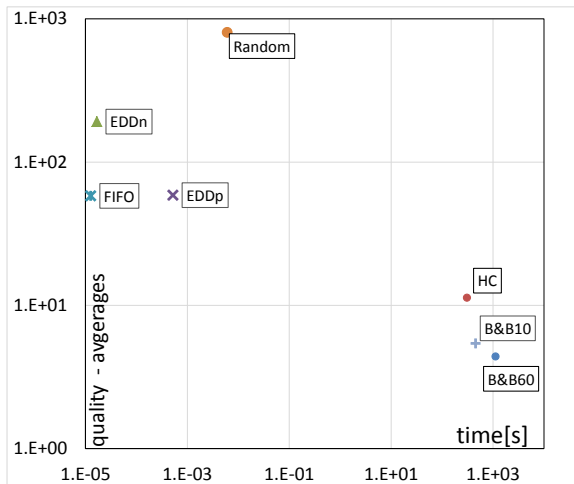
Lower is better. Why should this picture look this way?

Logistic problem on graphs. Each box is quartile $[Q_1, Q_3]$ range on runtime and on relative distance from the best known solution in a dataset of test instances.



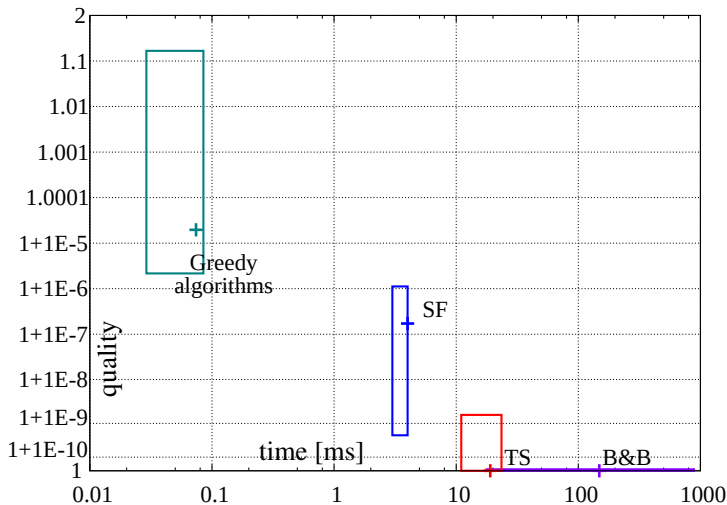
Lower is better. Actually, what is shown on this picture? Why should this picture look this way?

Some scheduling problem with deadlines. Average runtime and on relative distance from the best known solution in a dataset of test instances.



Lower is better. Actually, what is shown on this picture? Why should this picture look this way?

Some variant of 2D-packing problem



Lower is better. The same set of questions as before.

Algorithm Selection Problem

- Since there are so many different algorithms for CO problems, how should one choose the "best" one?
- There are so called "*No Free Lunch*" (NFL) Theorems,³¹ i.e. theoretical studies asserting that for any algorithm, any elevated performance over one class of problems is offset by performance over another class.
- The NFL theorems mean that ultimately there is no algorithm for a CO problem which dominates all other algorithms under all circumstances.

³¹D.Wolpert, W.Macready, No Free Lunch Theorems for Optimization, IEEE Trans. on Evol. Comput. 1, 67–82, 1997. doi:10.1109/4235.585893

- Selection of good algorithms is subject of the algorithm selection problem ...

Algorithm Selection Problem (ASP)³² is an optimization problem with solutions in the space of algorithms and consists in choosing the best algorithm to solve some typical instances from some field of application.

³²John R. Rice, The Algorithm Selection Problem (1975). Purdue University, Computer Science Technical Reports. Paper 99.

- Algorithm selection problem for some given application problem Π (e.g. CO problem like TSP, knapsack, etc) can be formalized in the following way:

- Given:

- (i) set D_Π of Π instances, called problem space,

- (ii) set $\mathcal{A}$ of algorithms for Π , called algorithm space,

- (iii) performance measure $p : D_\Pi \times \mathcal{A} \rightarrow R$,

find a mapping (i.e. a function) $S : D_\Pi \rightarrow \mathcal{A}$ which maximizes performance measure $p(I, S(I))$.

- In the alternative formulation given is additionally:

- (iv) set of instance characteristics $f(I)$ called feature space $\mathcal{F}$.

Then find mapping $S : \mathcal{F} \rightarrow \mathcal{A}$ which maximizes $p(I, S(f(I)))$.

- Runtime and solution quality are typical performance measures in CO. Yet, they are mutually interrelated. Hence, ASP is a multicriteria problem³³
- Example features in feature space $\mathcal{F}$ for graph problems:: number of nodes, edges, graph density, mean vertex degree, standard deviation of vertex degrees, mean distance between all pairs of vertices, graph diameter³⁴, girth of graph³⁵, graph spectrum³⁶, etc.
- For the above formalization of ASP, choosing the algorithm is instance classification problem $\Rightarrow$ machine learning.
That is, if algorithms $A \in \mathcal{A}$ have classes $D_{\Pi}^A \subset D_{\Pi}$ of instances which are solved best by algorithm A , then assign class D_{Π}^A to each instance $I \in D_{\Pi}$.

³³How to solve multicriteria problems $\rightarrow$ see decision support course.

³⁴length of the longest shortest path connecting any pair of nodes

³⁵length of the shortest cycle

³⁶ordered eigenvalues of graph Laplacian matrix

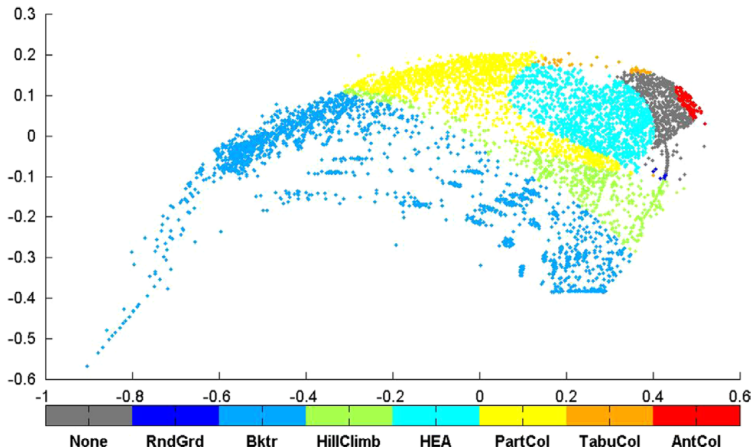


Fig. 4. Machine learning (SVM) recommendations about which algorithm to use in each region.

K.Smith-Miles, D.Baatar, B.Wreford, R.Lewis, Towards objective measures of algorithm performance across instance space, Computers & Operations Research 45(2014) 12–24, <http://dx.doi.org/10.1016/j.cor.2013.11.015> The above 2D projection with continuous axes (though features are integer) was obtained by use of Principal Component Analysis (PCA) and choosing the biggest two components. SVM= Support vector machines is a classification algorithm.

Greedy algorithms and matroids

Maciej Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

Let's reiterate. Greedy algorithms have the following features:

- A solution is constructed by adding elements one-by-one.
- The added element respects constraints of the solution and minimizes (resp. maximizes) some objective function.
- Greedy algorithms are single-pass:
 - 1) an element added to a solution is never removed,
 - 2) after constructing the solution greedy stops.

⇒ only one solution is built.
- They are fast, i.e. have low-order polynomial time complexity.
- It appears that greedy algorithms can construct optimum solutions for some problems.
- How come? How can they manage if so many CO problems are hard? What features such problems must have to be solved to optimality by a greedy algorithm.

Minimum Spanning Tree (MST)

→ say something about historical origins

Given graph $G(V, E)$ with weights $w_{i,j}$ on edges find a spanning tree³⁷ of minimum sum of weights of chosen edges.

Prim's Algorithm

Idea: add to the MST the closest³⁸ vertex.

Data structures:

$c[i]$ a vertex already in the MST, closest to vertex i

V set of vertices not yet in the MST

$W[i, j]$ edge weights matrix

$N[i]$ neighbors of vertex i

³⁷ Spanning tree connects all vertices of G .

³⁸ weights $w_{i,j}$ are interpreted as distances

```

function PrimMST;
1:  V:= V-1; tree={}; // {} is an empty set
2:  for all i∈V {c[i]:=1};
3:  while V≠ {}
    {
4:      p:=argmin{W[i,c[i]]: i∈V};
// argmin - choose argument i such that value in {} is minimum,
// i.e. any vertex p∈V such that W[p,c[p]]=min{W[i,c[i]]: i∈V}
5:      tree:=tree+{p,c[p]}; // add edge {p,c[p]} to the MST
6:      V:=V-{p};
7:      for all j∈N[p]; // for all neighbors of vertex p
8:          if (W[j,p]<W[j,c[j]]) {c[j]=p;}
// improve c[j] by choosing between old c[j] and p
    };
9:  return tree;

```

Computational complexity of Prim's algorithm is $O(m + m \log n)$ because we have to use, e.g., priority queue to find vertex p in line 4, and in line 8 all edges are enumerated.³⁹

Kruskal's algorithm

Idea: add to the MST the lightest edge such that no cycle is created.

Problem: How to recognize effectively that a cycle can be made. In other words, how to recognize disconnected subtrees which can be connected such that no cycle is made?

For this purpose UNION, FIND functions can be used.

UNION(x, y) unites two sets (here: two sub-trees) comprising elements x, y

FIND(x) finds an element representing set (here: a sub-tree) comprising element x

UNION, FIND can be executed in amortized constant time.⁴⁰

⁴⁰ Precisely in time $O(\alpha(n))$ where $\alpha()$ is an inverse function of Ackerman function which values grow to the exponential, exponential, ..., exponential power https://en.wikipedia.org/wiki/Ackermann_function ▶

union, find implementation by

<http://arufa.wordpress.com/2008/08/13/a-very-short-and-neat-code-for-union-find/>
[access: 10.XI.2019]

```
int parent[N];  
int find(int u)  
{  
return u == parent[u] ? u : parent[u] = find(parent[u]);  
}  
int union(int u,int v)  
{  
parent[find(u)] = find(v);  
}  
// init  
for(int i=0;i<N;++i) parent[i] = i;
```

└ Example - Minimum Spanning Tree (MST)

└ Kruskal's algorithm, but ... union, find

union, find implementation by

<http://sys.cs.rice.edu/course/comp314/09/lectures/9-disjoint-sets.html>

[access: 8.XII.2009]. BTW, indents are meaningful

MAKE_SET(x)

1. $p[x] = x$
2. $rank[x] = 0$

UNION(x,y)

1. LINK(FIND_SET(x), FIND_SET(y))

LINK(x,y)

1. if ($rank[x] > rank[y]$)
2. $p[y] = x$
3. else
4. $p[x] = y$
5. if ($rank[x] == rank[y]$)
6. $rank[y] = rank[y] + 1$

FIND_SET(x)

1. if ($x \neq p[x]$)
2. $p[x] = \text{FIND_SET}(p[x])$
3. return $p[x]$

Data structures:

$S[i]$ identifier (representative) of the subtree comprising vertex i

```
function KruskalMST
```

```
1:  tree={};
```

```
2:  for all  $i \in V$   $\{S[i] := i\}$ ; //every vertex is in its own subtree
```

```
3:  while  $|tree| < n-1$  do
```

```
{
```

```
4:   $\{i, j\} :=$  the lightest edge connecting two  
    disconnected subtrees, i.e.  $S[i] \neq S[j]$ ;
```

```
5:   $tree := tree + \{i, j\}$ ;
```

```
6:   $union(i, j)$ ;
```

```
};
```

```
7:  return tree;
```

Complexity $O(m \log m)$ because edges are sorted by weights.

Huffman coding

- Huffman coding algorithm is used in lossless compression by assigning variable-length binary codes to the input symbols.
- The length of the code for some input symbol depends on frequency (probability) of symbol occurrence.
- Notations:
 - s_i symbol i
 - p_i probability/frequency/weight of symbol i
 - b_i number of bits encoding symbol i
 - n number of symbols
- Huffman algorithm minimizes weighted length L of symbols string defined as

$$L = \sum_{i=1}^n b_i p_i$$

where p_i s are probabilities.

- Huffman algorithm operation can be interpreted as construction of a binary tree bottom-up.

function Huffman

1:create n binary trees with nodes (s_i, p_i) ;

2:while the number of binary trees > 1

{

 join two trees of i, j with minimum $(p_i + p_j)$ into a new tree t with weight $(p_i + p_j)$, append the original trees i, j as the left and right subtrees of t ;

}

3:traverse the final tree using DFS and assigning bit value 0 to the left and bit value 1 to the right subtree.⁴¹ Read a symbol encoding as the bits on the path from root to the leave.

⁴¹ the other way round is also possible

Example.

s_i	a	b	c	d
p_i	0.1	0.3	0.4	0.2
code	000	01	1	001

Weighted length of the encoded string $L = 3 * 0.1 + 3 * 0.2 + 2 * 0.3 + 1 * 0.4 = 1.9 < 2$.

4 trees: a, 0.1 d, 0.2 b, 0.3 c, 0.4

0.3

3 trees: a, 0.1 d, 0.2 b, 0.3 c, 0.4

0.6

0.3

2 trees: a, 0.1 d, 0.2 b, 0.3 c, 0.4

1

0.6

0.3

1 tree: a, 0.1 d, 0.2 b, 0.3 c, 0.4

red digits - assigned bits of code

More examples of problems solved to optimality by a greedy algorithm:

- Continuous knapsack problem⁴² – choose to the knapsack elements according to decreasing gain from a unit of space w_i/s_i (w_i - value, s_i - size i)
- Change making problem: Returning the rest in cash in minimum number of coins or notes. What is the requirement for the correctness of this algorithm?⁴³

⁴² see page 28

⁴³ homework, or if in trouble see <https://arxiv.org/abs/0809.0400>

- Scheduling problem $1||L_{max}$ – execute jobs according to non-decreasing due-dates d_j (earliest due-date/EDD order)
- $1||\sum c_j$ – execute jobs according to non-decreasing processing times p_j (shortest processing time, SPT, minimizes mean processing time)
- $1||\sum w_j c_j$ – execute jobs according to non-decreasing ratios w_i/p_i (weighted shortest processing time, WSPT)
- $1||\sum U_j$ – schedule jobs according to non-decreasing due-dates d_j (EDD order). Let $i < n$ be the first late job, (1) move the longest job in $1, \dots, i$ to the end of the schedule, (2) shift the jobs earlier to close the empty interval.

- $1|p_j = 1| \sum w_j U_j$ – for n jobs there are also n time slots of duration 1, assign jobs to time slots in the order of non-increasing weights w_j assign job j to the latest available time slot before its deadline d_j , if no such slot is available put job j in the last free slot.
- $1|p_j = d_j - r_j| \sum (1 - U_j)$ – so-called *interval scheduling*, jobs must be executed nonpreemptively from ready times (r_j) to due-dates (d_j), maximize the number of executed jobs $\sum (1 - U_j)$. Schedule whatever possible in the order of non-decreasing deadlines due-dates $d_1 \leq d_2 \leq \dots \leq d_n$.

Matroids

Assume that we are given:

- finite set E , called ground set
- family $J \subseteq 2^E$ i.e. some set J of E subsets

Definition

*We will call pair $M = (E, J)$ a **matroid**, if E is a finite set and $J \subseteq 2^E$, and M satisfies conditions:*

- (a)** $\emptyset \in J$,
- (b)** $A \in J \text{ \& } B \subseteq A \Rightarrow B \in J$,
- (c)** $A, B \in J \text{ \& } |B| = |A| + 1 \Rightarrow \exists e \in B \setminus A, \text{ such that } A \cup \{e\} \in J$.

→ say something about practical interpretation of conditions a,b,c

Assume :

- weight function $w : E \rightarrow \mathbb{Q}^+$ (i.e. weights $w(e)$ of elements e)⁴⁴
- $w(A) = \sum_{e \in A} w(e)$ is weight of set A
- matroid (E, J) with the above weight function is called **weighted matroid**.

Definition

Greedy algorithm:

1. *sort elements $e \in E$ according to non-decreasing weights $w(e_i)$ $w(e_1) \leq w(e_2) \leq \dots \leq w(e_n)$ (resp. non-increasing for maximization);*
2. $A = \emptyset$;
3. **for**($i=0; i < n; i++$) { if $A \cup \{e_i\} \in J$, then $A := A \cup \{e_i\}$ }

⁴⁴ mathematically-inclined one would rather declare $a(u, v) \in \mathbb{R}$, but since reals do not have a simple representation in computers we declare rational numbers $\mathbb{Q}$.

Practically all sets $A \in J$ are feasible solutions of the considered problems.⁴⁵

Theorem

If $M = (E, J)$ is a weighted matroid, then set A built by a greedy algorithm has the smallest weight (resp. the largest for maximization).

⁴⁵sets $A \in J$ where $M = (E, J)$ is a matroid are also called to be independent sets, which BTW, is different notion than independent set in graph theory.

Shortest paths

Maciej Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

Shortest paths

- We are given a directed graph $G = (V, A)$ and $\forall (u, v) \in A$ weight⁴⁶ $a(u, v) \in \mathbb{Q}$ called **arc weight** or distance from u to v .
- A directed⁴⁷ **path** in $G(V, A)$ is a sequence of vertices $v_1, \dots, v_p$, such that $\forall_i, (v_i, v_{i+1}) \in A$
- A **simple path** is a path with all vertices different.
- For simplicity of presentation we assume that each possible arc in A exists, but some of them may have infinite weight (∞), then such arcs cannot be traversed as if they didn't exist.
- In the following we assume $|V| = n$, $|E| = m$.

⁴⁶since reals $\mathbb{R}$ do not have a simple representation in computers we restrict to rational numbers⁴⁷

we refer to directed graphs here so "directed" will be skipped in the further text for brevity

Length of a path from vertex $s = v_1$ to vertex $t = v_p$ with some p nodes, alternatively distance from s to t , is

$$d(s, t) = \sum_{i=1}^p a(v_{i-1}, v_i).$$

Single-pair shortest path problem find the shortest path from source vertex u to destination vertex v .

Single-source shortest paths problem find all shortest paths from source vertex s to all other vertices in $V \setminus \{s\}$.

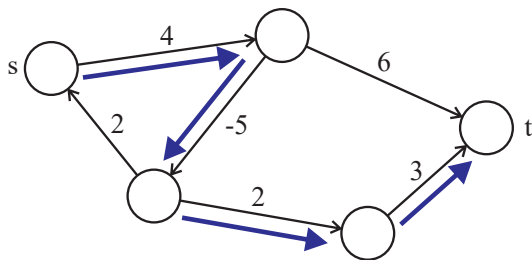
All-pairs shortest paths problem find all shortest paths between all pairs of vertices in $G(V, A)$.

Single-source shortest paths problem will be considered first.

Remarks:

1. Assume that each cycle in $G(V, A)$ has positive length.

- Question: is it advantageous to have any cycles in the shortest paths?
- Conclusion: if each cycle in $G(V, A)$ has positive length then all shortest paths are simple shortest paths (all vertices are different).

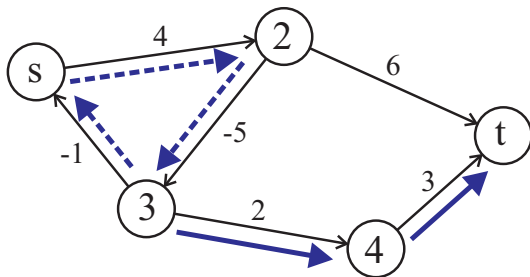


$$d(s, t) = 4$$

2. Assume there is a negative-length cycle in $G(V, A)$

- Question: what is the length of any path in $G(V, A)$, if we step into the negative-length cycle?
- Conclusion: if there is a negative-length cycle in $G(V, A)$ then distance between some nodes may be undefined (depends if a cycle is accessible and how many times one traverses the cycle).

→ Are there any applications with negative cycles?



$$d(s, t) = ?, d(2, t) = ?, d(4, t) = 3$$

Applications:

- 1 Shortest connections between locations in some logistic network, e.g. between cities, warehouses, etc.
- 2 Fastest connections in computer networks⁴⁸
- 3 Most reliable connection between two node in a network (as arc weight assume $a(u, v) = -\log p(u, v)$, where $p(u, v)$ is probability of reliable (correct) operation of connection (u, v))
- 4 Finding the *critical path* in PERT/CPM methods considering construction/ software/ development projects. Critical path is the longest path in $G(V, A)$ ⁴⁹ → examples will be given later.

⁴⁸ what performance index of node pairwise connection should be used as arc weight?

⁴⁹ Homework: 1. What can be the advantage of knowing the critical path in some project? 2. Is the longest path problem polynomially solvable at all? 3. How come that the critical=the longest path be found by algorithms for shortest paths? Hint: what type of graphs we must have when we plan/schedule some project. ▶ ◀ ≡ ▶ ≡

- If we know the shortest distances from source s to all other nodes v , then the path from source s to destination t as a sequence of nodes can be always reconstructed.
- Therefore, in the following only algorithms calculating the shortest distances will be presented.
- The shortest path connecting s with t , as a sequence of nodes, can be reconstructed by finding the intermediate nodes v sequentially among the nodes satisfying condition

$$d(s, t) = d(s, v) + a(v, t)$$

- Vertex v is penultimate on the shortest path from s to t .⁵⁰

Algorithm Shortest path for known distances

Input: distances $D[v]$ from source s to all vertices $v \in V \setminus \{s\}$;
chosen destination vertex t ; arc weight matrix $A[u, v] \forall u, v \in V$.

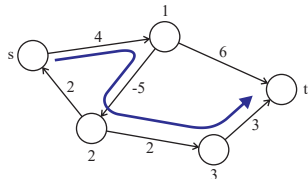
Output: stack STOS comprising sequence of nodes on the shortest path from s to t .

```
1:  begin
2:    STOS:= $\emptyset$ ; STOS $\leftarrow t$ ;  $v := t$ ;
3:    while  $v \neq s$  do
4:      begin
5:         $u :=$  vertex satisfying  $D[v] = D[u] + A[u, v]$ 
6:        STOS  $\leftarrow u$ ;
7:         $v := u$ ;
8:      end;
9:    end
```


Computational complexity.

- If in line 5 vertex u is chosen by enumerating all vertices $\Rightarrow$ algorithm complexity is $O(n^2)$.
- If in line 5 vertex u is chosen by scanning only predecessors of v , which can be provided on incidence list `before[v]` holding such nodes that $(u, v) \in A$, then complexity is $O(m)$.

Example



$$A[u, v] = \begin{bmatrix} 0 & 4 & \infty & \infty & \infty \\ \infty & 0 & -5 & \infty & 6 \\ 2 & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

$$D[v] = [0, 4, -1, 1, 4]$$

$$d(s, t) = 4$$

- $STOS = [t] \quad v := t$
- $u := 3 \quad D(t) = D(u) + A(u, t)$
 $4 = 1 + 3$

$STOS = [t, 3]$

$v := 3$

- $u := 2 \quad D(3) = D(u) + A(u, 3)$
 $1 = -1 + 2$

$STOS = [t, 3, 2]$

$v := 2$

- $u := 1 \quad D(2) = D(u) + A(u, 2)$
 $-1 = 4 + (-5)$

$STOS = [t, 3, 2, 1]$

$v := 1$

- $u := s \quad D(1) = D(u) + A(u, 1)$
 $4 = 0 + 4$

$STOS = [t, 3, 2, 1, s]$

$v := s$

General idea of the algorithms:

- In each iteration an upper bound on distance $D[v]$ from source s to all other nodes $v \in V$ is calculated.
- If $D(v) > D(u) + A(u, v)$ then it is profitable to arrive in node v from predecessor u , and hence the upper bound is improved (reduced)

$$D(v) := D(u) + A(u, v)$$

- This procedure stops when no upper bound can be improved. In this case all upper bounds $D[v]$ are the searched for minimum distances $d(s, v)$.
- The algorithms differ in the way (i.e. sequence) of evaluating nodes v .

A. Graphs without negative-length cycles

Algorithm Single-source shortest paths, Ford-Bellman, ~1956-58.

Input: graph $G(V, A)$ with $n = |V|$, arc weight matrix

$A[u, v] \forall u, v \in V$, selected source node s

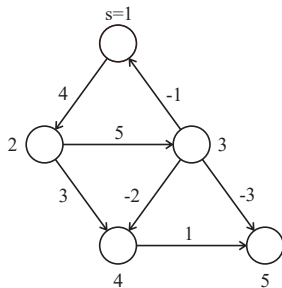
Output: shortest-path lengths $D[v] = d[s, v], \forall v \in V$ from s

```
1:  begin
2:    for  $v \in V$  do  $D[v] := A[s, v]; D[s] := 0;$ 
3:    for  $k := 1$  to  $n - 2$  do // is  $n - 2$  sufficient?
4:      for  $v \in V \setminus \{s\}$  do
5:        for  $u \in V$  do  $D[v] := \min\{D[v], D[u] + A[u, v]\}$ 
6:  end
```

- Computational complexity: $O(n^3)$.
- For sparse graphs, i.e. such that $m \ll n^2$, it is profitable to represent the graph using incidence lists `before[v]`. Then if line 5 is changed to
for $u \in \text{before}[v]$ do $D[v] := \min\{D[v], D[u] + A[u, v]\}$
we get complexity $O(nm)$, which is $O(n^2)$ for trees.
- This algorithm may stop if no value $D[v], v \in V$ is changed in loop 4, which may happen for $k < n - 2$. Yet, such a modification is not changing complexity of the algorithm.

- Correctness of the algorithm follows from the fact that in a graph with n nodes all simple paths with at most $n - 1$ arcs can be built in $n - 2$ steps, and the shortest path can be selected. Longer paths must not be simple and must have cycles which are not reducing path length (because there are no negative cycles).
- Conversely, results of this algorithm can be used to identify negative cycles (if $G(V, A)$ has them) by checking after the algorithm stops whether $\exists (u, v) \in A, D[u] + A[u, v] < D[v]$.

Example



$$A[u, v] = \begin{bmatrix} 0 & 4 & \infty & \infty & \infty \\ \infty & 0 & 5 & 3 & \infty \\ -1 & \infty & 0 & -2 & -3 \\ \infty & \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

$D[v]$ in iterations

k	$D[1]$	$D[2]$	$D[3]$	$D[4]$	$D[5]$
	0	4	∞	∞	∞
1	0	4	9	7	6
2	0	4	9	7	6
3	0	4	9	7	6

Algorithm Single-source shortest paths. Moore, Bellman, d'Esopo, Pape ~1980

Input: Graph $G = (V, A)$ represented by incidence list $\text{after}[v]$, $\forall v \in V$,
 chosen vertex s .

Output: shortest-path lengths $D[v] = d[s, v]$, $\forall v \in V$

Data structures: deq - double-ended queue (insert from both ends, remove from one)

```

1: begin
2:    $\text{deq} \leftarrow s$ ; for  $v \in V$  do  $D[v] := \infty$ ;  $D[s] := 0$ ;
3:   while  $\text{deq} \neq \emptyset$  do
4:     begin
5:        $u \leftarrow \text{deq}$ ; // take the first node from the queue
6:       for  $v \in \text{after}[u]$  do
7:         begin
8:            $\text{new\_dist} := D[u] + A[u, v]$ ;
9:           if  $\text{new\_dist} < D[v]$  then
10:            begin
11:               $D[v] := \text{new\_dist}$ ;
12:              if  $v$  was never in  $\text{deq}$  then  $\text{deq} \leftarrow v$  // append  $v$  at the end of  $\text{deq}$ 
13:            else if  $v \notin \text{deq}$  then  $v \Rightarrow \text{deq}$ ; // push  $v$  to the first position of  $\text{deq}$ 
14:            end
15:          end
16:        end
17:      end

```


Computational complexity: is hard to established analytically, because it is difficult to predict how many times any node can return to deq. It is possible to construct graphs for which complexity is exponential (i.e. $O(2^n)$). However, extensive computational experiments showed that for sprase graphs this is one of the fastest algorithms (certainly faster than Dijkstra's or Ford-Bellman's).

B. Graphs with non-negative arc weights

Algorithm Single-source shortest-paths, Dijkstra ~1956-59

Input: graph $G(V, A)$ with $n = |V|$, arc weight matrix

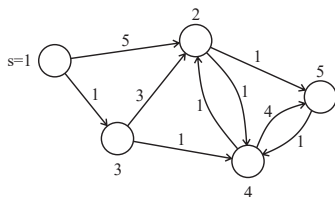
$A[u, v] \forall u, v \in V$, selected source node s

Output: shortest-path lengths $D[v] = d[s, v], \forall v \in V$ from s

```
1:  begin
2:    for  $v \in V$  do  $D[v] := A[s, v]; D[s] := 0$ ;
3:     $T := V \setminus \{s\}$ ;
4:    while  $T \neq \emptyset$  do
5:      begin
6:         $u := \operatorname{argmin}\{D[p] : p \in T\}$ ; //take any vertex in  $T$ 
        //minimizing  $D[p]$ 
7:         $T := T \setminus \{u\}$ ;
8:        for  $v \in T$  do  $D[v] := \min\{D[v], D[u] + A[u, v]\}$ ;
9:      end;
10:  end
```

- Computational complexity: $O(n^2)$ because loop 4 is executed $(n - 1)$ times, and in each iteration finding node u requires $O(n)$ time steps in line 6 and $O(n)$ steps in line 8.
- If set T is represented by 1) a binary out-tree such that u is a successor of v then $D[u] \geq D[v]$ (as heap in heapsort), 2) an effective implementation of a priority queue (like C++ multiset), then complexity is $O(n \log n + m)$.
- Correctness of the algorithm follows from the following invariant of loop 4:
 $\forall v \in V \setminus T, D[v] = d(s, v)$
 $\forall v \in T, D[v] = \text{length of the shortest path from } s \text{ to } v \text{ such that the penultimate node on this path is in } V \setminus T.$

Example



$$A[u, v] = \begin{bmatrix} 0 & 5 & 1 & \infty & \infty \\ \infty & 0 & \infty & 1 & 1 \\ \infty & 3 & 0 & 1 & \infty \\ \infty & 1 & \infty & 0 & 4 \\ \infty & \infty & \infty & 1 & 0 \end{bmatrix}$$

nodes remaining in T are underlined $\underline{v}$, nodes removed from T are in boxes

D(1)	D(2)	D(3)	D(4)	D(5)
0	<u>5</u>	<u>1</u>	<u>∞</u>	<u>∞</u>
0	<u>4</u>	1	<u>2</u>	<u>∞</u>
0	<u>3</u>	1	2	<u>6</u>
0	3	1	2	<u>4</u>

C. Acyclic graphs DAGs - directed Acyclic Graphs

- Nodes in any DAG can be numbered such that $\forall (i, j) \in A, i < j$.

Algorithm topological sorting, i.e. numbering nodes of a DAG such that $\forall (i, j) \in A, i < j$

Input: DAG $G = (V, A)$ represented by incidence list `after[v]`, $\forall v \in V$.

Output: Node numbers in `NR[v]` such that $\forall (i, j) \in A, i < j$.

Other data structures:

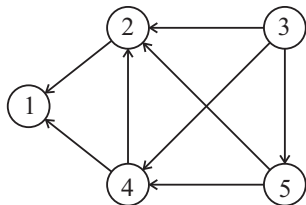
`LID0[v]` number of arcs ending in vertex v

`STOS` stack of nodes which can be numbered

`num` node counter

```
1:  begin
2:    for  $v \in V$  do LIDO[ $v$ ] := 0;
3:    for  $u \in V$  do
4:      for  $v \in \text{after}[u]$  do LIDO[ $v$ ] := LIDO[ $v$ ] + 1;
5:    STOS :=  $\emptyset$ ;
6:    for  $v \in V$  do
7:      if LIDO[ $v$ ] = 0 then STOS  $\leftarrow v$ ;
8:    num := 0;
9:    while STOS  $\neq \emptyset$  do
10:      begin
11:         $u \leftarrow \text{STOS}$ ; num := num + 1; NR[ $u$ ] = num;
12:        for  $v \in \text{after}[u]$  do
13:          begin
14:            LIDO[ $v$ ] := LIDO[ $v$ ] - 1;
15:            if LIDO[ $v$ ] = 0 then STOS  $\leftarrow v$ ;
16:          end;
17:        end;
18:      end
```

- Computational complexity is $O(m)$ because each arc is considered twice: in line 4 and in line 12.
- Correctness of the algorithm follows from the facts:
 - there is node u without predecessors in every DAG;
 - by giving node u the smallest available number the required ordering of the nodes is observed;
 - by removing arcs departing from node u we obtain a new DAG with the number of nodes reduced by 1;
 - this procedure can be repeated until removing all arcs from the DAG and numbering all nodes.

Exampleafter[1] = ϕ

after[2] = [1]

after[3] = [2, 4, 5]

after[4] = [1, 2]

after[5] = [2, 4]

STOS	LIDO[1], NR[1]		LIDO[2], NR[2]		LIDO[3], NR[3]		LIDO[4], NR[4]		LIDO[5], NR[5]	
	0,	-	0,	-	0,	-	0,	-	0,	-
$\emptyset$	2,	-	3,	-	0,	-	2,	-	1,	-
[3], $u = 3$	2,	-	2,	-	0,	1	1,	-	0,	-
[5], $u = 5$	2,	-	1,	-	0,	1	0,	-	0,	2
[4], $u = 4$	1,	-	0,	-	0,	1	0,	3	0,	2
[2], $u = 2$	0,	5	0,	4	0,	1	0,	3	0,	2

Algorithm Single-source shortest paths in DAG

Input: DAG $G = (V, A)$ such that $\forall (i, j) \in A, i < j$, represented by incidence list $\text{before}[v]$, $\forall v \in V$.

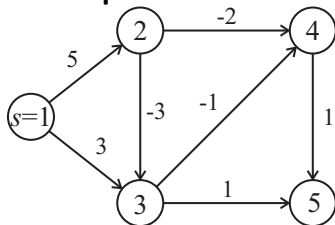
Output: shortest-path lengths $D[v] = d[s, v], \forall v \in V$

```
1: begin
2:    $D[1] := 0$ ;
3:   for  $j = 2$  to  $n$  do  $D[j] := \infty$ ;
4:   for  $j = 2$  to  $n$  do
5:     for  $i \in \text{before}[j]$  do  $D[j] := \min\{D[j], D[i] + A[i, j]\}$ ;
6: end
```

Complexity is $O(m)$ because each arc is visited once in line 5.

Correctness follows from the fact that when calculating $D[j]$ in line 4 all predecessors i have $D[i] = d(s, i)$ and all predecessors i of j are verified.

The same algorithm can be applied to find longest paths from s by either a) assuming $A'[i, j] := -A[i, j]$ and finding the shortest distances, or b) taking $\max\{\}$ in line 5 instead of $\min$.

Example

$$A[i,j] = \begin{bmatrix} 0 & 5 & 3 & \infty & \infty \\ \infty & 0 & -3 & -2 & \infty \\ \infty & \infty & 0 & -1 & 1 \\ \infty & \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

updates of $D[v]$ in iterations

j	$D[v_1]$	$D[v_2]$	$D[v_3]$	$D[v_4]$	$D[v_5]$
	0	∞	∞	∞	∞
2	0	5	∞	∞	∞
3	0	5	2	∞	∞
4	0	5	2	1	∞
5	0	5	2	1	2

Application⁵¹

- The single-source longest path⁵² algorithm is applied in *Critical Path Method* (CPM) and *Project Evaluation & Review Technique* (PERT), two methods of *project scheduling*.
- The project is represented by the so-called activity network (a.k.a. activity-on-arc network) $G(V, A)$ which is a connected DAG, arcs in A are activities with certain durations given as positive weights and nodes are states in the project progress.

⁵¹important: DAGs

⁵²it is the previous algorithm but modified for finding the longest distances

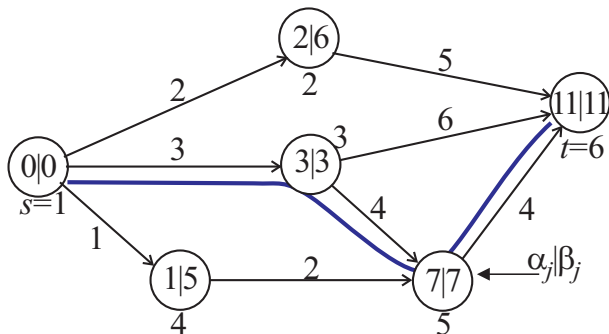
$$\alpha_1 = 0, \quad \alpha_j = \max_{i \in \text{before}[j]} \{\alpha_i + A[i, j]\} \text{ for } j = 2, \dots, n$$

- also the latest time β_j to leave node j to not violate the overall project duration is calculated, all activities after node (state) j must fit after leaving state j :

$$\beta_n = \alpha_n = CP \quad \beta_j = \min_{i \in \text{after}[j]} \{\beta_j - A[i, j]\} \text{ for } j = 1, \dots, n-1$$

~~assuming that node n is a node without successors.~~

Example



$$CP = 11$$

→ more CPM examples

- CPM uses only one time for an activity. PERT attempts modeling activity time variability by considering 4 different possible durations: optimistic (o), pessimistic (p), most likely (m) and expected= $(o + 4m + p)/6$.
- CPM and PERT do not take into account disruptions in project execution (times on arcs may be different in reality) and resource constraints (the same resource may be needed by several activities executed simultaneously).
- The above limitations are addressed in the so-called resource-constrained project scheduling and various activity duration modeling approaches (e.g. probabilistic and simulation).

D.All-pairs shortest paths

- First idea: apply n times Ford-Bellman algorithm by assuming all possible source nodes. Complexity: $O(n^4)$.
- Idea of Floyd⁵⁴ algorithm: This algorithm calculates distances $d_{ij}^{(k)}$ which are lengths of the shortest paths from node i to j with intermediate nodes in set $\emptyset \cup \{1, \dots, k\}$.
- These values can be calculated from recursive equations:⁵⁵

$$d_{ij}^{(0)} = a_{ij} \tag{F1}$$

$$d_{ij}^{(k+1)} = \min\{d_{ij}^{(k)}, d_{i,k+1}^{(k)} + d_{k+1,j}^{(k)}\} \tag{F2}$$
- Formula (F1) represents direct connections between i and j .

⁵⁴ there were also other discoverers: S.Warshall, B.Roy, ~1959-62.

⁵⁵ what is the minimum memory size required to tabulate $d_{ij}^{(n)}$?

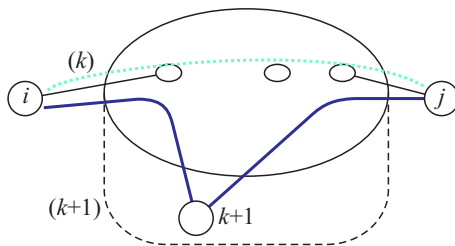
- Equation (F2) is justified as follows:

Consider the shortest path from i to j with the intermediate nodes in set $\{1, \dots, k, k+1\}$.

If node $k+1$ **is not** comprised in this path, then $d_{ij}^{(k+1)} = d_{ij}^{(k)}$

If node $k+1$ **is** comprised in this path, then it must have part from i to $k+1$ and from $k+1$ to j and we get

$$d_{ij}^{(k+1)} = d_{i,k+1}^{(k)} + d_{k+1,j}^{(k)}$$



Hence, the algorithm tabulating $d_{ij}^k, \forall i, j, k$ is correct.

Algorithm All-pairs shortest paths, Floyd algorithm.

Input: arc weight matrix $A[u, v]$ of size $n \times n$

Output: all-pairs shortest path lengths $D[i, j] = d_{i,j}^{(n)}$

1:begin

2: for $i := 1$ to n do

3: for $j := 1$ to n do $D[i, j] := A[i, j]$; (F1)

4: for $i := 1$ to n do $D[i, i] := 0$;

5: for $k := 1$ to n do

6: for $i := 1$ to n do

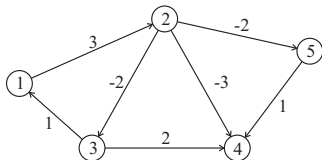
7: for $j := 1$ to n do

8: $D[i, j] := \min\{D[i, j], D[i, k] + D[k, j]\}$; (F2)

9:end

Computational complexity: $O(n^3)$

Example



$$A[i, j] = \begin{matrix} & \begin{matrix} \xrightarrow{j} \\ 0 & 3 & \infty & \infty & \infty \end{matrix} \\ \begin{matrix} \downarrow i \\ \infty \\ 1 \\ \infty \\ \infty \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & \infty & \infty \\ \infty & 0 & -2 & -3 & -2 \\ 1 & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 1 & 0 \end{bmatrix} \end{matrix}$$

$k = 0 \quad D[i, j] = A[i, j]$

$$k = 1 \quad \begin{bmatrix} 0 & 3 & \infty & \infty & \infty \\ \infty & 0 & -2 & -3 & -2 \\ 1 & 4 & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 1 & 0 \end{bmatrix}$$

$$k = 2 \quad \begin{bmatrix} 0 & 3 & 1 & 0 & 1 \\ \infty & 0 & -2 & -3 & -2 \\ 1 & 4 & 0 & 1 & 2 \\ \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 1 & 0 \end{bmatrix}$$

$$k = 3 \quad \begin{bmatrix} 0 & 3 & 1 & 0 & 1 \\ -1 & 0 & -2 & -3 & -2 \\ 1 & 4 & 0 & 1 & 2 \\ \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 1 & 0 \end{bmatrix}$$

For $k = 4, 5$ array $D[i, j]$ remains unchanged. In general lack of changes in two consecutive iterations does not mean that there is no changes in the later iterations. (Means that you have to compute matrices to the last iteration n).

Transitive closure of homogeneous binary relation

- Recall what is relation in general, binary relation over a countable finite sets U, V , transitive binary relation.
- Binary relation $A \subseteq V \times V$ is called *homogeneous* or *endorelation*.
- Homogeneous binary relation can be represented as a directed graph.
- A *transitive closure* for graph $G(V, A)$, or a transitive closure for homogeneous binary relation, is defined as

$$A^* = \{(x, y) : \text{exists a path in } G \text{ from } x \text{ to } y\}$$

.

→ give examples of applications

- Alternatively, a transitive closure A^* on homogeneous binary relation A can be defined as the smallest *transitive* relation that contains A . It means that for any transitive binary relation F such that $A \subseteq F$, we have that $A^* \subseteq F$.
- Calculation of transitive closure A^* (given A)
 1. Represent A as a graph $G(V, A)$, $\forall (i, j) \in A, w(i, j) = 1$.
// if needed set $\forall (i, j) \notin A, w(i, j) = \infty$
 2. Run Floyd algorithm.
 3. $(i, j) \in A^* \Leftrightarrow D[i, j] < \infty$.
- Complexity: $O(n^3)$.

- Alternatively, apply Warshall algorithm which runs on binary values:

$$A[i,j] := \begin{cases} 0 & \text{if } (i,j) \notin A \\ 1 & \text{if } (i,j) \in A \end{cases}$$

- In the Floyd algorithm change line 8 into:

$$D[i,j] := D[i,j] \vee (D[i,k] \wedge D[k,j])$$

- Then, as a result:

$$D[i,j] := \begin{cases} 1 & \Rightarrow (i,j) \in A^* \\ 0 & \Rightarrow (i,j) \notin A^* \end{cases}$$

On connection between transitive closure of homogeneous binary relation and matrix multiplication

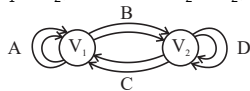
Problem of calculating transitive closure of homogeneous binary relation can be reduced to a product of two matrices.

Theorem

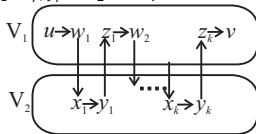
Given an algorithm that calculates product of two matrices of size $n \times n$ in time $M(n)$, it is possible to calculate transitive closure of homogeneous binary relation in time $T(n) \leq cM(n)$ for some constant c , if additionally $M(2n) \geq 4M(n)$.

Proof. (sketch)⁵⁶

- Assume that graph representing relation A has arcs with positive weights (e.g. all arc weights are 1) and a missing arc is represented as 0. Then lengths of all paths from node i to node j is positive if $(i, j) \in A^*$ and is equal to 0 if $(i, j) \notin A^*$.
- Let X be incidence matrix of homogeneous binary relation A (or the corresponding directed graph).
- Let us assume that size n of matrix X satisfies $n = 2^k$ for some positive integer k . In the opposite case empty rows and columns have to be appended to X to get matrix size equal to the nearest power of 2: $\begin{bmatrix} X & 0 \\ 0 & 1 \end{bmatrix}$.
- Finding transitive closure on a graph with n nodes (binary relation on n elements, matrix of size $n \times n$) may be reduced to finding transitive closure for graphs of size $n/2$ (relations on $n/2$ elements, matrices of sizes $\frac{n}{2} \times \frac{n}{2}$).
- To this end matrix X of size $2^k \times 2^k$ is divided into 4 matrices of size $2^{k-1} \times 2^{k-1}$: $X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$ where:
 A - arcs from V_1 to V_1 , B - arcs from V_1 to V_2 , C - arcs from V_2 to V_1 , D - arcs from V_2 to V_2 .

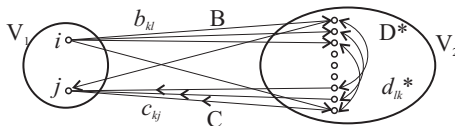


- For example, let us study if $(u, v) \in A^*$ where $v_1, v_2 \in V_1$
- If yes (i.e. $(u, v) \in A^*$), then there must be a path from u to v . This path is a) either not leaving V_1 , or b) $\exists k \geq 1, \exists w_i, x_i, y_i, z_i$ and $w_i, z_i \in V_1, x_i, y_i \in V_2$. The path can be visualized as in the figure:



- Assume that D^* (transitive closure for V_2 using only paths in V_2) is calculated, then:
 - $B \cdot D^* \cdot C$ - represents all all paths starting with one arc from V_1 to V_2 some number of arcs in V_2 and one arc from V_2 to V_1 , because

$$x_{ij} \in B \cdot D^* \cdot C = \sum_{k=1}^{\frac{n}{2}} \underbrace{\left(\sum_{\ell=1}^{\frac{n}{2}} b_{i\ell} d_{\ell k}^* \right) c_{kj}}_{B \cdot D^*}$$



- $A + B \cdot D^* \cdot C$ - represents all paths beginning with a direct connection in V_1 and the connections described in the above point "a");
 - $(A + B \cdot D^* \cdot C)^* = J$ - represents all paths between nodes of V_1 with arbitrary number of arcs in whole A .
- In an analogous way it can be shown that

$$E^* = \begin{bmatrix} J & J \cdot B \cdot D^* \\ D^* \cdot C \cdot J & D^* + D^* \cdot C \cdot J \cdot B \cdot D^* \end{bmatrix} = \begin{bmatrix} J & F \\ G & H \end{bmatrix}$$

- Matrices J, F, G, H can be calculated using the following procedure:
 $T_1 = D^*$; $T_2 = B \cdot T_1$; $J = (A + T_2 \cdot C)^*$; $F = J \cdot T_2$; $T_3 = T_1 \cdot C$; $G = T_3 \cdot J$; $H = T_1 + G \cdot T_2$.
- In this way transitive closure on a graph with n nodes (relation on n objects, $n \times n$ matrix) is reduced to finding transitive closure on two relations of size $\frac{n}{2}$, 6 multiplications of matrices of size $n \times n$ and 2 matrix additions.
- It can be shown that the above procedure (of substituting transitive closure with matrix multiplication and addition) applied to matrices of recursively reduced sizes has complexity $T(n) \leq c \cdot M(n)$, where $M(n)$ is complexity of $2 \times n \times n$ matrices multiplication, c - is some constant, provided that $M(2n) \geq 4M(n)$.



- Classic matrix multiplication algorithm has complexity $O(n^3)$, and Floyd/Warshall algorithms used for transitive closure also have complexity $O(n^3)$.
- So the above theorem on equivalence between transitive closure and matrix multiplication does not give any practical advantage, unless a matrix multiplication is possible below $O(n^3)$ complexity.

Algorithm multiplication of two 2×2 matrices, V.Strassen 1969**Theorem**

Product of two 2×2 matrices can be calculated in 7 multiplications and 18 additions/subtractions.

Proof.

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} =$$

$$\begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

can be calculated in the following way:

$$\begin{aligned} m_1 &= (a_{11} + a_{22})(b_{11} + b_{22}); & m_2 &= (a_{21} + a_{22})b_{11}; \\ m_3 &= a_{11}(b_{12} - b_{22}); & m_4 &= a_{22}(-b_{11} + b_{21}); & m_5 &= (a_{11} + a_{12})b_{22}; \\ m_6 &= (-a_{11} + a_{21})(b_{11} + b_{12}); & m_7 &= (a_{12} - a_{22})(b_{21} + b_{22}); \\ c_{11} &= m_1 + m_4 - m_5 + m_7; \\ c_{12} &= m_3 + m_5; & c_{21} &= m_2 + m_4; & c_{22} &= m_1 - m_2 + m_3 + m_6. \end{aligned}$$



- The above procedure can be applied to matrices of size $n \times n$, where $n = 2^k$ and k is some positive integer, by recursive division into 4 matrices of sizes $\frac{n}{2} \times \frac{n}{2}$ (the original matrices can be padded with columns and rows with 0s until necessary size)
- It can be shown that the above method has complexity $O(n^{\log_2 7}) \approx O(n^{2.807})$ by solving recursive equation on runtime:
 $T(n) = 7T(\frac{n}{2}) + 18(\frac{n}{2})^2$ for $n \geq 2$, $7T(\frac{n}{2})$ - multiplications;
 $18(\frac{n}{2})^2$ - addition/subtraction. The "7" in $O(n^{\log_2 7})$ results from the number of, multiplications.
- Fastest known matrix multiplication algorithms:⁵⁷
 $O(n^{2.375477})$ - Winograd-Coppersmith 1990,⁵⁸
 $O(n^{2.374})$ - Stothers 2010,
 $O(n^{2.3728642})$ - Vassilevska Williams 2011,
 $O(n^{2.3728639})$ - Le Gall 2014.

⁵⁷ what is the lower bound on complexity?

⁵⁸ disputable usability

Network Flows

Maciej Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

Maximum Network Flows

Natural applications of network flows

Field of application	arcs, nodes	flow
telecommunication	cables, routers, switches	data, voice, video
hydraulics	pipes, pumps, reservoirs	water, gas, oil
electric circuits	wires, junctions	electric current
mechanics	rods, beams, springs	heat, energy
transportation	highways, railroads, airline/shipping routes	people, cars, commodities

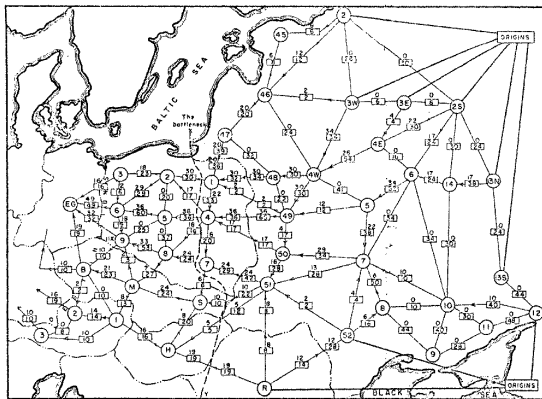


Figure 2

From Harris and Ross [1955]: Schematic diagram of the railway network of the Western Soviet Union and Eastern European countries, with a maximum flow of value 163,000 tons from Russia to Eastern Europe, and a cut of capacity 163,000 tons indicated as “The bottleneck”.

Acknowledgements. I thank Sasha Karzanov for his efficient help in finding Tolstov’s paper in the (former) Lenin Library in Moscow, Irina V. Karzanova for accurately providing me with an English translation of it, Kim H. Campbell and Joanne McLean at Air Force Pentagon for declassifying the Harris-Ross report, and Richard Bancroft and Gustave Shubert at RAND Corporation for their mediation in this.

Excerpt from: A.Schrijver, On the history of the transportation and maximum flow problems. Mathematical Programming 91, 437-445 (2002). <https://doi.org/10.1007/s101070100259>

Def. *Network* is a directed graph $G(V, A)$ without loops or parallel arcs, with distinguished source node $s \in V$ and sink node $t \in V$ (a.k.a. target, drain) and positive weights on arcs $\forall e \in A, c(e) \in \mathbb{N}_0$ referred to as capacities⁵⁹.

Def. *Network flow* $\Phi : e \in A \rightarrow \mathbb{N}_0$ (i.e. network flow is a function from arcs to positive numbers), satisfying conditions:

$$\forall e \in A, 0 \leq \Phi(e) \leq c(e), \quad (C1)$$

$$\sum_{e \in IN(i)} \Phi(e) - \sum_{e \in OUT(i)} \Phi(e) = \begin{cases} -F & \text{for } i = s \\ 0 & \text{for } i \in V \setminus \{s, t\} \\ F & \text{for } i = t \end{cases} \quad (C2)$$

where $IN(i)$ and $OUT(i)$ are sets of arcs arriving in node i and emanating from node i , respectively.

⁵⁹

In many sources weights are non-negative reals $\mathbb{R}$ but since reals do not have a simple computer representation we refrain to non-negative integers, all algorithms remain correct also for non-negative fractional numbers

Def. *Value of the flow* is number:

$$F = \sum_{e \in IN(t)} \Phi(e) - \sum_{e \in OUT(t)} \Phi(e)$$

Def. *Maximum flow problem*: find flow⁶⁰ Φ which maximizes value of the flow F .

Let S denote some set of nodes from V including $s \in S$ and $t \notin S$, and let $\bar{S} = V \setminus S$, i.e. $\bar{S}$ is a complement of S in V .

Def. An $s - t$ cut $[S, \bar{S}]$ is the set of arcs starting in S and ending in $\bar{S}$.⁶¹

Def. *Capacity of an $s - t$ cut*:⁶² $c[S, \bar{S}] = \sum_{e \in (S, \bar{S})} c(e)$

⁶⁰ since it is "flow" it satisfies conditions (C1),(C2)

⁶¹ It is called $s - t$ cut because $[S, \bar{S}]$ separates s from t .

⁶² i.e. capacity of an $s - t$ cut is the sum of forward arc capacities.

- ⁶³BTW, what about (C1),(C2) conditions? Does it work for other $s - t$ cuts?

Theorem (Ford, Fulkerson)

For any flow Φ with value of the flow F and any $s - t$ cut $F \leq c[S, \bar{S}]$.

- If $\exists [S, \bar{S}]$ such that $F = c[S, \bar{S}]$, then F is maximum value of flow and $[S, \bar{S}]$ is minimum capacity cut⁶⁴. Hence we have a theorem:

Theorem (Max-Flow Min-Cut Theorem)

The maximum value of the flow from a source s to a sink t equals the minimum capacity among all $s - t$ cuts.

⁶⁴ also called bottleneck

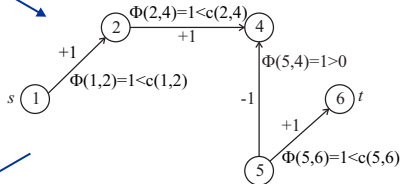
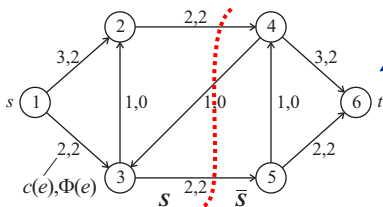
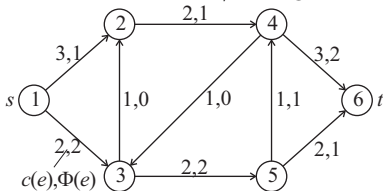
Def. *Residual network* for some given flow Φ is network $G(V, A)$ with the remaining arc capacities. For an arc (i, j) the remaining capacity is $c(e) - \Phi(e)$ but also $\Phi(e)$ upstream (i.e. in direction (j, i)) because arc flow can canceled.

Def. *Flow augmenting path*⁶⁵ P for some given flow Φ is a path from s to t which can be used to increase value of the flow. If arc $e \in P$ is directed from s to t , then it must satisfy $\Phi(e) < c(e)$, otherwise $\Phi(e) > 0$.

→ contraflows cancel existing flows

Example

Network and the initial/existing flow Φ



Augmenting path for flow Φ . Value of the flow may be increased by 1.

New flow. This is a maximum flow because $c[S, \bar{S}] = F = 4$.

- That's a bright idea☺! Just build the maximum flow by exploiting augmenting paths.
- But how to exploit all possible paths exhaustively, and do it effectively (i.e. build as few paths as possible)?
- Progress in algorithms for maximum network flows:⁶⁶
 - Ford, Fulkerson, 1962 - operated by finding and exploiting augmenting paths, pseudopolynomial (complexity dependent of capacity of min cut)
 - Edmond, Karp, 1972, $O(n^3 m)$ - used shortest paths
 - Dinic, 1970, $O(n^2 m)$ - applied layered⁶⁷ graph
 - Karzanow, 1974, $O(n^3)$
 - Czerkalski, 1976, $O(n^2 m^{\frac{1}{2}})$
 - Malhotra, Kumow, Makeswari, 1979, $O(n^3)$
 - Goldberg, Tarjan, 1988, $O(n^3)$

⁶⁶ $n = |V|$ - number of vertices, $m = |E|$ - number of arcs

⁶⁷ a.k.a. level graph

Dinic Algorithm

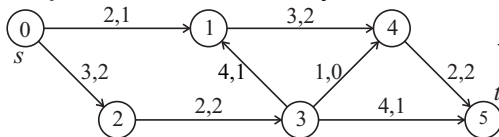
- The idea: a) the algorithm uses layered network comprising all shortest augmenting paths for the given flow Φ ,⁶⁸ b) parallel augmenting paths are built in the layered network thus increasing flow until blocking $s - t$ connection, c) repeat until there is no $s - t$ connection in the residual network.

Def. *Usable arc:* Arc $e = (u, v)$ conducting flow $\Phi(e)$ is *usable* from u to v , if one of the conditions hold: 1. $\Phi(u, v) < c(e)$; 2. $\Phi(v, u) > 0$.

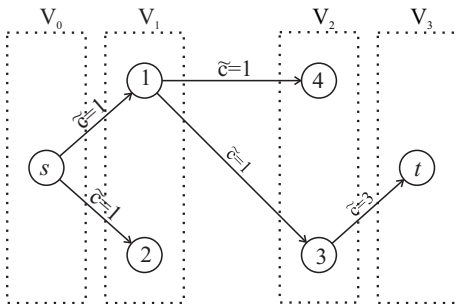
⁶⁸that is in the residual network for Φ

Algorithm 1. Layered network for $G(V, A)$ and flow Φ .

1. $V_0 := \{s\}, i := 0$.
2. $T := \{v : v \notin V_j \text{ for } j \leq i \text{ and there is a usable arc } (u, v) \text{ where } u \in V_i\}$.
3. If $T = \emptyset$, then STOP value of flow Φ is maximum.
4. If $t \in T$ then $\ell := i + 1, v_\ell := \{t\}$ and goto 6.
5. $V_{i+1} := T, i := i + 1$ and goto 2.
6. For $j = 1, \dots, \ell$ do
 $E_j := \{(u, v) : (u, v) \text{ is usable from } u \in V_{j-1} \text{ to } v \in V_j\}$.
 For each arc $(u, v) \in E_j$ do
 set residual capacity $\tilde{c}(u, v)$ and direction:
 (a) $u \in V_{j-1}, v \in V_j$ then $\tilde{c}(u, v) = c(u, v) - \Phi(u, v)$,
 (b) $v \in V_{j-1}, u \in V_j$ then $\tilde{c}(u, v) = \Phi(u, v)$, invert direction
 to (v, u) such that it points from V_{j-1} to V_j

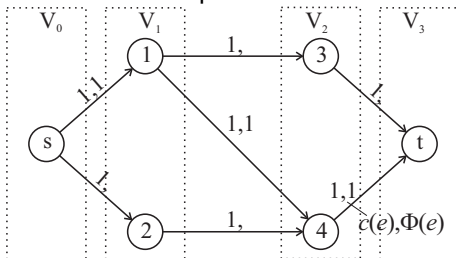
Example: construction of layered network⁶⁹

← Network and the initial/existing flow Φ



← layered network for Φ .

- In the layered network initial flow $\tilde{\Phi}(e) = 0$ is assumed for each arc e and augmenting paths are built until achieving maximum flow in the layered network.
- Maximum flow in the layered network means that for any path $v_0 = s, v_1, \dots, v_\ell = t$ where $\forall (v_{j-1}, v_j) \in E_j$, and $v_j \in V_j$, there is at least one arc e_j such that $\tilde{\Phi}(e_j) = \tilde{c}(e_j)$.
- Maximum flow in the layered network need not be maximum in the original network. For example



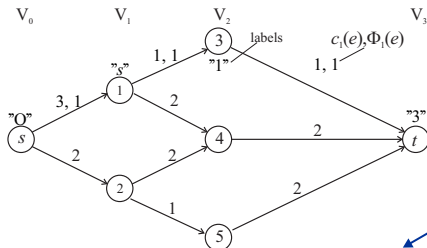
The obtained value of flow is $F = 1$ but maximum is $F = 2$.

Algorithm 2. Maximum flow in layered network (blocking flow, augmenting paths by assigning labels).

1. For each arc e the layered network set $\Phi_1(e) := \tilde{\Phi}(e) := 0; c_1(e) := \tilde{c}(e);$
2. Assign label "0" to vertex s ;
3. Assume that label is already assigned to some node $u \in V_j, j = 0, \dots, \ell - 1$, but is not assigned to any node from layer V_{j+1} . Assign label " u " to any node $v \in V_{j+1}$, such that there is arc $e = (u, v)$.⁷⁰ Repeat this step until:
 - a) either node t gets a label $\Rightarrow$ then goto 4;
 - b) or it is not possible to extend path to the next layer (i.e. there is no augmenting path from s to t) $\Rightarrow$ STOP because flow $\tilde{\Phi}$ is maximum in the current layered network.
4. Recover the augmenting path P by backtracking from t and using labels assigned to the nodes. $\Delta = \min_{e \in P} \{c_1(e)\}$; For each $e \in P$ do $\Phi_1(e) := \Delta, \tilde{\Phi}(e) = \tilde{\Phi}(e) + \Phi_1(e), c_1(e) := c_1(e) - \Delta$;⁷¹
5. Remove from the layered network arcs e such that $c_1(e) = 0$. Remove recursively vertices without in-arcs or out-arcs.
6. For each e remaining in the layered network $\Phi_1(e) := 0$; goto 2.

⁷⁰ labels play the role of an ad hoc stack in DFS

⁷¹ $\Phi_1(e)$ - augmenting path flow, $\tilde{\Phi}(e)$ - layered network flow

Example: augmenting paths in a layered network

← Initial layered network.

$$c_1(e) = \tilde{c}(e) \quad \Phi_1(e) = \tilde{\Phi}(e) = 0$$

Iteration 1

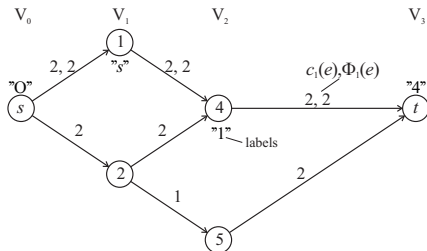
Augmenting path: $s, 1, 3, t$

$$\Delta := \min\{3, 1, 1\} = 1$$

for arcs e on path $s, 1, 3, t$:

$$\Phi_1(e) := \Delta = 1; \quad \tilde{\Phi}(e) = \tilde{\Phi}(e) + 1;$$

$$c_1(e) = c_1(e) - 1.$$



← Iteration 2

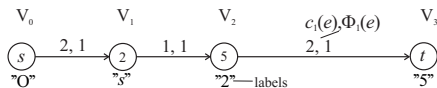
Augmenting path: $s, 1, 4, t$

$$\Delta := \min\{2, 2, 2\} = 2$$

for arcs e on path: $s, 1, 4, t$:

$$\Phi_1(e) := 2; \quad \Phi(e) := \Phi(e) + 2; \quad c_1(e) :=$$

$$c_1(e) - 2.$$



← Iteration 3.

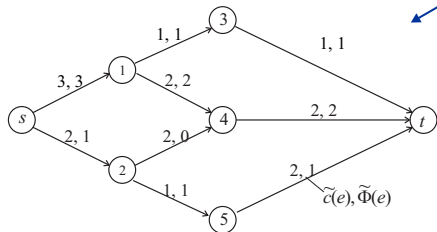
Augmenting path: $s, 2, 5, t$

$\Delta := \min\{2, 1, 2\} = 1$

for arcs e on path $s, 2, 5, t$:

$\Phi_1(e) := \Delta = 1; \tilde{\Phi}(e) = \tilde{\Phi}(e) + 1;$

$c_1(e) = c_1(e) - 1.$



← Maximum flow $\tilde{\Phi}$ in layered network

- The flow in layered network is added to the flows in the original network.
- Algorithms 1 (layered network) and 2 (parallel shortest paths, blocking flow) are repeated until no new layered network can be constructed. Then the flow is maximum.
- Hence, complete Dinic algorithm is as follows:

Algorithm Maximum network flows, Dinic, 1970

- 1 For all $e \in A$ do $\Phi(e) := 0$;
 - 2 Apply algorithm 1 to build layered network G_L for flow Φ . If layered network cannot be built, then STOP: flow Φ is maximum.
 - 3 Find maximum flow $\tilde{\Phi}$ in layered network G_L by use of algorithm 2.
 - 4 Update flow Φ in the original network:
for all arcs $u \in V_{j-1}, v \in V_j$ in the layered network do
 - (a) if arc $(u, v) \in A$ then $\Phi(e) := \Phi(e) + \tilde{\Phi}(e)$;
 - (b) if arc $(v, u) \in A$ then $\Phi(e) := \Phi(e) - \tilde{\Phi}(e)$;goto 2.
- Computational complexity: $O(n^2 m) \approx O(n^4)$
 - Do more examples.

From max-flow to min-cut

- The min-cut (bottleneck), i.e. cut $[S, \bar{S}]$ with minimum capacity $c[S, \bar{S}]$ can be found on the basis of maximum flow $\Phi(e)$.
- The saturated arcs of $[S, \bar{S}]$ separate source s from sink t .
- Thus, the network has to be searched from source s (e.g. by DFS, BFS), but only by traversing non-saturated arcs.
- The nodes which were accessible from s are in set S of min-cut, the remaining nodes are in $\bar{S}$ of min-cut. The arcs connecting S with $\bar{S}$ constitute the cut with minimum capacity $c[S, \bar{S}]$.

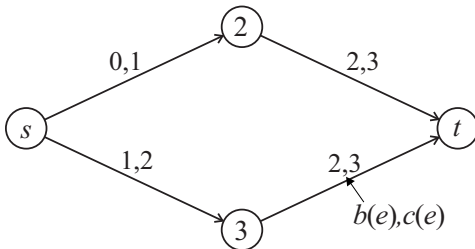
→ Do examples...

Arc flow lower bounds

Def. If an arc e has a nonzero lower bound $b(e)$ on the flow, then the arc flow must satisfy

$$b(e) \leq \Phi(e) \leq c(e)$$

Example:



- What is wrong with this example?

- Conclusion: there can be infeasible cases.
- The max-flow algorithms (like Dinic algorithm) start from zero flow. However, with arc flow lower bounds it is not always feasible.
- Thus, the max-flow problem with arc lower bounds can be solved in two steps:
 1. finding feasible flow;
 2. increasing flow to the maximum.
- The algorithm given below checks for a feasible flow and constructs it if such flow exists (step 1). It is done by solving max-flow in an equivalent substitute network $G' = (V', A')$ with $\forall e \in A', b(e) = 0$.
- Then, e.g., Dinic algorithm can be applied to build max-flow (step 2) if feasible exists.

Algorithm Feasible flow in $G(V, A)$ with lower bounds Ford, Fulkerson.

- 1 $A' := \emptyset, V' := \{s', t'\} \cup V$ // s', t' are new/replacement/substitute source and sink
- 2 for all $v \in V$ do: // $OUT(v)$ is the set of arcs emanating from v
 $A' := A' \cup \{(v, t')\}, c'(v, t') := \sum_{e \in OUT(v)} b(e), b'(v, t') := 0;$
- 3 for all $v \in V$ do: // $IN(v)$ is the set of arcs arriving into v
 $A' := A' \cup \{(s', v)\}, c'(s', v) := \sum_{e \in IN(v)} b(e), b'(s', v) := 0;$
- 4 for all $e \in A$ do:
 $A' := A' \cup \{e\}, c'(e) := c(e) - b(e), b'(e) := 0;$
- 5 $A' := A' \cup \{(t, s)\}, c'(t, s) := \infty, b'(t, s) := 0;$
- 6 Calculate max-flow Φ' in $G'(V', A')$.

Theorem

A feasible flow in the original network $G(V, A)$ exists if and only if maximum flow Φ' in $G'(V', A')$ saturates all arcs emanating from s' , i.e., $\forall e \in OUT(s'), \Phi'(e) = c'(e)$.⁷²

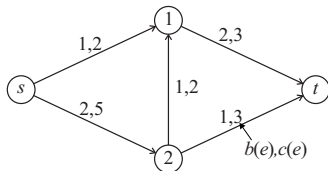
If Φ' satisfies the above condition, then a feasible flow Φ in the original network $G(V, A)$ can be calculated from formula:⁷³

$$\Phi(e) = \Phi'(e) + b(e)$$

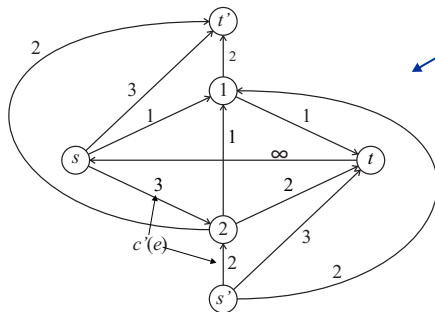
⁷² then are arcs in $IN(t')$ saturated?

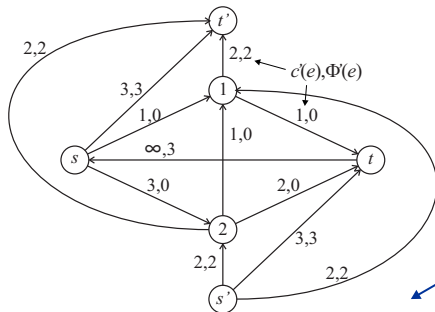
⁷³ BTW, this is feasible flow but not necessarily optimum, do not forget to about the 2nd step to maximize flow.

→ do the example from page 181

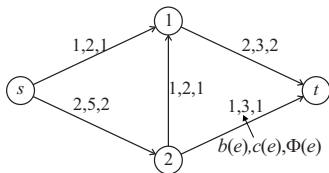
Example: feasible flow in a network with flow lower bounds

← Original network with arc flow lower bounds

← Replacement network $G'(V', A')$. Lower bounds $b'(e) = 0$ are omitted.



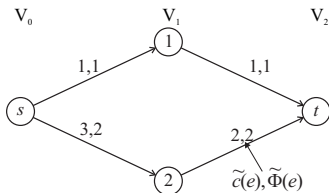
← Max flow in the replacement network
BTW, what is the relationship between
 $\sum_{e \in OUT(s')} \Phi'(e)$ and $\sum_{e \in IN(t')} \Phi'(e)$?



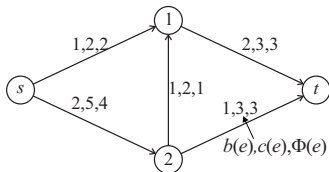
← Feasible flow in the original network.

- After finding a feasible flow, it should be further maximized.
- Dinic algorithm can be applied for this purpose, however, it must be modified to disallow reducing arc flows below the lower bounds. The necessary changes are as follows:
 - In the definition of usable arc usable from u to v (page 171) point 2.: $\Phi(e) > b(e)$.
 - In Algorithm 1 (layered network, page 172), point 6.(b):
 $v \in V_{j-1}, u \in V_j, \tilde{c}(u, v) = \Phi(u, v) - b(u, v)$.

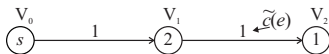
Example continued



← Layered network and its maximum flow



← Feasible flow in the original network.



← The next layered network. Sink t is unreachable, and hence, value of the flow is maximum.

Assignment of Distributed Application in Two-Processor System

- The computing platform consists of two processors P_1, P_2 connected by a communication link.
- The distributed application is represented as the so-called *task graph* with nodes representing tasks (modules, programs, processes) T_i for $i = 1, \dots, n$ and edges representing communications.
- Task graph is not directed and edges can be equivalently represented as a pair of edges in the opposite directions.
- Weight a_{ij} of edge $\{i, j\}$ is the cost of communication between tasks T_i and T_j (e.g. energy, monetary). If the sender and the receiver are on the same processor, then there is no communication and no related cost. We assume that $a_{ij} = a_{ji}$.
- Executing task T_i on processor P_j bears cost w_{ij} .

- Assign tasks to the two processors (i.e. split the task graph into two parts) such that the sum of communication and task execution costs is minimum.
- This problem can be solved by a reduction to network flow and finding minimum cut in the network.
- Equivalent network construction:
 - source s corresponds P_1 , sink t with P_2 ,
 - arcs between tasks T_i, T_j have capacities $c_{ij} = a_{ij}$,
 - $c_{si} = w_{2i}$ (cost of running T_i on P_2 , but source s is P_1)
 - $c_{it} = w_{1i}$ (cost of running T_i on P_1 , though sink t is P_2)
- Minimum cut in the above network is the minimum cost set of edges separating P_1 from P_2 . These edges must be cut for a minimum cost solution.
- Therefore, e.g., arc (s, i) has cost of running T_i on P_2 (not on P_1), because if we do not choose P_1 for running task T_i , then we will separate with the min-cut T_i from P_1 and bear cost w_{2i} of executing T_i on P_2 .

Example 1

w_{1j}	P_1	P_2
T_1	5	10

→do this example

Example 2

$$c_{1,2} = c_{2,1} = c_{1,3} = c_{3,1} = c_{2,3} = c_{3,2} = 1$$

w_{ij}	P_1	P_2
T_1	2	10
T_2	3	8
T_3	10	1

→do this example

Problem $P|pmtn, r_j|L_{max}$

- The above $P|pmtn, r_j|L_{max}$ means:
 - m parallel identical processors $P_1, \dots, P_m$,
 - n tasks $T_1, \dots, T_n$ which are preemptable (can be suspended and restarted without time cost), beyond this other assumptions are "standard": no precedence constraints, no extra resources required, ...
 - task T_j is defined by: processing time p_j , ready time r_j , *due-date* d_j ,
 - optimality criterion is maximum lateness:

$$L_{max} = \max_{j=1}^n \{c_j - d_j\},^{74} \quad (1)$$

where c_j is completion time of task T_j .

⁷⁴ note that $L_{max} < 0$ is possible. What does $L_{max} < 0$ mean?

- The algorithm for $P|pmtn, r_j|L_{max}$ is based on two key ideas/components:
 - A.** schedule feasibility testing for some trial value of L_{max} using network flow as an oracle;
 - B.** Determining the right trial values of L_{max} .

A. Network flow oracle for $P|pmtn, r_j|L_{max}$

- Let us denote by L the given test value of maximum lateness
- The requirement that $L_{max} \leq L$, by the above equation (1) is equivalent to $\forall T_j : c_j \leq d_j + L$.
- We will call $d'_j = d_j + L$ a *deadline* of T_j .
- Let all events in the system, i.e. all ready times r_j and deadlines d'_j be ordered non-decreasingly into a sequence of events $(e_1, \dots, e_\ell)$, where $e_1 < e_2 < \dots < e_\ell$.⁷⁵

- Oracle network construction:

→ set of nodes includes:

- source s , sink t ,
- vertices $T_1, \dots, T_n$ to represent the tasks,
- vertices $[e_1, e_2), \dots, [e_{\ell-1}, e_\ell]$ representing time intervals between events $e_1, \dots, e_\ell$.

→ set of arcs includes:

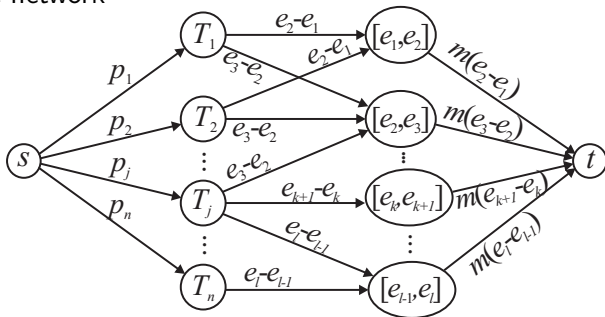
- $\forall T_j : (s, T_j)$ with capacity p_j ,
- $\forall T_j : (T_j, [e_k, e_{k+1}])$ with capacity $e_{k+1} - e_k$ if task T_j can be executed in interval $[e_k, e_{k+1}]$, i.e., $r_j \leq e_k, e_{k+1} \leq d'_j$,
- $\forall [e_k, e_{k+1}] : ([e_k, e_{k+1}], t)$ with capacity $m(e_{k+1} - e_k)$.

- Flow Φ saturating all arcs (s, T_j) indicates that a feasible schedule exists for the tasks and the test value L . The network flow oracle gives "yes" answer. Value of this flow is $F = \sum_{j=1}^n p_j$.

- If value of flow $F < \sum_{j=1}^n p_j$, then answer is negative.

- Computational complexity: $O((n + \ell))^3 \sim O(n^3)$ because $\ell \leq 2n$.

- Oracle network



- This network represents flow of work from tasks to time intervals.
- Capacities of arcs $(T_j, [e_k, e_{k+1}])$ and $([e_k, e_{k+1}], t)$ limit amount of work that can be done in interval $[e_k, e_{k+1}]$.⁷⁶

⁷⁶ Prove that a feasible schedule always exists in some interval $[e_k, e_{k+1}]$ provided these constraints hold. Hint: McNaughton algorithm.

B. Determining trial values of L_{max}

• It is possible to apply binary search over possible values of L_{max} . However, we do not know when to stop the search because L_{max} can be fractional. Hence, more advanced approach is proposed:

STEP 1:

• With the changes of trial values of L_{max} structure of the network oracle is changing because different sets of tasks compete for processors.

• This may happen when $d_j + L_{max} = r_i$ for some tasks T_j, T_i . Consequently, there are $O(n^2)$ such situations and network oracle has to be called $O(\log n)$ times in binary search over these values to fix the oracle network.

• After fixing structure of the network we remain with an interval of L_{max} values $[L_L, L_U]$ where value of the flow for L_L is $\Phi(L_L) < \sum_{j=1}^n p_j = P$ and answer is negative, and $\Phi(L_U) > P$ and answer is positive but L_U is not optimum.

STEP 2:

- In the fixed oracle network bottleneck capacity still changes with the changes of trial values of L_{max} .
- If arcs $(T_j, [e_k, e_{k+1}])$ are in the min-cut then capacity of the min-cut may increase by at most δ when L_{max} increases by δ .
- If arcs $([e_k, e_{k+1}], t)$ are in the min-cut then capacity of the min-cut may increase by at most $m\delta$. Since there are at most $2n$ such arcs, min-cut capacity may increase by at most $2mn\delta$.
- Since arbitrary combination of arcs may go into min-cut the capacity of the min-cut may increase by $\mu\delta$, where μ is a multiplier in range $1, \dots, 2nm$.
- The optimum increase of L_{max} above L_L can be found by binary search over multiplier μ values.

```

 $LB := 1, UB := 2mn, \mu = (UB + LB)/2;$ 
while  $UB - LB > 1$  do
   $\mu := (UB + LB)/2, \delta = (P - \Phi(L_L))/\mu;$  // how to fix  $\mu$  when
  //  $(UB + LB)/2$  is not integer?
  if  $\Phi(L_L + \delta) = P$  then STOP //  $L_L + \delta$  is optimum value of lateness
  else if  $\Phi(L_L + \delta) > P$  then  $LB = \mu$ 
  else  $UB = \mu$ 

```

- The above binary search requires $O(\log n + \log m)$ calls to network flow oracle.
- Together with step 1, whole algorithm has complexity $O(n^3(\log n + \log m))$.

Example

$$m = 3, n = 4,$$

$$\bar{p} = [2, 3, 4, 2] \text{ - vector of processing times,}$$

$$\bar{d} = [1, 2, 3, 1] \text{ - vector of due-dates,}$$

$$\bar{r} = [0, 0, 1, 1] \text{ - vector of ready times.}$$

Step 1: calculating test values for L_{max} when network structure changes from $\forall T_j, T_j : r_i = d_j + L_{max} \Rightarrow \forall T_j, T_j : L_{max} = r_i - d_j$.

$r_j \backslash d_j$	1	2	3	1
0	-1	-2	-3	-1
0	-1	-2	-3	-1
1	0	-1	-2	0
1	0	-1	-2	0

• Hence, L_{max} test values are: -3, -2, -1, 0 and intervals of fixed network structure can be written:

$$(-\infty, -3), [-3, -2), [-2, -1), [-1, 0), [0, \infty)$$

- The search for the correct L_{max} interval can be sped up by cutting of intervals which are infeasible because

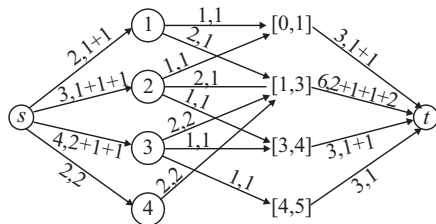
$$\forall T_j : r_j + p_j \leq d_j + L_{max} \Rightarrow L_{max} \geq \max_{j=1}^n \{r_j + p_j - d_j\}$$

$$L_{max} \geq \max\{2 - 1, 3 - 2, 5 - 3, 3 - 1\} = 2.$$

- Hence, test values must be greater than 2, and the only valid interval of L_{max} where network structure is fixed is $[2, \infty)$.

Step 2:

test value $L = 2 \rightarrow \overline{d'} = [3, 4, 5, 3]$ together with the ready times 0, 1 we get events: $e_1 = 0, e_2 = 1, e_3 = 3, e_4 = 4, e_5 = 5$.



← oracle network and augmenting paths

$s4[1, 3]t, \Delta = 2$

$s1[0, 1]t, \Delta = 1$

$s1[1, 3]t, \Delta = 1$

$s2[0, 1]t, \Delta = 1$

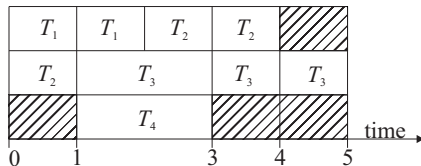
$s2[1, 3]t, \Delta = 1$

$s2[3, 4]t, \Delta = 1$

$s3[1, 3]t, \Delta = 2$

$s3[3, 4]t, \Delta = 1$

$s3[4, 5]t, \Delta = 1$



← Final schedule

Minimum cost flows

- We will be studying the flows with costs. Now each arc $e \in A$ is weighted not only by capacities but also by cost per unit of flow $a(e) \in \mathbb{Q}$, $a(e) \geq 0$.

Def. *Cost of flow Φ* is $K = \sum_{e \in A} a(e)\Phi(e)$

Def. *Minimum cost flow problem* consists in finding flow Φ of minimum cost K and value of flow equal to F .

Def. *Membership function* of arc (i, j) in path or cycle q

$$\delta_{ij}(q) = \begin{cases} 0 & (i, j) \notin q \\ 1 & (i, j) \in q \text{ and } (i, j) \text{ is directed according to } q \\ -1 & (i, j) \in q \text{ and } (i, j) \text{ is directed opposite to } q \end{cases}$$

Def. *Flow augmenting cycle:* cycle q is said to be augmenting flow Φ if it is possible to calculate flow $f(q) > 0$ satisfying

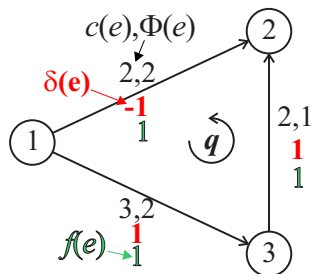
$$\forall (i, j) \in q : 0 \leq \Phi(i, j) + \delta_{ij}(q)f(q) \leq c(i, j) \quad (2)$$

Def. *Unitary cost of augmenting path or cycle P is*

$$\sum_{e \in P} a(e)\delta_e(P)$$

- Observe that the required flow Φ can be constructed as a superposition of path and cycle flows.
- Observe that flow $f(q)$ in augmenting cycle does not change net value of source-sink flow, but can change flow cost K by $f(q) \sum_{e \in q} a(e)\delta_e(q)$.

Example



← • Note that cycle $q = 1, 3, 2, 1$ is flow augmenting while cycle $q' = 1, 2, 3, 1$ is not, because no flow $f(q')$ satisfying equation (2) can be defined.

• Assume that $a(1, 2) > a(1, 3) + a(3, 2)$ then by circulating flow $f(q)$ we can reduce cost K of the flow.

Def. *Negative cost cycle* is an augmenting cycle⁷⁷ with negative unitary cost.

⁷⁷ flow $f(q) > 0$ exists

Theorem (Negative cycle optimality condition)

Flow Φ of value F is optimum with respect to cost if the residual network for Φ contains no negative cost cycle.

Theorem

Increasing optimum cost flow Φ with value F by value ε using the minimum-cost augmenting path results in constructing optimum cost flow Φ' with flow value to $F + \varepsilon$.

- The above two theorems give rise to two classes of algorithms solving minimum cost flow problem:⁷⁸
 - **primal** - first finding feasible flow with value F and then minimizing cost,
 - **dual** - increasing flow until reaching F while the intermediate solutions are cost-optimal.

⁷⁸ names of these types come from linear programming formulation of min-cost network flows, more on linear programming in the graph coloring section.

Remarks on primal algorithms

- After finding flow with the required value F , the minimum-cost flow may be obtained by canceling negative cost cycles. The negative cost cycles can be discovered by a reduction to calculating distances⁷⁹ in the residual networks and removing the negative cycles by circulating in them maximum flows.
- Care must be taken to prevent executing excessive number of cycle-canceling operations.
- For example, so-called network simplex algorithm uses the set of fundamental cycles for some given spanning tree.

⁷⁹ will be shown on the following slides

Dual algorithm

- *Observation:* If arc costs $\forall e \in A : a(e) \geq 0$, then flow $\forall e \in A : \Phi(e) = 0$ contains no negative cost cycles.
- Thus, a dual algorithm can start from zero flow.
- *Observation:* Cost minimization may be reduced to shortest paths problem by transformation of cost to arc (i, j) generalized distances $\bar{a}_{ij}$:

$$\bar{a}_{ij} = \begin{cases} -a_{ji} & \text{if } \Phi_{ij} \leq c_{ij} \wedge \Phi_{ji} > 0 \\ a_{ij} & \text{if } \Phi_{ij} < c_{ij} \wedge \Phi_{ji} = 0 \\ +\infty & \text{if } \Phi_{ij} = c_{ij} \wedge \Phi_{ji} = 0 \\ +\infty & \text{if } (i, j) \notin A. \end{cases}$$

- Line 1: canceling contraflow Φ_{ji} on arc (j, i) reduces cost by a_{ji} per unit of flow.
- Line 2: increasing flow Φ_{ij} on arc (i, j) increases cost by a_{ij} per unit of flow.
- Line 3 and 4: connection between i and j cannot be used in an augmenting path.
- *Observation:* The shortest $s - t$ path calculated using the generalized distances is also the minimum-cost augmenting path.
- *Observation:* Generalized distances $\bar{a}_{ij}$ must be calculated dynamically because they depend on flow Φ and usually are asymmetric, i.e., $\bar{a}_{ij} \neq \bar{a}_{ji}$.

Algorithm Minimum cost flows, Jewell 1958, Busacker, Gowen 1961, Klein 1967

1. Let Φ be an optimum flow with value $F' < F$; // Φ may be a zero-flow.
2. while ($F' < F$) do
3. Calculate generalized distances $\bar{a}_{ij}$;
4. Apply shortest paths algorithm to find the shortest $s - t$ path using generalized distances $\bar{a}_{ij}$;
5. If there is no $s - t$ path STOP; // flow with value F does not exist
6. Increase flow Φ along the shortest $s - t$ path by ϵ such that $F' + \epsilon \leq F$ and Φ remains feasible; // capacities not exceeded

- *Computational complexity*: If one starts from zero-flow and arc capacities are non-negative integers, then flow will be increased at most F times. Each flow augmentation requires shortest distance calculation in $O(n^3)$ time. Hence, complexity of this algorithm is $O(Fn^3)$.
- *Observation*: this is a pseudopolynomial algorithm.⁸⁰
- Polynomial and strongly polynomial⁸¹ algorithms by Edmonds, Karp, Röck, Orlin, Tardos are also known. Polynomial algorithm will be outlined after transportation problem (see in the following).

⁸⁰ i.e. exponential-time! like dynamic programming algorithm for knapsack.

⁸¹ strongly-polynomial – complexity depends on network structure elements like number of arcs m and nodes n but does not depend on numerical parameters like value of flow F , capacities, costs.

Transportation problem

- In the transportation problem flow conservation condition (C2, page 164) has a different formulation:

$$\forall i \in V : \sum_{e \in IN(i)} \Phi(e) - \sum_{e \in OUT(i)} \Phi(e) = b_i \quad (C2')$$

$$b_i = \begin{cases} < 0, & \text{then node } i \text{ is a supply node;} \\ > 0, & \text{then node } i \text{ is a demand node;} \\ = 0, & \text{then node } i \text{ is a transshipment node.} \end{cases}$$

- It is required to find minimum-cost flow subject to feasibility constraints (C2') (but condition (C1) is dropped!).
- Note that for feasibility additionally the supply must be at least equal to the demand:

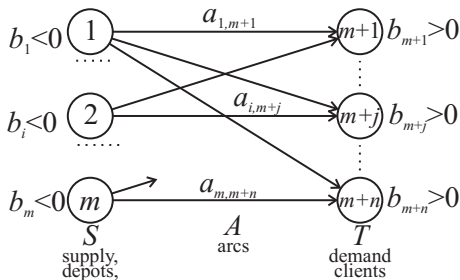
$$\sum_{\{i: b_i < 0\}} |b_i| \geq \sum_{\{i: b_i > 0\}} b_i = F$$

- Instances of the transportation problem can be transformed to minimum-cost network flow instances, and vice versa a standard minimum-cost network flow instance can be transformed to a transportation problem instance.
- The differences between the two formulations:

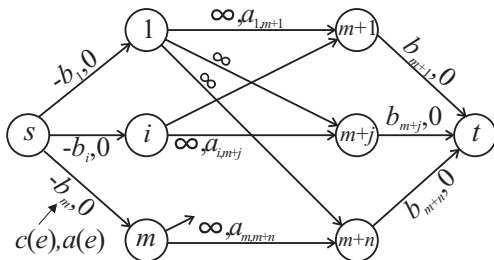
network object	min-cost net.flow	transportation problem
source, sink	one s , one t	many
node capacities	$b_i = 0$	arbitrary b_i
arc capacities	limited	unlimited
arc costs	non-negative	non-negative

Transportation problem $\rightarrow$ *minimum cost network flow*

- Consider Hitchcock-Koopmans transportation problem formulation in a bipartite graph $G = (S, T, A)$. Remember that $c(e) = \infty$.



- Transformed network



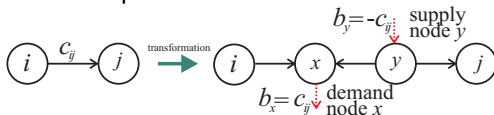
- Details of the transformation:

- Source s is introduced and connected with supply nodes i by arc (s, i) with capacity $c(s, i) = -b_i$ and cost $a_{si} = 0$.
- Sink t is introduced and connected with demand nodes j by arc (j, t) with capacity $c(j, t) = b_j$ and cost $a_{jt} = 0$.
- Arcs $(i, m+j)$ get capacities $c(i, m+j) = \infty$ and their original cost, where $i, j \notin \{s, t\}$.

- In the transformed network standard flow conservation rule is obeyed. Algorithms finding minimum cost flow of value F (total demand) also solve the transportation problem.

Minimum cost network flow $\rightarrow$ transportation problem

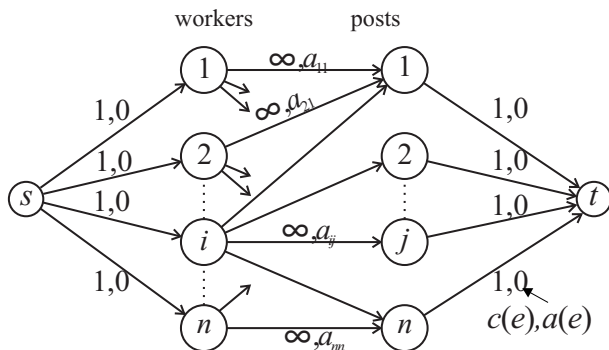
- Note that except for limited arc capacities, minimum cost flows are like transportation problems.
- Hence, it is necessary to replicate limited arc capacities by some other means present in the transportation problem.
- Elimination of arc capacities:



- Final remark: we reduced solving transportation problem to solving minimum cost flow problem, but algorithms dedicated to transportation problems also exist. Actually polynomial algorithms are formulated rather for transportation than minimum-cost problems.

Assignment problem

- Given n workers, n posts, cost a_{ij} of assigning worker i to position j , find the cheapest assignment of workers to positions such that each worker has one position and each position has one worker, i.e. find cheapest perfect matching between workers and posts.
- Assignment problem may be solved by constructing minimum flow in the following network:



- Minimum-cost integer-valued⁸² flow with value F determines solution of the assignment problem.
- A successive shortest path algorithm (as shown before) is polynomial because value of flow is $F = n$, hence its complexity here is $O(n^4)$.
- Dedicated algorithm exists with complexity $O(n^3)$ – a variant of the so called Hungarian method.⁸³

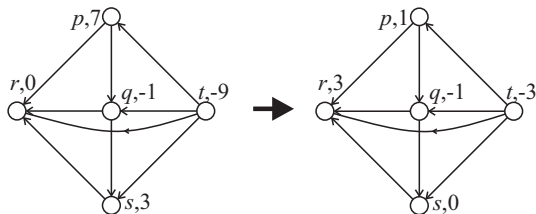
⁸²if we use augmenting path capacities then the flow will be integer-valued

⁸³this algorithm was discovered first by C.G.Jacobi in 1890, then in 1955, and improved in later years. ▶

Edmonds and Karp scaling algorithm⁸⁴

- Assume that we are about to solve transportation problem $(G, \bar{a}, \bar{b})$, where $G(V, A)$ is transportation network, $\bar{a}$ is arc cost vector, $\bar{b}$ is node demand/supply vector.
- We assume that G comprises node r such that $r \in V$, $b_r = 0$, $\forall_{v \in V} (r, v) \notin A, (v, r) \in A, a_{vr} = 0$.
- Node r with such features can be added to G . The purpose of node r is to guarantee feasibility of the instance, i.e. guaranteeing that the sum of demands is equal to the sum of supplies in the course of scaling algorithm operations. In the final solution node r is immaterial because $b_r = 0$ (that is, it is neither providing nor consuming any flow).

- **Def** *Scaling of instance $(G, \bar{a}, \bar{b})$ by Δ* consists in substituting node flow balances b_i with $b'_i = \lfloor b_i/\Delta \rfloor$ for $i \neq r$ and setting $b'_r = -\sum_{i \in V \setminus \{r\}} b'_i$.
- Example of scaling by $\Delta = 4$



- Let $(G, \bar{a}, \bar{b}, k)$ denote instance $(G, \bar{a}, \bar{b})$ scaled by 2^k , for $k \geq 0$.
- Note that (G, a, b, k) is obtained by removing k least significant bits from b_i , and that $b^0 = b$.
- Assume that optimum flow Φ' is known for $(G, a, b, k + 1)$.
Then $\Phi = 2\Phi'$ is a feasible flow for (G, a, b^k) .
- In the optimum flow for (G, a, b, k) : $b_i^k = 2b_i^{k+1}$ or $b_i^k = 2b_i^{k+1} + 1$ for $i \in V \setminus \{r\}$ and $b_r^k \leq 2b_r^{k+1}$ for node r .
- Hence, the number of nodes which demands are not satisfied is not greater than $n - 1$ and whole lacking flow value is not greater than $n - 1$.
- If we apply minimum cost flow algorithm with starting flow Φ , then at most $n - 1$ calls to the algorithm constructing the shortest (i.e. cheapest) path will be made to solve (G, a, b, k) .

Scaling algorithm for transportation problem Edmonds, Karp 1972

flow $MCF(G, \bar{a}, \bar{b}, k, \phi)$ – function returning minimum cost flow for (G, a, b, k) starting with initial flow ϕ , uses the shortest paths algorithm for the cheapest paths.

- 1 $K = \max_{i \in V} \{\lceil \log_2 |b_i| \rceil\};$
- 2 $\Phi := 0;$
- 3 for $k := K$ down to 0 do
- 4 $\Phi = MCF(G, \bar{a}, \bar{b}, k, 2\Phi)$

- Let (G, a, b) be a feasible instance of transportation problem, with n nodes, m arcs and node flow balances $b_i \in \mathbb{N}$.

Theorem

Edmonds-Karp algorithm solves (G, a, b) in time $O(nS(n, m)(1 + \log F))$, where $S(n, m)$ - is the complexity of determining the shortest path in a network with n nodes and m arcs, $F = \sum_{i \in V} |b_i|$.⁸⁵

⁸⁵ Complexity is $O(mS(n, m)(1 + \log F))$ if it is necessary to convert minimum cost network flow instance to transportation problem (because each arc may require 2 nodes to simulate limited capacities).

Bipartite graph matching

Maciej Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

Bipartite graph matching

- Plan for graph matching
 - 1 Definitions
 - 2 Augmentation theorem for general graphs
 - 3 Bipartite graphs
 - Definitions
 - Theorems
 - Alternative formulations
 - Labeling algorithms
 - Hopcroft-Karp algorithm
 - Applications

Matching - general definitions

Def. Matching in graph $G(V, E)$ is a set of edges $M \subseteq E$ such that no pair of edges in M share a vertex. →example

Def. Maximum matching problem: find matching M in G with the greatest cardinality $|M|$.

Def. In perfect matching every vertex of V coincides with one edge of M , consequently also $|M| = |V|/2$.

Let us assume that there is some matching M (possibly empty, i.e. $M = \emptyset$) in G .

Def. Edge e is matched in M if $e \in M$, alternatively: e is covered/ saturated/ matching/ used in M .

Def. Edge e is unmatched in M is $e \notin M$, alternatively: e is not covered by/ does not belong to/ not saturated in/ M .

Def. Some vertex v is matched/ covered/ saturated/ in M if it is incident edge $e \in M$.

Def. Some vertex v is not matched or is **free** in M when it is not incident to $e \in M$.

Alternating path P in M is a simple path⁸⁶ whose edges are alternately covered and not covered in M . →example

Augmenting path in M is an alternating path beginning and ending in two different free vertices. (different nodes → it cannot be a cycle). →example

⁸⁶ simple path=all nodes/edges are different

Observe that:

- augmenting path P has odd number of edges,
- changing the matched edges in P into the unmatched edges, and vice versa, the unmatched edges into matched edges increases the number of matched edges,
- such a change of edge matching status will be called a **symmetric difference** of two sets M, P of edges and denoted $M \oplus P$.
- augmenting paths can be used to find maximum matching (but how?).

Augmentation theorem - arbitrary graphs

Theorem (Augmentation theorem, Berge 1957)

If M is not a maximum matching in G , then there is an augmentation path P , such that $M' = M \oplus P$ is a matching in G and $|M'| = |M| + 1$.

Proof (outline)

1. Assume M is maximum and an augmenting path P exists $\Rightarrow$ increase matching to $M' = M \oplus P \Rightarrow M$ was not maximum matching.
2. Assume that M^* is not a maximum cover and $|M| < |M^*|$. Calculate $H = M^* \oplus M$. Since $|M| < |M^*|$, there is at least one alternating path P in $M^* \oplus M$ beginning and ending with edge(s) in M^* . P is a path augmenting M .



Bipartite graphs - definitions

Def. A **bipartite graph** $G = (X, Y, E)$ consists of set $X \cup Y$ of nodes and set E of edges, such that $\forall \{u, v\} \in E : u \in X, v \in Y$.

- Observe that assignment of nodes to sets X, Y need not be unique, that is can be done in many ways.
- In the following we assume, that $n = |X| \leq |Y|$ and $|E| = m$.
- Let $N(i)$ denote set of node i neighbors.
Let $N(J)$ denote the set of neighbors for set J of nodes, i.e.
 $N(J) = \cup_{i \in J} N(i)$.

Bipartite graphs - theorems

Theorem (Hall's condition, Hall, 1935)

For a bipartite graph $G = (X, Y, E)$ it is possible to match every vertex of X with a vertex of Y if and only if for all subsets $J \subseteq X$:

$$|J| \leq |N(J)|$$

Proof (outline)

It is a necessary condition because otherwise matching of cardinality $|J|$ cannot exist. It will be shown by induction over the size of set X that it is also a sufficient condition.

1. $|X| = 1 \rightarrow$ choose any edge.

2. Assume that this theorem is satisfied for all graphs with $|X| = k$. Consider graph $G' = (X', Y', E')$ satisfying Hall's conditions, where $|X'| = k + 1$.

2a. $|X'| < |N(X')| \Rightarrow$ match any vertex $i \in X'$ with $j \in N(i)$. Remove i, j and all edges incident with i, j . Every vertex has lost at most one neighbor. Hence, the resulting graph $G'' = (X'', Y'', E'')$ is still satisfying Hall's conditions and has $|X''| \leq k$. Therefore matching must exist.

2b. $|X'| = |N(X')| \Rightarrow$ match any node $i \in X'$ with an *appropriate* node $j \in N(i)$ (appropriate = there must be such a vertex). Remove i, j and all edges incident with i, j . If the resulting graph $G'' = (X'', Y'', E'')$ with $|X''| = k$ is not satisfying Hall's conditions, then also graph G' was not satisfying them, which contradicts the earlier assumption.



Note that Hall's theorem has little utility from the algorithms point of view ... actually why?⁸⁷

⁸⁷ How many sets J have to be checked?

Vertex cover⁸⁸ given graph $G(V, E)$ (not necessarily bipartite)
vertex cover is set of nodes $V' \subseteq V$ such that each edge is incident
with at least one node in V' . →examples

Theorem (König, 1916)

In a bipartite graph the size of the minimum vertex cover C is equal to the size of the maximum matching M :

$$\min_{C \text{ is vertex cover in } G} |C| = \max_{M \text{ is matching in } G} |M|$$

→examples

⁸⁸See also page 47

Theorem

Hall's and König's theorems are equivalent.

Proof (outline)

1. Hall's theorem $\Rightarrow$ König's theorem:

- By Hall's theorem there is matching M in $G(X, Y, E)$ covering all nodes of X , that is $|M| = |X|$.
- $\exists$ matching $M \Rightarrow$ exists cover $C_1 = X$: $|C_1| = |M|$, but possibly also some other cover $|C| < |M|$ can exist.
- We will show that $|X|$ is minimum size of any vertex cover.
- Any cover C comprises some vertices from X and some vertices from Y .
- Vertices in $Y \cap C$ cover edges incident with $X' = X - C \neq \emptyset$.
- $N(X') \subseteq Y \cap C$ because otherwise C is not a vertex cover. And $|N(X')| \geq |X'|$ by Hall's theorem.
- Hence, cardinality of any cover C must satisfy $|C| = |X \cap C| + |Y \cap C| \geq |X \cap C| + |N(X')| \geq |X \cap C| + |X'| = |X| = |M|$.

2. König's theorem $\Rightarrow$ Hall's theorem

- Generally for any matching M and vertex cover C , $|M| \leq |C|$. It is necessary to show that for any cover C there is a matching of size $|M| = |C|$.
- Let C be minimum cover.
- Construct $G' = (X', Y', E') : X' = X \cap C, Y' = Y - C, E' = E \cap (X' \times Y')$.
- Assume Hall's conditions are not satisfied in G' , then there must be a set of nodes $W \subseteq X'$ such that $|W| > |N(W)|$.
- Construct a new cover: $C' = (X' - W) \cup N(W) \cup (Y \cap C)$. This is correct cover because every edge in G' is covered by a vertex from $X' - W$ (for them there is matching in G'), or by a vertex from $N(W)$ (for them there is no matching in G'), the remaining edges are covered by $Y \cap C$.
- Cardinality of C' is $|C'| = |(X' - W)| + |N(W)| + |Y \cap C| < |X' - W| + |W| + |Y \cap C| = |X \cap C| + |Y \cap C| = |C|$.
- That is $|C'| < |C|$, which contradicts the assumption that C is minimum.
- Conclusion: Hall's conditions must be satisfied: every vertex in $w \in X \cap C$ can be matched with a vertex in $Y - C$. and a vertex in $Y \cap C$ can be matched with a vertex in $X - C$. Hence, there must be matching M of size $|C|$.

Equivalent formulations

Marriage problem

Set X represents gentlemen, set Y represents ladies, set E of edges represent who knows whom. Find the biggest set of marriages.⁸⁹

Systems of different/distinct representatives

Let $(Y_1, Y_2, \dots, Y_n)$ be a sequence of not necessarily disjunctive sets. We call sequence $(u_1, \dots, u_n)$ such that $\forall i : u_i \in Y_i, \forall i \neq j : u_i \neq u_j$ a system of distinct/different representatives for $(Y_1, Y_2, \dots, Y_n)$.

Commission problem

Consider some representative body which elects some commissions (e.g. parliament). A chairman of one commission is not allowed to preside any other commission. There can be only once chairman of each commission. Given sets of commissions members, find the feasible chairmen.

⁸⁹ bigamy is, of course, disallowed.

- Commission and distinct representatives problems can be reduced to constructing matching in a bipartite graph:

- Assume that:

$X = \{1, \dots, n\}$ is a set of commission chairmen positions,
 $Y_i = N(i)$ is a set of representatives, delegates, members of parliament belonging to commission i ,

$$Y = \bigcup_{i=1}^n Y(i), E = \{\{i, j\} : i \in X, j \in Y_i\}.$$

This problem is represented as bipartite graph $G = (X, Y, E)$.

- A matching of size $|X| = n$ in G is also an assignment of delegates to chairmen positions for $(Y_1, Y_2, \dots, Y_n)$. →picture

Binary matrix cover problems⁹⁰

For binary matrix A find:

Problem 1: minimum set C of rows and columns such that all 1s in A are covered. Some entry in A is covered if it is in the chosen column/row.

Problem 2: maximum set M of entries $a_{ij} = 1$ such that each one is in a different row and column. →examples

• If we interpret rows of A as elements of set X , columns of A as elements of set Y , and entries in A like edges:

$a_{ij} = 1 \Rightarrow \{i, j\} \in E$, $a_{ij} = 0 \Rightarrow \{i, j\} \notin E$, then:

- problem 1 defines a $G = (X, Y, E)$ vertex cover,
 - problem 2 defines a matching in $G = (X, Y, E)$,
- which are equivalent in the sense of König theorem.

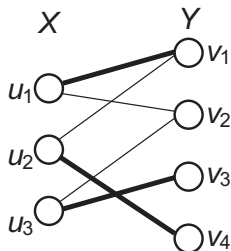
Reduction to max-flow

Given graph $G = (X, Y, E)$ construct an equivalent network $S(G) = (V, A)$:

- $V = \{s, t\} \cup X \cup Y$,
- $A = \{(s, i) : i \in X\} \cup \{(j, t) : j \in Y\} \cup \{(i, j) : \{i, j\} \in E\}$,
- arc capacities: $\forall (i, j) \in A : c(i, j) = 1$.

Example

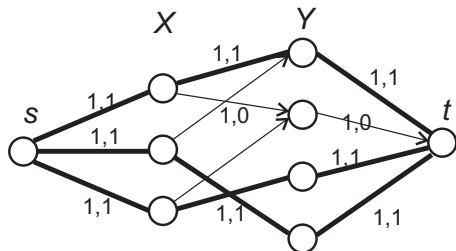
$$G = (X, Y, E)$$



$$M = \{\{u_1, v_1\}, \{u_2, v_3\}, \{u_3, v_4\}\}$$

$$|M| = 3$$

$$S(G)$$



value of the flow $F = 3$.

Theorem

There is one-to-one correspondence between matching M in G and 0-1 flows in $S(G)$.

- *For matching $M = \{\{u_1, v_1\}, \dots, \{u_k, v_k\}\}$ there is flow in $S(G)$: $\Phi(s, u_i) = \Phi(u_i, v_i) = \Phi(v_i, t) = 1$ for $i = 1, \dots, k$ and $\Phi(a, b) = 0$ on the remaining arcs of $S(G)$.*
- *For flow Φ in $S(G)$ of value k there is a matching $M = \{\{u, v\} : u \in X, v \in Y, \Phi(u, v) = 1\}$ of cardinality k .*

- It can be concluded that maximum matching in a bipartite graph can be found by using, e.g., Dinic algorithm in time $O(n^4)$.
- Note that requiring binary values of arc flows in Φ is crucial here.

Bipartite graphs - labeling algorithm

- Let M be a matching in $G = (X, Y, E)$, while $M = \emptyset$ is admissible.
- Let $L \subseteq X$ be a set of free nodes with respect to M .
- Let $l[u]$ be a label of vertex u , where initially $l[u]$ s are all empty.

Labeling_algorithm(G, M)

calculate L ; remove all labels $l[]$; $R := \emptyset$;

while $L \cup R \neq \emptyset$ {

 choose vertex u from $L \cup R$;

if $u \in X$ scan_left(u) **else** scan_right(u);

}

```

scan_left( $u$ ) {
 $L := L - \{u\}$ ;
while there is edge  $\{u, v\}$  with  $v$  unlabeled {
    label  $v$  with  $l[v] := u$ ; // remember where from we arrived into  $v$ 
     $R := R \cup \{v\}$ ;
} // end of while
} // end of scan_right

```

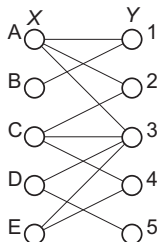
```

scan_right( $u$ ) {
 $R := R - \{u\}$ ;
if there is edge  $\{v, u\} \in M$  where  $v$  has no label {
    label  $v$  with  $l[v] := u$ ; // remember where from we arrived into  $v$ 
     $L := L \cup \{v\}$ ;
} else { // there is an augmenting path  $M$ 
    backtrack using labels to reconstruct alternating path:  $P = (\dots, l[l[u]], l[u], u)$ ;
     $M := M \oplus P$ ;
    calculate new  $L$  for  $M$ ;  $R := \emptyset$ ; remove all labels  $l[]$ ;
} // end of scan_left

```

→example

Example



We assume that we choose from $L \cup R$ according to LIFO (stack) order, elements with smaller number or alphabetically earlier are inserted in L, R earlier.

Initially $L \cup R = (A, B, C, D, E)$.

Pop E , $\text{scan_left}(E)$ pushes $R := (3, 4)$, $\text{scan_right}(4)$ makes matching $\{E, 4\}$. Restart. $L \cup R = (A, B, C, D)$.

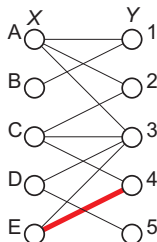
Pop D , $\text{scan_left}(D)$ pushes $R := (3, 5)$, $\text{scan_right}(5)$ makes matching $\{D, 5\}$. Restart. $L \cup R = (A, B, C)$.

After $\text{scan_left}(C)$ we have $L \cup R = (A, B, 2, 3, 4)$, $\text{scan_right}(4)$ pushes vertex E to L , $\text{scan_right}(E)$ goes deeper no more because 3,4 already have labels. $\text{scan_right}(3)$ adds $\{C, 3\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}\}$. Restart. $L \cup R = (A, B)$.

$\text{scan_left}(B)$ pushes vertex 1 to R , $\text{scan_right}(1)$ adds $\{B, 1\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}, \{B, 1\}\}$. Restart. $L \cup R = (A)$.

$\text{scan_left}(A)$ pushes into $L \cup R = (1, 2, 3)$. After $\text{scan_right}(3)$ we have $L \cup R = (1, 2, C)$. After $\text{scan_left}(C)$ we have $L \cup R = (1, 2, 4)$ because 2,3 already have labels. $\text{scan_right}(4)$ gives $L \cup R = (1, 2, E)$. $\text{scan_left}(E)$ is not going deeper any further because 3,4 have labels, we remain with $L \cup R = (1, 2)$. $\text{scan_right}(2)$ adds $\{A, 2\}$ to the matching. Restart. $L \cup R = \emptyset$. Finish.

Example



We assume that we choose from $L \cup R$ according to LIFO (stack) order, elements with smaller number or alphabetically earlier are inserted in L, R earlier.

Initially $L \cup R = (A, B, C, D, E)$.

Pop E , $\text{scan_left}(E)$ pushes $R := (3, 4)$, $\text{scan_right}(4)$ makes matching $\{E, 4\}$. Restart. $L \cup R = (A, B, C, D)$.

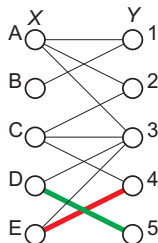
Pop D , $\text{scan_left}(D)$ pushes $R := (3, 5)$, $\text{scan_right}(5)$ makes matching $\{D, 5\}$. Restart. $L \cup R = (A, B, C)$.

After $\text{scan_left}(C)$ we have $L \cup R = (A, B, 2, 3, 4)$, $\text{scan_right}(4)$ pushes vertex E to L , $\text{scan_right}(E)$ goes deeper no more because 3,4 already have labels. $\text{scan_right}(3)$ adds $\{C, 3\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}\}$. Restart. $L \cup R = (A, B)$.

$\text{scan_left}(B)$ pushes vertex 1 to R , $\text{scan_right}(1)$ adds $\{B, 1\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}, \{B, 1\}\}$. Restart. $L \cup R = (A)$.

$\text{scan_left}(A)$ pushes into $L \cup R = (1, 2, 3)$. After $\text{scan_right}(3)$ we have $L \cup R = (1, 2, C)$. After $\text{scan_left}(C)$ we have $L \cup R = (1, 2, 4)$ because 2,3 already have labels. $\text{scan_right}(4)$ gives $L \cup R = (1, 2, E)$. $\text{scan_left}(E)$ is not going deeper any further because 3,4 have labels, we remain with $L \cup R = (1, 2)$. $\text{scan_right}(2)$ adds $\{A, 2\}$ to the matching. Restart. $L \cup R = \emptyset$. Finish.

Example



We assume that we choose from $L \cup R$ according to LIFO (stack) order, elements with smaller number or alphabetically earlier are inserted in L, R earlier.

Initially $L \cup R = (A, B, C, D, E)$.

Pop E , $\text{scan_left}(E)$ pushes $R := (3, 4)$, $\text{scan_right}(4)$ makes matching $\{E, 4\}$. Restart. $L \cup R = (A, B, C, D)$.

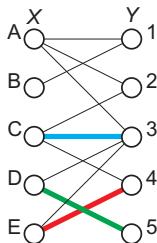
Pop D , $\text{scan_left}(D)$ pushes $R := (3, 5)$, $\text{scan_right}(5)$ makes matching $\{D, 5\}$. Restart. $L \cup R = (A, B, C)$.

After $\text{scan_left}(C)$ we have $L \cup R = (A, B, 2, 3, 4)$, $\text{scan_right}(4)$ pushes vertex E to L , $\text{scan_right}(E)$ goes deeper no more because 3,4 already have labels. $\text{scan_right}(3)$ adds $\{C, 3\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}\}$. Restart. $L \cup R = (A, B)$.

$\text{scan_left}(B)$ pushes vertex 1 to R , $\text{scan_right}(1)$ adds $\{B, 1\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}, \{B, 1\}\}$. Restart. $L \cup R = (A)$.

$\text{scan_left}(A)$ pushes into $L \cup R = (1, 2, 3)$. After $\text{scan_right}(3)$ we have $L \cup R = (1, 2, C)$. After $\text{scan_left}(C)$ we have $L \cup R = (1, 2, 4)$ because 2,3 already have labels. $\text{scan_right}(4)$ gives $L \cup R = (1, 2, E)$. $\text{scan_left}(E)$ is not going deeper any further because 3,4 have labels, we remain with $L \cup R = (1, 2)$. $\text{scan_right}(2)$ adds $\{A, 2\}$ to the matching. Restart. $L \cup R = \emptyset$. Finish.

Example



We assume that we choose from $L \cup R$ according to LIFO (stack) order, elements with smaller number or alphabetically earlier are inserted in L, R earlier.

Initially $L \cup R = (A, B, C, D, E)$.

Pop E , $\text{scan_left}(E)$ pushes $R := (3, 4)$, $\text{scan_right}(4)$ makes matching $\{E, 4\}$. Restart. $L \cup R = (A, B, C, D)$.

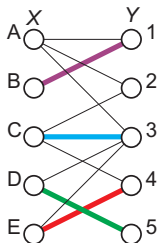
Pop D , $\text{scan_left}(D)$ pushes $R := (3, 5)$, $\text{scan_right}(5)$ makes matching $\{D, 5\}$. Restart. $L \cup R = (A, B, C)$.

After $\text{scan_left}(C)$ we have $L \cup R = (A, B, 2, 3, 4)$, $\text{scan_right}(4)$ pushes vertex E to L , $\text{scan_right}(E)$ goes deeper no more because 3,4 already have labels. $\text{scan_right}(3)$ adds $\{C, 3\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}\}$. Restart. $L \cup R = (A, B)$.

$\text{scan_left}(B)$ pushes vertex 1 to R , $\text{scan_right}(1)$ adds $\{B, 1\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}, \{B, 1\}\}$. Restart. $L \cup R = (A)$.

$\text{scan_left}(A)$ pushes into $L \cup R = (1, 2, 3)$. After $\text{scan_right}(3)$ we have $L \cup R = (1, 2, C)$. After $\text{scan_left}(C)$ we have $L \cup R = (1, 2, 4)$ because 2,3 already have labels. $\text{scan_right}(4)$ gives $L \cup R = (1, 2, E)$. $\text{scan_left}(E)$ is not going deeper any further because 3,4 have labels, we remain with $L \cup R = (1, 2)$. $\text{scan_right}(2)$ adds $\{A, 2\}$ to the matching. Restart. $L \cup R = \emptyset$. Finish.

Example



We assume that we choose from $L \cup R$ according to LIFO (stack) order, elements with smaller number or alphabetically earlier are inserted in L, R earlier.

Initially $L \cup R = (A, B, C, D, E)$.

Pop E , $\text{scan_left}(E)$ pushes $R := (3, 4)$, $\text{scan_right}(4)$ makes matching $\{E, 4\}$. Restart. $L \cup R = (A, B, C, D)$.

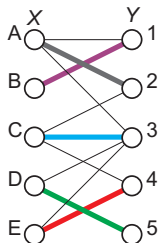
Pop D , $\text{scan_left}(D)$ pushes $R := (3, 5)$, $\text{scan_right}(5)$ makes matching $\{D, 5\}$. Restart. $L \cup R = (A, B, C)$.

After $\text{scan_left}(C)$ we have $L \cup R = (A, B, 2, 3, 4)$, $\text{scan_right}(4)$ pushes vertex E to L , $\text{scan_right}(E)$ goes deeper no more because 3,4 already have labels. $\text{scan_right}(3)$ adds $\{C, 3\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}\}$. Restart. $L \cup R = (A, B)$.

$\text{scan_left}(B)$ pushes vertex 1 to R , $\text{scan_right}(1)$ adds $\{B, 1\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}, \{B, 1\}\}$. Restart. $L \cup R = (A)$.

$\text{scan_left}(A)$ pushes into $L \cup R = (1, 2, 3)$. After $\text{scan_right}(3)$ we have $L \cup R = (1, 2, C)$. After $\text{scan_left}(C)$ we have $L \cup R = (1, 2, 4)$ because 2,3 already have labels. $\text{scan_right}(4)$ gives $L \cup R = (1, 2, E)$. $\text{scan_left}(E)$ is not going deeper any further because 3,4 have labels, we remain with $L \cup R = (1, 2)$. $\text{scan_right}(2)$ adds $\{A, 2\}$ to the matching. Restart. $L \cup R = \emptyset$. Finish.

Example



We assume that we choose from $L \cup R$ according to LIFO (stack) order, elements with smaller number or alphabetically earlier are inserted in L, R earlier.

Initially $L \cup R = (A, B, C, D, E)$.

Pop E , $\text{scan_left}(E)$ pushes $R := (3, 4)$, $\text{scan_right}(4)$ makes matching $\{E, 4\}$. Restart. $L \cup R = (A, B, C, D)$.

Pop D , $\text{scan_left}(D)$ pushes $R := (3, 5)$, $\text{scan_right}(5)$ makes matching $\{D, 5\}$. Restart. $L \cup R = (A, B, C)$.

After $\text{scan_left}(C)$ we have $L \cup R = (A, B, 2, 3, 4)$, $\text{scan_right}(4)$ pushes vertex E to L , $\text{scan_right}(E)$ goes deeper no more because 3,4 already have labels. $\text{scan_right}(3)$ adds $\{C, 3\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}\}$. Restart. $L \cup R = (A, B)$.

$\text{scan_left}(B)$ pushes vertex 1 to R , $\text{scan_right}(1)$ adds $\{B, 1\}$ to the matching. $M = \{\{E, 4\}, \{D, 5\}, \{C, 3\}, \{B, 1\}\}$. Restart. $L \cup R = (A)$.

$\text{scan_left}(A)$ pushes into $L \cup R = (1, 2, 3)$. After $\text{scan_right}(3)$ we have $L \cup R = (1, 2, C)$. After $\text{scan_left}(C)$ we have $L \cup R = (1, 2, 4)$ because 2,3 already have labels. $\text{scan_right}(4)$ gives $L \cup R = (1, 2, E)$. $\text{scan_left}(E)$ is not going deeper any further because 3,4 have labels, we remain with $L \cup R = (1, 2)$. $\text{scan_right}(2)$ adds $\{A, 2\}$ to the matching. Restart. $L \cup R = \emptyset$. Finish.

Complexity:

- Each augmenting path P is increasing by 1 cardinality of matching M .
- Therefore, there can be at most $|X| = n$ extensions of M .
- Calculating P requires scanning at most m edges.
- Hence, complexity is $O(nm)$.

Remarks

- Depending on the rules of drawing u from $L \cup R$, graph G is searched in DFS, BFS, or yet another order.
- In order to speed up the algorithm, it is advantageous to start with $M \neq \emptyset$, e.g. M calculated by some heuristics
 - Example heuristics 1: scan vertices $u \in X$ sequentially and add to M arbitrary admissible edges incident with u ;
 - Example heuristics 2: scan vertices $u \in X \cup Y$ in the order of increasing degrees and add to M arbitrary admissible edges incident with u .
- Observe similarity of a matching augmenting path and flow augmenting path in network flow algorithms (e.g. Dinic algo.).

Hopcroft-Karp Algorithm, 1973

- General idea: use layered networks to construct many parallel (mutually disjoint) augmenting paths.
- Let X_0, Y_0 be sets of free nodes in X and Y , respectively.
- Let $\Delta M = (P_1, \dots, P_k)$ be a maximal set of node-disjoint augmenting paths with resp. to M . *Maximal* means that there is no any other augmenting path P_x with resp. to M of the same minimum length⁹¹ which could be added to ΔM , though there can be some other set $(\Delta M)'$ which has more augmenting paths.

⁹¹ recall that layered networks comprise all shortest paths of the same length

Hopcroft_Karp algorithm(G, M)calculate X_0, Y_0 ;**while** $X_0 \neq \emptyset$ {

build_layered_network;

 construct ΔM ; $M := M \oplus \Delta M$; update X_0, Y_0 ;

}

```

build_layered_network {
 $L_0 := X_0; k^* := k := 0;$ 
while  $L_k \neq \emptyset$  {
    for each  $i \in L_k$  {
         $N(i) := \{j : \{i, j\} \in E - M, j \notin L_0 \cup L_1 \dots L_{k-1}\}$ 
    };
     $L_{k+1} := \cup_{i \in L_k} N(i);$ 
    if  $L_{k+1} = \emptyset$  { STOP!  $M$  is maximum };
    if  $L_{k+1} \cap Y_0 \neq \emptyset$  { // we are among free nodes on the  $Y$  side
         $k^* := k + 1;$  // it is possible to build augmenting paths of length  $k^* = k + 1$ 
         $L_{k+2} := \emptyset$  // return to the main function
    } else {
         $L_{k+2} := \{i : \{i, j\} \in M, j \in L_{k+1}\}$ 
    };
     $k := k + 2;$ 
} // end while
} // build_layered_network

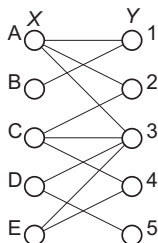
```

```

construct_ΔM { // builds maximal set ΔM of disjoint augmenting paths for M
remove from  $L_{k^*}$  covered vertices; // remains only  $L_{k^*} \cap Y_0$ 
mark all vertices in the layered network as unvisited;
 $k := 1$ ; // path length counter
while  $L_0 \neq \emptyset$  {
  choose  $x_0 \in L_0$  and remove it from  $L_0$ ;
   $\ell := 0$ ;
  while  $\ell \geq 0$  {
    while  $x_\ell$  has unvisited neighbor in  $L_{\ell+1}$  {
      choose an unvisited neighbor  $x_{\ell+1}$  and mark it as visited;
       $\ell := \ell + 1$ ;
      if  $\ell = k^*$  {
         $P_k = (x_0, x_1, \dots, x_{k^*})$ ; // since nodes in  $P_k$  are already scanned, they will not be
         $k := k + 1$ ; // used any more, so they can be removed from the layered network
      }; // end if
    }; // end while  $x_\ell$ 
    if  $\ell < k^*$  {  $\ell := \ell - 1$  } else {  $\ell := -1$  }; // backtrack á la DFS, or make another path
  }; // end while  $\ell \geq 0$  - one path
}; // end while  $L_0 \neq \emptyset$  - all paths
} // end of construct_ΔM

```

Example



It is assumed that elements are removed from sets L_k according to LIFO (stack) order. Elements with lower number, or earlier in the alphabet are considered first.

$X_0 = L_0 = (A, B, C, D, E)$, $Y_0 = Y$.

`build_layered_network` constructs $L_1 = (1, 2, 3, 4, 5)$, $k^* = 1$, $L_2 = \emptyset$.

According to the order of popping from L_1 `construct_ΔM` builds augmenting paths:

$P_1 = (E, 3)$, $P_2 = (D, 5)$, $P_3 = (C, 2)$, $P_4 = (B, 1)$.

In the main function we get matching

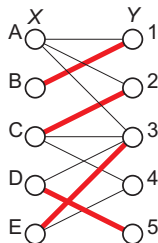
$M = \{\{E, 3\}, \{D, 5\}, \{C, 2\}, \{B, 1\}\}$ $X_0 = \{A\}$, $Y_0 = \{4\}$. Next iteration.

`build_layered_network` constructs $L_0 = (A)$, $L_1 = (1, 2, 3)$, $L_2 = (B, C, E)$, $L_3 = (4)$, $k^* = 3$. `construct_ΔM` builds augmenting path $P_1 = (A, 2, C, 4)$.

In the main function we get matching

$M = \{\{E, 3\}, \{D, 5\}, \{C, 4\}, \{B, 1\}, \{A, 2\}\}$.

Example



It is assumed that elements are removed from sets L_k according to LIFO (stack) order. Elements with lower number, or earlier in the alphabet are considered first.

$X_0 = L_0 = (A, B, C, D, E)$, $Y_0 = Y$.

`build_layered_network` constructs $L_1 = (1, 2, 3, 4, 5)$, $k^* = 1$, $L_2 = \emptyset$.

According to the order of popping from L_1 `construct_ΔM` builds augmenting paths:

$P_1 = (E, 3)$, $P_2 = (D, 5)$, $P_3 = (C, 2)$, $P_4 = (B, 1)$.

In the main function we get matching

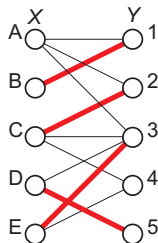
$M = \{\{E, 3\}, \{D, 5\}, \{C, 2\}, \{B, 1\}\}$ $X_0 = \{A\}$, $Y_0 = \{4\}$. Next iteration.

`build_layered_network` constructs $L_0 = (A)$, $L_1 = (1, 2, 3)$, $L_2 = (B, C, E)$, $L_3 = (4)$, $k^* = 3$. `construct_ΔM` builds augmenting path $P_1 = (A, 2, C, 4)$.

In the main function we get matching

$M = \{\{E, 3\}, \{D, 5\}, \{C, 4\}, \{B, 1\}, \{A, 2\}\}$.

Example



It is assumed that elements are removed from sets L_k according to LIFO (stack) order. Elements with lower number, or earlier in the alphabet are considered first.

$X_0 = L_0 = (A, B, C, D, E)$, $Y_0 = Y$.

build_layered_network constructs $L_1 = (1, 2, 3, 4, 5)$, $k^* = 1$, $L_2 = \emptyset$.

According to the order of popping from L_1 construct ΔM builds augmenting paths:

$P_1 = (E, 3)$, $P_2 = (D, 5)$, $P_3 = (C, 2)$, $P_4 = (B, 1)$.

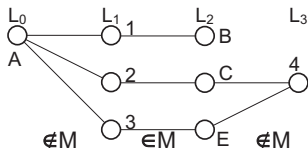
In the main function we get matching

$M = \{(E, 3), \{D, 5\}, \{C, 2\}, \{B, 1\}\}$ $X_0 = \{A\}$, $Y_0 = \{4\}$. Next iteration.

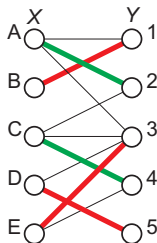
build_layered_network constructs $L_0 = (A)$, $L_1 = (1, 2, 3)$, $L_2 = (B, C, E)$, $L_3 = (4)$, $k^* = 3$. construct ΔM builds augmenting path $P_1 = (A, 2, C, 4)$.

In the main function we get matching

$M = \{(E, 3), \{D, 5\}, \{C, 4\}, \{B, 1\}, \{A, 2\}\}$.



Example



It is assumed that elements are removed from sets L_k according to LIFO (stack) order. Elements with lower number, or earlier in the alphabet are considered first.

$X_0 = L_0 = (A, B, C, D, E)$, $Y_0 = Y$.

`build_layered_network` constructs $L_1 = (1, 2, 3, 4, 5)$, $k^* = 1$, $L_2 = \emptyset$.

According to the order of popping from L_1 `construct_ΔM` builds augmenting paths:

$P_1 = (E, 3)$, $P_2 = (D, 5)$, $P_3 = (C, 2)$, $P_4 = (B, 1)$.

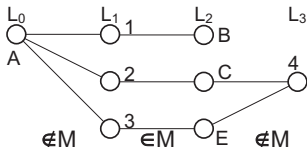
In the main function we get matching

$M = \{(E, 3), \{D, 5\}, \{C, 2\}, \{B, 1\}\}$ $X_0 = \{A\}$, $Y_0 = \{4\}$. Next iteration.

`build_layered_network` constructs $L_0 = (A)$, $L_1 = (1, 2, 3)$, $L_2 = (B, C, E)$, $L_3 = (4)$, $k^* = 3$. `construct_ΔM` builds augmenting path $P_1 = (A, 2, C, 4)$.

In the main function we get matching

$M = \{(E, 3), \{D, 5\}, \{C, 4\}, \{B, 1\}, \{A, 2\}\}$.



Theorem

Complexity of Hopcroft-Karp algorithm is $O(m\sqrt{n})$.

Proof (outline)

At most $O(m)$ edges are scanned in each iteration, so it is necessary to prove that the number of iterations is $O(\sqrt{n})$. Let M^* denote maximum matching. In each iteration the size of matching increases by at least 1. If $|M^*| \leq \sqrt{n}$, then we are done (proof finishes). Assume that $|M^*| > \sqrt{n}$. Then there is iteration k such that the size of matching achieved by k is $|M_k| < |M^*| - \sqrt{n}$, and after this iteration for the first time $|M_k| \geq |M^*| - \sqrt{n}$. Then in the following $\sqrt{n}$ iterations maximum matching is finalized (constructed). It is necessary to show that M_k is constructed in at most $O(\sqrt{n})$ iterations.

This can be achieved by demonstrating that layered network in iteration k has at most $2\sqrt{n} + 1$ layers, and therefore was constructed in at most $\sqrt{n}$ iterations, because in each iteration it extends by at least two layers. Symmetric difference $M_k \ominus M^*$ contains more than $\sqrt{n}$ vertex disjoint augmenting paths starting and ending in a free vertex in M_k (vertices to be added after iteration k). Not all of these paths can have length greater than $2\sqrt{n} + 1$ because each such path contains more than $\sqrt{n}$ vertices of set X . Since there are more than $\sqrt{n}$ paths this would mean that augmenting paths have more than $n = |X|$ vertices of X . Consequently, there is at least one path of length shorter than $2\sqrt{n} + 1$. Since the layered network comprises the shortest paths with respect to matching M_k , this matching has less than $2\sqrt{n} + 1$ layers. $\square$

BTW: if maximum matching is missing ℓ vertices from set X , then what length may have ℓ shortest augmenting paths starting in these nodes until there are no free nodes in X ?

Example application of matching in bipartite graphs

- Imagine a router with n inputs (ins) and s outputs (outs).
- There are packets in the input buffers waiting to be delivered to defined outputs: Packet $e = \{i, j\}$ is to be delivered from input i to output j . Packets are indivisible (are atomic units of data transfer).
- Packets compete for the outputs - only one packet can be output via a given output in one time unit.
- Represent inputs as set X , outputs as set Y , data transfer of the packets as set of edges E , then matching M in $G(X, Y, E)$ is the set of transfers which can be executed without a collision.

→example.

Graph coloring

Maciej Drozdowski

Institute of Computing Science, Poznań University of Technology

February 17, 2025

Coloring - definitions

- Let $G = (V, E)$ denote a graph without parallel edges⁹² or self loops, with set $V = \{v_1, v_2, \dots, v_n\}$ of nodes and set $E = \{e_1, e_2, \dots, e_m\}$ of edges.
- Two edges are *adjacent* in some node if they share this node.
Two vertices v_i, v_j are *neighboring* if there is an edge $e = \{v_i, v_j\} \in E$. Edge e is *incident* to v_i and to v_j , while v_i, v_j are adjacent to e .
- **Def. Node v degree** $\deg(v)$ is equal to the number of edges incident to v .
- **Def. Graph degree** is $\Delta = \max_{v \in V} \deg(v)$.

• **Def.** Given $V' \subseteq V$, $G(V') = (V', E \cap (V' \times V'))$ is a **subgraph induced** by V' .

• **Def.** $\overline{G} = (V, \overline{E})$ is a **complement** of G , if $\overline{E}$ satisfies:
 $\forall i \neq j : \{v_i, v_j\} \in E \Rightarrow \{v_i, v_j\} \notin \overline{E}, \{v_i, v_j\} \notin E \Rightarrow \{v_i, v_j\} \in \overline{E}.$ ⁹³

• **Def.** In **line graph** $L(G)$ of graph G edges of G are represented as nodes. Two nodes e_1, e_2 of $L(G)$ are neighboring, if the corresponding edges in G are adjacent in some node of G .

• **Def. Independent set** or **stable set** in $G = (V, E)$ is such set $S \subseteq V$ that no two nodes in S are neighbors.

- **Def. Maximum** independent set of G is the independent set of the biggest cardinality. $\alpha(G)$ will denote size of this set. $\alpha(G)$ is called an **independent number** of G .
- **Def. Maximal** independent set in G is an independent set which cannot be extended (made larger). In other words it is not embedded in any other stable set.

- **Def. Clique** (or a **complete** subgraph) in G , is a subgraph $G(K)$ such that every two distinct vertices in $K \subseteq V$ are adjacent.
- **Def. Maximum clique** in G is a clique with the largest number of vertices. $\omega(G)$ denotes the number of vertices in such a clique and is called *clique number* of G .

Note that clique $G(K)$ is an independent set in $\overline{G}$.

- **Def. Coloring** (or node coloring) of graph G is an assignment of colors $\{1, 2, \dots\}$ to nodes such that neighboring nodes have different colors. Formally, node coloring is function $f : V \rightarrow \{1, 2, \dots\}$ such that $\{v_i, v_j\} \in E \Rightarrow f(v_i) \neq f(v_j)$.
- Note that set of nodes with the same color is an independent set.
- **Def.** graph is **k -colorable** if it can be colored with at most k colors.
- **Def. Chromatic number** $\chi(G)$ is the minimum number k such that G is k -colorable.
- **Def. Graph coloring problem:**
(decision version) Instance data: G, k . Question: is G k -colorable?
(optimization version) Instance data: G . Request: Find $\chi(G)$.

- **Def. Edge coloring** of G is an assignment of colors $\{1, 2, \dots\}$ to edges such that adjacent to each other have different colors. Formally, edge coloring is a function $g : E \rightarrow \{1, 2, \dots\}$ such that $\forall e_1 = \{v_i, v_j\}, e_2 = \{v_i, v_l\} \in E \Rightarrow g(e_1) \neq g(e_2)$.
- **Def.** The minimum number of colors necessary to edge coloring of G is called **chromatic index** $\chi'(G)$ of G .
- Note that edge coloring of G is equivalent to node coloring of $L(G)$.
- What is the set of edges with the same color in edge coloring?

Applications with graph coloring

- Nonpreemptive unit execution time task scheduling for minimum schedule length

Tasks are represented as nodes in a graph. Edges represent conflicts in task resource requirements. This is so-called *conflict graph*. Color the conflict graph. Nodes with the same color can be executed in parallel (the same color $\rightarrow$ no connecting edges $\rightarrow$ no resource conflicts $\rightarrow$ can be executed together). Minimum number of colors determines schedule length.

- Designing electronic equipment test schedules

Electronic equipment is made of many modules/functional blocks which test each other, e.g. during power-on self test (POST).

Usually a number of such tests is performed on each block in pairs and subsets with other blocks but only one test at a time. These tests can be represented as nodes, edges connect tests (nodes) which cannot be executed in parallel (because they share tested blocks). One color determines the set of tests which can be executed simultaneously.

- By the same token graph coloring can be applied to schedule activities, meetings, timetables etc. Activities, meetings, transfers are tasks/nodes edges represent conflicts between them.

- Frequency band assignment

Radio transmitters, in their locations, can be represented as graph nodes. Possibility of interference can be represented by edges between the nodes (the transmitters). The minimum number of colors is the minimum number of frequency bands necessary for interference-free broadcasting.

- Printed circuit board (PCB) testing

PCBs support mechanically and connect electrically electronic components (e.g. integrated circuits, transistors, junctions, etc) using conductive tracks. There can be shorts between traces due to faulty PCB production. In the production process it is necessary to check if the tracks are mutually isolated electrically. Checking each pair of tracks is unacceptably time-consuming because there there can be as many as $n \approx 1\text{E}3\text{-}1\text{E}5$ different tracks on a contemporary PCB.

It is not necessary to check tracks which are far apart. Let G be a graph with nodes representing tracks and edges connecting nodes (the tracks) which are close enough for allowing a short. Color this graph. Nodes (the tracks) with the same color are far enough and will not short. For the time of testing, the tracks with the same color can be connected together and tested in pairs with tracks of a different color. This is less time-consuming because the number k of colors is usually in range $k = 5, \dots, 12$ colors.

BTW, currently electric probes are not attached directly to the tracks or special pads on PCBs. Necessary information about statuses, voltages can be read via JTAG (Joint Test Action Group) bus (IEEE Standard 1149.1-1990, boundary scan testing). See <https://en.wikipedia.org/wiki/JTAG>

- Assignment of registers to variables

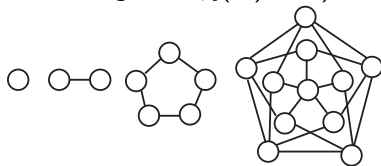
Holding program variables in CPU registers is advantageous for the performance of code execution. Usually there are fewer registers than variables, and hence, the registers have to be shared by variables which do not exist simultaneously. Let nodes in a graph represent variables, edges represent collisions between the variables. Two variables collide if one of them is used before and after using the other one (i.e. they are coded to exist simultaneously). Color this graph. Nodes=variables with the same color can be assigned without a collision to the same register. If the chromatic number is smaller than the number of registers, then it is possible to assign the variables collision-free to the registers. Otherwise, so called *spill code* must be used to allow using variables held in core memory (RAM). Nodes corresponding to variables moved to the core memory are removed from the graph until chromatic number is smaller than the number of registers. The graph of collisions is an *interval graph*.
→ example

Bounds on chromatic number and index

Node coloring - chromatic number

- $\chi(G) \geq \omega(G)$, because vertices of the largest clique must have different colors.

However, there are graphs with limited $\omega(G)$ but with arbitrarily large chromatic number. For example, Mycielski graphs (Mycielski graphs of order k : no triangles, o $\chi(G) = k$).



Mycielski graphs for $k = 1, 2, 3, 4$

- $\chi(G) \geq \lceil n/\alpha(G) \rceil$, because each color is an independent set.

However, there are graphs for which this bound can be arbitrarily weak: e.g. for Mycielski graphs $3 > n/\alpha(G)$.

- $\chi(G) \leq \Delta + 1$ – because this is the mode of operation of a greedy algorithm.
- $\chi(G) \leq \Delta$ – if G is not a clique and not a cycle of odd length [Brooks 1941].
- $\chi(G) = 2$ – for bipartite graphs.
- $\chi(G) = 4$ – for planar graphs [Appel, Haken, 1976] (maps).
- $\chi(G, g) = \lfloor \frac{7 + \sqrt{1 + 48g}}{2} \rfloor$ – for graphs embedded on a surface of genus g ($\approx$ number of holes).

[Heawood conjecture 1890, proved in 1960. (but not for planar graphs), except for Klein bottle].

Edge coloring - chromatic index

- $\chi'(G) \in \{\Delta, \Delta + 1\}$ [Vizing 1968]
- $\chi'(G) = \Delta$ for bipartite graphs [König 1916].

Home work:

Determine the complexity of verifying if the above conditions (both for node and edge coloring).

Determine how far the above bounds can get from the actual $\chi(G), \chi'(G)$.

Computational complexity of coloring

- Node coloring is NPc [Karp 1972] (decision version).
Coloring is NPc for $k \geq 3$ [Garey, Johnson, Stockmeyer 1976].
- If $P \neq NP$ then there is no approximation algorithm with performance ratio guarantee better than 2 [Garey, Johnson 1976].
- If $P \neq NP$ then finding coloring with $n^{1-\epsilon}\chi(G)$ colors is NPc, for any $\epsilon > 0$.
- There are algorithms with worst case performance guarantee (approximability) $O(n(\log \log n)^2/(\log n)^3)$, and $\min\{\Theta(\Delta^{1-2/k}), \Theta(n^{1-3/(k+1)})\}$.
- Edge coloring is NP-hard (actually NP-complete).
Observe that edge coloring problem is computationally hard despite having good lower and upper bounds on chromatic index.

Algorithms for graph coloring

Formulation as integer linear, or nonlinear, programming

- Remark: all integer linear programs (ILPs, IPs) can be solved using specialized software (based on B&B, heuristics, e.g. CPLEX, Gurobi, glpk, lp_solve).
- **F1.** Decision variables: $y_k = 1 \iff$ color k was used, $y_k = 0$ otherwise; $x_{ik} = 1 \iff$ node v_i has color k , $x_{ik} = 0$ otherwise.

$$\min. \quad \sum_{k=1}^n y_k \quad (3)$$

$$\text{s.t.} \quad \sum_{k=1}^n x_{ik} = 1 \quad \forall i = 1, \dots, n \quad (4)$$

$$x_{ik} + x_{jk} \leq 1 \quad \forall \{v_i, v_j\} \in E \quad (5)$$

$$y_k \geq x_{ik} \quad \forall i = 1, \dots, n, k = 1, \dots, n \quad (6)$$

$$y_k, x_{ik} \in \{0, 1\} \quad \forall i = 1, \dots, n, k = 1, \dots, n \quad (7)$$

• **F2.** Decision variables:

γ - # of used colors,

x_i - # of the color assigned to node v_i ,

$\delta_{ij} = 1 \iff x_j > x_i, \delta_{ij} = 0$ otherwise.

$$\text{min.} \quad \gamma \quad (8)$$

$$\text{s.t.} \quad x_i \leq \gamma \quad \forall i = 1, \dots, n \quad (9)$$

$$x_i - x_j - 1 \geq -n\delta_{ij} \quad \forall \{v_i, v_j\} \in E \quad (10)$$

$$x_j - x_i - 1 \geq -n(1 - \delta_{ij}) \quad \forall \{v_i, v_j\} \in E \quad (11)$$

$$\delta_{ij} \in \{0, 1\}, x_i \in \mathbb{N}^+ \quad \forall i = 1, \dots, n, k = 1, \dots, n \quad (12)$$

- **F3** Let $S_1, \dots, S_t$ denote all independent sets in G . The number t of independent sets grows exponentially with n in the worst case. Knowing $S_1, \dots, S_t$ it is possible to calculate $a_{ij} = 1 \iff v_i \in S_j$, $a_{ij} = 0$ otherwise.

Decision variables:

$s_j = 1 \iff S_j$ is chosen as some color, $s_j = 0$ otherwise.

$$\min. \quad \sum_{j=1}^t s_j \quad (13)$$

$$s.t. \quad \sum_{j=1}^t a_{ij}s_j = 1 \quad i = 1, \dots, n \quad (14)$$

$$s_j \in \{0, 1\}, \quad j = 1, \dots, t \quad (15)$$

- **F4** Decision variables:

$x_{ik} = 1 \iff$ vertex v_i has color k , $x_{ik} = 0$ otherwise.

Note that:

- ▷ Graph G is r -colorable if the goal function equals to 0.
- ▷ So formulation F4 is an oracle rather than a direct optimizer of the color number.
- ▷ The goal function (16) is nonlinear.

$$\min. \quad \sum_{k=1}^r \sum_{\{i,j\} \in E} x_{ik} x_{jk} \quad (16)$$

$$\text{s.t.} \quad \sum_{k=1}^r x_{ik} = 1 \quad i = 1, \dots, n \quad (17)$$

$$x_{ik} \in \{0, 1\} \quad i = 1, \dots, n, k = 1, \dots, r \quad (18)$$

Dedicated exact algorithms

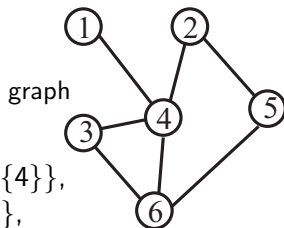
- [Christofides 75]

▷ If the considered graph is k -colorable, then some maximal independent set S_1 can be colored with color 1, some other maximal independent set S_2 in $G(V - S_1)$ can be colored with color 2, $\dots$, some independent set S_k in $G(V - \cup_{l=1}^{k-1} S_l)$ can be colored with color k . The challenge consists in determining the independent sets $S_1, \dots, S_k$.

▷ Let $\mathcal{T}_\ell$, $\ell = 1, 2, \dots$, denote a family (i.e. set of sets) of all independent sets which induce ℓ -colored subgraphs of G .

Christofides algorithm builds families $\mathcal{T}_0 = \{\emptyset\}, \mathcal{T}_1, \mathcal{T}_2, \dots$, until achieving $V \in \mathcal{T}_k$, then $k = \chi(G)$. Complexity: $O(mn2.445^n)$.

Coloring algorithm 1 [Christofides 75]
 $k := 0; \mathcal{T}_0 := \{\emptyset\}$
while $\mathcal{T}_k$ does not comprise a set covering whole V **do begin**
 $F := \emptyset$; **forall** $H \in \mathcal{T}_k$ **do**
forall independent set W in $G(V - H)$ **do** $F := F \cup \{G(H \cup W)\}$;

 $k := k + 1$; $\mathcal{T}_k :=$ maximal independent set in F ; **end**;
Example
 $\mathcal{T}_1 = \{\{1,2,3\}, \{1,2,6\}, \{1,3,5\}, \{4,5\}, \}$
 $\mathcal{T}_2 = \{ \{ \{1,2,3\}, \{4,5\} \}, \{ \{1,2,3\}, \{6\} \},$
 $\{ \{1,2,6\}, \{3,5\} \}, \{ \{1,2,6\}, \{4,5\} \},$
 $\{ \{1,3,5\}, \{2,6\} \}, \{ \{1,3,5\}, \{4\} \} \}$
 $\mathcal{T}_3 = \{ \{ \{1,2,3\}, \{4,5\}, \{6\} \}, \{ \{1,2,6\}, \{3,5\}, \{4\} \},$
 $\{ \{1,2,6\}, \{4,5\}, \{3\} \}, \{ \{1,3,5\}, \{2,6\}, \{4\} \}, \}$


- [Brown 72]

It is an implicit enumeration algorithm, interchangeably executing forward and backtracking phases.

▷ *Initial forward*: Assign numbers to vertices e.g. $(v_1, v_2, \dots, v_n)$, and color them in this sequence using minimum admissible colors. In this way the given graph is colored using UB colors.

▷ *backtracking*: by backtracking from vertex v_n find node v_j which can be feasibly colored with color $a < UB$ which was not previously used for v_j (i.e we traverse nodes and enumerate colors in each of the nodes). Color v_j using color a .

▷ *forward*: color vertices $v_{j+1}, \dots, v_n$, until:

a) vertex v_i is found which requires color $UB \Rightarrow$ stop forward phase, return to the last node which can be recolored;

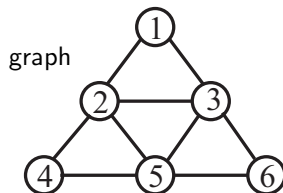
b) arriving in vertex $v_n \Rightarrow$ update UB and start backtracking.

▷ stop when v_1 is reached (in backtracking).

Example

vertex #	1	2	3	4	5	6
init. forward	1	2	3	1	4	1
backtrack	1	2	3	3	x	x
forward	1	2	3	3	1	2

x - immaterial color/vertex color invalidated

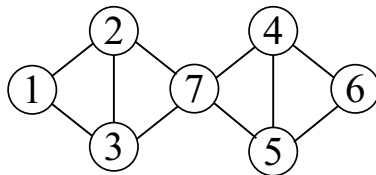


Remarks on Brown algorithm

- ▷ In the backtracking phase, while looking for a node which can be colored with fewer than UB colors, we can jump immediately to the first (smallest index) node which uses color number $\geq UB$.
- ▷ In the backtracking phase, when some node color is invalidated (x) also the list of checked colors is invalidated (erased).
- ▷ Note that Brown algorithm is an enumerative backtracking algorithm. As a consequence, if some node v_i color was changed, forward phase failed in some node v_k , but in the backtracking phase some other node v_j , where $i < j < k$ color can be changed to color $a < UB$ which was not previously used for v_j , then we backtrack to v_j first, not to v_i .

Home work:

Color this graph with Brown algorithm:



- [Brelaz 79, Korman 79]

Improvements in Brown algorithm:

- ▷ in choosing the vertex for assigning a different color (while backtracking) choose a vertex with minimum number of alternative colors.
 - ▷ as alternative color of vertex v_j choose the color which is infeasible for the greatest set of not colored vertices $v_{j+1}, \dots, v_n$.
- Exact algorithms routinely can solve to optimality graph coloring instances with 75-80 nodes (as of 1998). Update 2020 $\sim$ 90 nodes (not more than 100).

Dedicated greedy heuristics

- The greedy algorithms presented in the following are single-pass and the color assigned to some node is not changed.
- Assume that $K = (v_1, v_2, \dots, v_n)$ is some sequence of the vertices. Greedy algorithms operate according to the scheme:

`greedy_coloring( $G, K$ )`

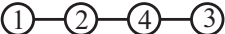
for $v := v_1$ **to** v_n **do** assign the smallest possible color to v ;

- Note that in each step of the above algorithm the color being assigned to the current vertex must be different than colors of at most Δ neighbors. Hence, greedy algorithm will use at most $\Delta + 1$ different colors to color any graph.

- **Def.** Graph G is **hard to color** for algorithm A if any implementation of algorithm A colors G non-optimally.
- **Def.** Graph G is **slightly hard to color** for algorithm A if there is some implementation of algorithm A which colors G non-optimally.
- Different implementations are possible because vertex sequencing rules may be incomplete.
- Greedy algorithms will be presented together with their hard/slightly hard to color graphs because approximation guarantees are weak.

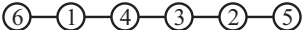
- Random Sequential (**RS**) algorithm: the order of vertices is random. The worst case performance ratio: $S_{RS} \geq O(n)$.⁹⁴

Graph hard to color does not exist.

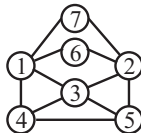
Minimum graph slightly hard to color: 

- Largest First (**LF**) algorithm [Welsh, Powell 1967]: vertices are ordered according to non-increasing degrees.

Rationale: Nodes with low degree have few neighbors and allow for more flexibility in choosing colors for them. Hence, they should be colored last. The worst case performance ratio: $S_{LF} \geq O(n)$.

Minimum graph slightly hard to color: 

Minimum graph hard to color:



⁹⁴ there are graphs for which $S_{RS}(G) \geq O(n)$

- Smallest Last (**SL**) algorithm, vertices are sequenced as follows:

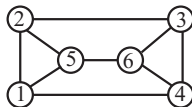
$K := \emptyset$; **while** $V - K \neq \emptyset$ **do**

{ add to K vertex v with minimum $\deg(v)$ in $G(V - K)$ };

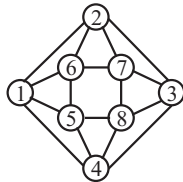
invert sequence of K ;

The worst case performance ratio: $S_{SL} \geq O(n)$.

Minimum graph slightly hard to color:



Minimum graph hard to color:



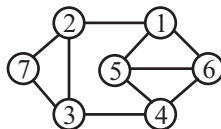
- Saturation Largest First, Dsatur (**SLF**) [Brelaz 1979].

Rationale: the number of different-colored neighbors decides about flexibility of coloring a vertex. *Saturation degree* $\rho(v)$ of node v is number of different colors used to color neighbors of v .

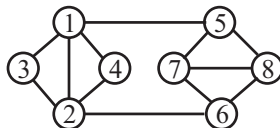
The nodes are sequenced online – color nodes from the largest $\rho(v)$ to the smallest, in case of a tie from the largest to the lowest node degree.

The worst case performance ratio: $S_{SLF} \geq O(n)$.

Minimum graph slightly hard to color:

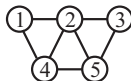


Minimum graph hard to color:

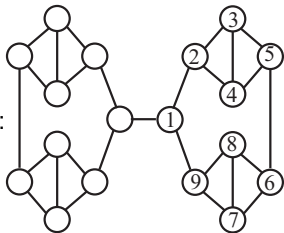


- Connected Sequential (**CS**) algorithm: sequence nodes such that each one (except the first) has at least one already colored neighbor. This can be, e.g., the order of DFS, or BFS in the graph. The worst case performance ratio: $S_{CS} \geq O(n)$.

Minimum graph slightly hard to color:



The smallest known graph hard to color:
(Minimum graph hard to color is not known.)



- Greedy Independent Set (**GIS**) algorithm [Johnson 74]

Rationale: this algorithm is greedily exploiting independent sets. Some node v_i gets color c if it is not colliding with other nodes of color c . After exhausting opportunities to color vertices with color c , nodes with this color are removed from the graph, and the remaining (not colored yet) nodes are colored with the next color.

The worst case performance ratio: $S_{GIS} \geq O(n / \log n)$.

$GIS_coloring(G)$

$c := 1;$

while there are color-less nodes in G **do**

begin

$H :=$ color-less nodes in G ;

while $H \neq \emptyset$ **do**

begin

$v :=$ the lowest-degree node in H ;

 color v with color c ;

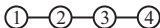
 remove v and its neighbors from H ;

end;

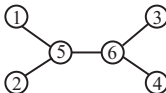
$c := c + 1$;

end;

Minimum graph slightly hard to color:



Minimum graph hard to color:



- Recursive Largest First (RLF) algorithm [Leighton 1979]

Rationale: operates like GIS, but chooses first node v which has the largest number of neighbors with forbidden color c because coloring v with color c will not change status of these neighbors.

The worst case performance ratio: $S_{RLF} \geq O(n/\log n)$.

RLF_coloring(G)

$c := 1$;

while there are color-less nodes in G **do**

begin

$H :=$ color-less nodes in G ;

$Z := \emptyset$; (* nodes which cannot get color c *)

while $H \neq \emptyset$ **do**

begin

$v :=$ a node with the largest degree in subgraph $H(\{v\} \cup Z)$, in case of a tie choose node with the smallest degree in H ;

 color v with color c ;

 remove v and its neighbors from H ;

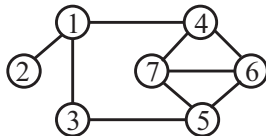
 add v and its neighbors v to Z ;

end;

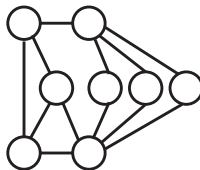
$c := c + 1$;

end;

Minimum graph slightly hard to color
for RLF (and 13 other)



Minimum graph hard to color
for RLF (and 3 other)



Edge coloring

- Arbitrary greedy algorithms have $S_{greedy} = 2$.
- Graph slightly hard to color for Random Sequential: P_5 (a path on 5 nodes).

Alternative coloring problems

- *Hypergraph coloring*. Hypergraph generalizes graphs by allowing edges (hyperedges) to connect more than two nodes. For example, in time-tabling applications a hyperedge may connect a student group, teacher and a laboratory (place). Color of such a hyperedge is time slot when the class is supposed to be held.
- *Online coloring*. Information about the graph to be colored is incomplete and is revealed (uncovered) while executing the coloring algorithm. An online adversary reveals the nodes in some sequence (possibly disadvantageous for the coloring algorithm) together with the adjacent edges. The revealed node must be colored before the next node is provided. The assigned colors are irreversibly.

- *Total coloring* = node+edge coloring, i.e. nodes and edges receive colors such that neighboring nodes have different colors, edges incident in the same node have different colors, nodes and edges adjacent to these nodes have different colors.
- *Weighted or fractional coloring* – nodes have weights $(w_1, w_2, \dots, w_n)$, w_i different colors are assigned to node v_i and neighboring nodes have different colors. This type of coloring corresponds with preemptive scheduling of conflict graphs (see applications of coloring) but with discretized preemption times.
- *Interval coloring* – is a variant of weighted coloring additionally requiring that the assigned colors make a compact interval. This type of coloring corresponds with non-preemptive scheduling of conflict graphs.

- *Summation coloring* – is a node coloring minimizing sum of used color numbers.
- *Strong coloring* – node coloring additionally requiring that colors of neighboring nodes and neighbors of the neighbors were different. This coloring version is used in frequency band assignment to strengthen band separation.
- *Contrasting coloring* – like strong coloring, but additionally distance between nodes with the same color must be at least 3, and neighboring nodes have color numbers differing by at least 2. Applicable in frequency assignment.

Cutting and packing

Maciej Drozdowski

Institute of Computing Science, Poznań University of Technology

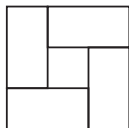
February 17, 2025

Introduction

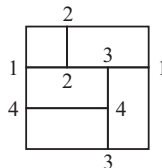
- Cutting and packing problems have many formulations. Generally this problem consists in placing smaller geometric objects in larger (target) areas of limited sizes.
- Fore example, given a set of geometric objects $O_1, \dots, O_n$ put them:
 - ▷ all in one large area as small as possible, e.g. on a strip of a given width but as short as possible;
 - ▷ all in one large area as small as possible, e.g. in rectangle of as small total area as possible (i.e. bounding box as small possible, rectangle packing);
 - ▷ put them all in as few as possible larger containers of limited size (bin packing).

- Further differences may pertain to:
 - ▷ number of geometric dimensions (1D,2D,3D),
 - ▷ objective function (e.g. minimizing area of used material for a given set of geometric objects $O_1, \dots, O_n$ vs maximizing value of selected geometric objects for given limited area(s)),
 - ▷ allowing object rotation (Yes/No),
 - ▷ allowing for holes in the larger (target) areas (Yes/No),
 - ▷ online/off-line operation.

▷ in 2D case allowing only guillotine cuts (Yes/No), i.e. edge-to-edge cuts parallel to the edges of the bin.



non-guillotine cuts



guillotine cuts

- BTW (curiosity), there are at least three different notations for cutting and packing problems.

Applications

- In all industries when something is cut from raw material: shipbuilding, paper (books, newspaper, carton/cardboard boxes), textile industries, gaskets, wood economy etc.
- In all industries when some geometric objects have to be laid out: architecture, factory floor planning, integrated circuits, GUIs etc.
- In all industries when something smaller is fit into larger areas, e.g. electrical engineering - how many round wires in a given cross-section? how many wood slabs in this log?
- web engineering - CSS-sprites.

Complexity

- Cutting and packing problems are **NP**-hard in general.
- 1D case:
 - ▷ - packing in 2 boxes is **NP**-hard by reduction to partition problem $\Rightarrow$ there is no approximation algorithm with the worst case performance guarantee smaller than $\frac{3}{2}$.
 - ▷ - packing in arbitrary number of boxes is strongly **NP**-hard by reduction to 3-partition problem
- These results can be extended to higher dimensions.

1D packing=bin packing problem

Bin packing problem – formulation

- There are bins (1D boxes) of size 1, potentially in unlimited number.
- There are n elements (segments) of sizes⁹⁵ $0 < a_i \leq 1$, for $i = 1, \dots, n$
- Assign elements to minimum number of bins, such that for set B_j of elements in bin j : $\sum_{a_i \in B_j} a_i \leq 1$.⁹⁶

⁹⁵ elements with $a_1 = 1$ are trivial in this sense that each of them needs one box, so they can be easily dealt with and removed from the further consideration

⁹⁶ i.e. elements are not sticking out of the bins

Approximation algorithms

- **Next Fit (NF)** algorithm

- ▷ consider elements in an arbitrary order
- ▷ assign elements to the current bin as long as possible, when some element a_i does not fit in the current bin b , open new bin $b + 1$, put a_i in this bin and assume bin $b + 1$ is the current bin.

Theorem

$$S_{NF} = 2 - \frac{1}{c^*},$$

where c^* is the optimum number of bins.

Proof. Let us denote by:

n - number of elements

k - number of bins used by NF

$f(i)$ - index of the box holding element i of size a_i

1. $OPT \geq \lceil \sum_{i=1}^n a_i \rceil$ - because number of bins results from sizes of elements, and this particular number of boxes would suffice if elements were divisible (could be split between boxes)

2. $\sum_{i:f(i) \in \{2j-1, 2j\}} a_i > 1$ for $j = 1, \dots, \lfloor \frac{k}{2} \rfloor$ - because otherwise bin $2j$ would not be opened after bin $2j-1$

3. $\sum_{i=1}^n a_i \geq \sum_{j=1}^{\lfloor \frac{k}{2} \rfloor} \sum_{i:f(i) \in \{2j-1, 2j\}} a_i > \sum_{j=1}^{\lfloor \frac{k}{2} \rfloor} 1 = \lfloor \frac{k}{2} \rfloor$

first inequality because the number of bins can be odd then there are more elements on the left side than on the right; second inequality from step 2.

4. $\frac{k-1}{2} \leq \lfloor \frac{k}{2} \rfloor$ - arithmetics, check even $k = 2\ell$ and for odd $k = 2\ell + 1$

5. $\lfloor \frac{k}{2} \rfloor \leq \lceil \sum_{i=1}^n a_i \rceil - 1$ - From point 3. because $\sum_{i=1}^n a_i$ is either fractional or integer.

6. $\frac{k-1}{2} \leq \lceil \sum_{i=1}^n a_i \rceil - 1$ - From 4. and 5.

7. $k \leq 2\lceil \sum_{i=1}^n a_i \rceil - 1 \leq 2OPT - 1$ - First inequality from 6., second from 1.

Instance achieving this ratio: $\bar{a} = [2\epsilon, 1 - \epsilon, 2\epsilon, 1 - \epsilon, \dots]$, $NF(I) = |\bar{a}| = n$, $OPT(I) = \lfloor \frac{n}{2} \rfloor + 1$ assume that $n = 2\ell + 1$ (n is odd) $\Rightarrow OPT(I) = \ell + 1 = (n - 1)/2 + 1 = NF(I)/2 + 1/2 \Rightarrow 2OPT(I) - 1 = NF(I)$

- **First Fit (FF)** algorithm

- ▷ assume there is set $OB = \{b_k, \dots, b_\ell\}$ of open bins (initially $k = \ell = 1$)
- ▷ consider elements a_i in an arbitrary order
- ▷ **if** (there is no bin capable of accommodating a_i)
 - {open a new bin j and put a_i in this bin;
 - if** ($a_i < 1$) {add bin j to set OB ; } }
- ▷ **else** {assign current element a_i to the first feasible bin j in OB };
- ▷ **if** (the current bin j is completely filled)
 - {remove bin j from set OB }.

- **Best Fit (BF)** algorithm

- ▷ assume there is set $OB = \{b_k, \dots, b_\ell\}$ of open bins (initially $k = \ell = 1$)
- ▷ consider elements a_i in an arbitrary order
- ▷ **if** (there is no bin capable of accommodating a_i)
 - {open a new bin j and put a_i in this bin;
 - if** ($a_i < 1$) {add bin j to set OB ; } }
- ▷ **else** {assign current element a_i to such bin $j \in OB$ that the space remaining in the bin is minimum in set OB };
- ▷ **if** (the current bin j is completely filled)
 - {remove bin j from set OB }.

Theorem

Absolute performance ratio for FF and BF is

$$S_{FF} = S_{BF} = \frac{17}{10} + \frac{2}{c^*},$$

where c^ - is the optimum number of bins.*

- **First Fit Decreasing (FFD)** algorithm

- ▷ Order elements according to decreasing⁹⁷ sizes and apply FF.

- **Best Fit Decreasing (BFD)** algorithm

- ▷ Order elements according to decreasing sizes and apply BF.

- Which of the algorithms: NF, BF, FF, BFD, NFD can be applied online and which cannot be?

- Worst case performance guarantees of FFD, BFD algorithms depend on the range $(0, a]$ of element sizes, i.e. $\forall a_i : a_i \in (0, \alpha]$.

Theorem

$$S_{FFD}^{\infty}(\alpha) = \begin{cases} \frac{11}{9} & \alpha \in (\frac{1}{2}, 1] \\ \frac{71}{60} & \alpha \in (\frac{8}{29}, \frac{1}{2}] \\ \frac{7}{6} & \alpha \in (\frac{1}{4}, \frac{8}{29}] \\ \frac{23}{20} & \alpha \in (\frac{1}{5}, \frac{1}{4}] \end{cases}$$

Theorem

$$S_{BFD}^{\infty}(\alpha) = \begin{cases} \frac{11}{9} & \alpha \in (\frac{1}{2}, 1] \\ \frac{71}{60} & \alpha \in (\frac{8}{29}, \frac{1}{2}] \\ \frac{7}{6} & \alpha \in (\frac{1}{4}, \frac{8}{29}] \\ 1 + \frac{k-2}{(k-1)k} & \alpha \leq \frac{1}{4} \end{cases} \text{ where }^{98} k = \frac{1}{\alpha}.$$

Exact algorithm

- It is assumed that all sizes a_i of the elements are in set $\{\frac{1}{p}, \frac{2}{p}, \dots, \frac{p-1}{p}, 1\}$.
- More generally, this method can be applied when set of element sizes is discrete and limited.
- Therefore, data of an instance can be represented as vector $\bar{v} = [v_1, \dots, v_p]$, where v_i is the number of elements with size $\frac{i}{p}$.
- The goal is to pack elements given in vector $\bar{v}$ using minimum number of unit size bins.
- We will call a **feasible set** a set of elements E satisfying condition $\sum_{i \in E} a_i \leq 1$, i.e. elements that fit together in a bin.

- Note that for a fixed number of different element sizes p the number of different feasible sets K is also constant and limited from above by $O(p^p)$.
- Each feasible set i can be represented as vector $\bar{b}_i$ where b_{ij} is the number of elements of size $\frac{j}{p}$.
- Note that solution of the problem is a decomposition of vector $\bar{v}$ representing multiplicity of elements of certain size into a set feasible sets with corresponding vectors $\bar{b}_1, \bar{b}_2, \dots, \bar{b}_p$, where certain feasible set/vector $\bar{b}_i$ is repeated α_i times.

- Hence, bin-packing problem can be formulated as an integer linear program:

$$\min \quad \sum_{i=1}^K \alpha_i \quad (19)$$

$$\text{s.t.} \quad \sum_{i=1}^K \alpha_i \bar{b}_i \geq \bar{v}, \quad (20)$$

$$\alpha_i \geq 0 \quad (21)$$

where:

- ▷ α_i are decision variables,
- ▷ (19) is the objective function counting number of used bins,
- ▷ constraint (20) guarantees that all given items are packed.

- Integer linear programming is an **NP**-hard in general, however, here the number of variables is limited. Integer linear programming with fixed (i.e. limited) number of variables is polynomial time solvable.
- In our case solving (19)-(21) is possible in "polynomial" time $O(2^{K^2} (Kp \log n)^{c^K}) < O(n)$ for fixed p , where K is $O(p^p)$ and c is some constant.
- Though formulation (19)-(21) is practically usable, its polynomial-time solvability is rather curiosity of polynomial algorithms than practical usability indication.

2D packing and cutting

Regular 2D packing/cutting problems

- In the case of "*regular*" 2D packing/cutting rectangles are placed in rectangular (target) area.⁹⁹
- Assume set R of n rectangles with sizes $(a_1, b_1), \dots, (a_n, b_n)$ are given.
- By convention let us assume that the first size (a_i) is horizontal, the second (b_i) is vertical.
- We assume that rotation of elements is not allowed.¹⁰⁰

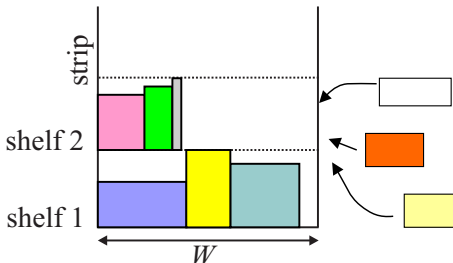
⁹⁹ Note that "*irregular*" 2D cutting/packing, i.e. involving arbitrary 2D shapes (polygons, circles, ellipses, etc.) is conceptually much harder because, e.g., it is hard to discretize element positions and even enumerate feasible solutions.

¹⁰⁰ This limitation is introduced for simplicity of exposition. Element rotation can be re-introduced by modification of the presented algorithms.

- **Def. 2D bin packing.** An unlimited number of identical 2D bins, each having width W and height H , is available. Pack rectangles from set R in as few bins as possible.
- **Def. Strip packing.** An infinite strip of width W is given. Pack rectangles from set R in as short (low) strip as possible.
Application: e.g. placing ads in web page column, newspaper classified advertising.
- **Def. Rectangle packing.** Pack rectangles from set R into the smallest area rectangle. Application: e.g. CSS-sprites.

Greedy algorithms for strip packing

- **Shelf algorithms** extend 1D bin packing methods to strip packing by putting the rectangles on shelves which are an equivalent of 1D bins.



- The elements on one shelf are aligned to the bottom of the shelf, left-shifted and must fit in strip horizontally.¹⁰¹
- Height of a shelf is set by the highest element on the shelf.

¹⁰¹like in 1D bins

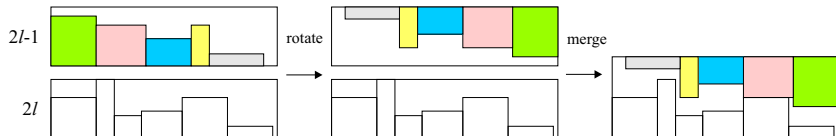
• Versions of the generic shelf algorithm differ in:

- ▷ (1) item sorting rule *ISR* → choose elements in the order of: increasing/decreasing widths a_i / heights b_i / areas $a_i b_i$ of the elements;
- ▷ (2) shelf choice rule *SCR* → use the shelf with: the smallest remaining horizontal space after inserting the item (Best Fit), the biggest remaining horizontal space after inserting the item (Worst Fit); current if possible, otherwise next shelf (Next Fit), the first shelf that fits (First Fit).
- ▷ use of extensions and modifications:

Fit Zero: if there is such shelf j that after inserting the current item will have at most ϵ free horizontal space, then choose shelf j ;

Fit two: if putting the current item i on shelf j would leave less horizontal space than required by the narrowest remaining item, then check if there are two remaining items k, ℓ , which both fit on shelf j but leave less remaining horizontal space than with i .

Rotation and merging: is a post-optimization executed after all items are put on the shelves. In this process pairs of neighboring shelves are rotated (one shelf by 180°) and merged by putting the rotated shelf on top of the other one.



Generic Greedy Shelf Packing

input: $TSR, SCR, F2, RM$, set R of items;

output: assignment c of items to shelves;

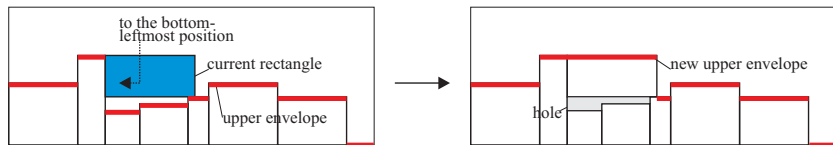
```

1:  $c = \text{NEW shelf assignment}$ ;           // initialize
2:  $L = \text{items in } \mathcal{R} \text{ sorted according to } TSR$ ; // order items
3: WHILE  $L \neq \emptyset$                  //  $\emptyset$  means here an empty list
3.1:  $t = \text{first item from } L$ ; remove  $t$  from  $L$ ;
3.2:  $s = \text{shelf for } t \text{ from } c \text{ according to } SCR$ ; // find best shelf for  $t$ , may include Fit Zero
3.3: IF  $s = \text{NIL}$ 
3.3.1:  $s = \text{NEW shelf}$ ;                // open a new shelf for  $t$ 
3.3.2:  $c = c + s$ ;                      // append the new shelf to assignment  $c$ 
3.4: ENDIF;
3.5: IF  $F2$                              // if option Fit Two is used, try substituting
3.5.1:  $s = s + \text{SUBSTITUTE}(t, s, L)$ ; // item  $t$  on shelf  $s$  with a better pair
3.6: ELSE
3.6.1:  $s = s + t$ ;                      // put item  $t$  on shelf  $s$ 
3.6: ENDIF;
4: ENDWHILE;
5: IF  $RM \text{ ROTATE\&MERGE}$ ;
6: RETURN  $c$ .

```

- Worst-case performance guarantees of certain versions of the shelf greedy algorithm:
 - ▷ Next Fit Decreasing Height (NFDH): $S_{NFDH}^{\infty} \leq 2$
 - ▷ First Fit Decreasing Height (FFDH): $S_{FFDH}^{\infty} \leq 1.7$

- **Bottom-Left algorithms** consider the rectangles in some item sorting rule *ISR* and place them as low as possible in a left-shifted position in the strip. In our implementation Bottom-left heuristics can be implemented to run in $O(n^2)$.



- Bottom-Left Decreasing Width has $S_{BLDF}^{\infty} \leq 2$.

Approximation algorithm for 2D bin packing

- **Hybrid First Fit, HFF:**

- ▷ Use FFDH shelf algorithm for strip packing.
- ▷ Assign the obtained levels to 2D-bins using FFD algorithm and level heights.

- $S_{HFF}^{\infty} \leq \frac{17}{8}$

